DynaChange Rules Guide

version 2021.2

Disclaimer

This document is for informational purposes only and is subject to change without notice. This document and its contents, including the viewpoints, dates and functional content expressed herein are believed to be accurate as of its date of publication. However, Epicor Software Corporation makes no guarantee, representations or warranties with regard to the enclosed information and specifically disclaims any applicable implied warranties, such as fitness for a particular purpose, merchantability, satisfactory quality or reasonable skill and care. As each user of Epicor software is likely to be unique in their requirements in the use of such software and their business processes, users of this document are always advised to discuss the content of this document with their Epicor account manager. All information contained herein is subject to change without notice and changes to this document since printing and other important information about the software product are made or published in release notes, and you are urged to obtain the current release notes for the software product. We welcome user comments and reserve the right to revise this publication and/or make improvements or changes to the products or programs described in this publication at any time, without notice.

The usage of any Epicor software shall be pursuant to an Epicor end user license agreement and the performance of any consulting services by Epicor personnel shall be pursuant to Epicor's standard services terms and conditions. Usage of the solution(s) described in this document with other Epicor software or third party products may require the purchase of licenses for such other products. Where any software is expressed to be compliant with local laws or requirements in this document, such compliance is not a warranty and is based solely on Epicor's current understanding of such laws and requirements. All laws and requirements are subject to varying interpretations as well as to change and accordingly Epicor cannot guarantee that the software will be compliant and up to date with such changes. All statements of platform and product compatibility in this document shall be considered individually in relation to the products referred to in the relevant statement, i.e., where any Epicor software is stated to be compatible with one product and also stated to be compatible with another product, it should not be interpreted that such Epicor software is compatible with both of the products running at the same time on the same platform or environment. Additionally platform or product compatibility may require the application of Epicor or third-party updates, patches and/or service packs and Epicor has no responsibility for compatibility issues which may be caused by updates, patches and/or service packs released by third parties after the date of publication of this document.

Epicor® is a registered trademark and/or trademark of Epicor Software Corporation in the United States, certain other countries and/or the EU. All other trademarks mentioned are the property of their respective owners.

Copyright © Epicor Software Corporation 2021.

All rights reserved. No part of this publication may be reproduced in any form without the prior written consent of Epicor Software Corporation.

Contents

About This Manual 8

DynaChange Rules Overview 9

- Glossary 9
- What Is a Business Rule? 10
  - Anatomy of a Business Rule 10
- What Business Rules Are Not 11
- Support 11

Setup 12

- Prerequisites 12
  - Required Software 12
  - Required Skills for Rules Developers 12
- Set Up DynaChange Rules 12
  - Run the Licensing Script 13
  - Set Up System Settings 13
  - Set Up Users 15
  - Test That the DynaChange Rules Setup Was Successful 15

Designing Business Rules 17

- Research the Business Need 17
- Describe the Business Need 18
- Define the Requirements 19
  - Example 19

A Developer's View of Business Rules 21

- Create a Visual Studio Project 22
  - Best Practice: One Rule Per Project 22
- Adding a New Rule Class 29
  - Creating the Required Class Methods 31
- Build and Deploy to Prophet 21 34
- Sample Rules 34

Developer's View of Web Visual Rules 35

- What Is a Web Visual Rule? 35
- Set Up Your Visual Studio Solution 36
- Create a New Rule 38
  - Add a New Area 38
  - Create a Model 38
  - Create a Controller 39
  - Create a View 40
  - Test Your Rule with the Test Harness 41

Connect Your Web Visual Rule with Prophet 21 43
Visual Response Attributes 45
What They Cannot Do 45
How They Work 45
Coding Callback Rules 46
Managing Business Rules 47
How Managing Business Rules Works 47
Types of Business Rule Links 48
Link Business Rules to Prophet 21 50
Link Business Rules to Imports 55
Link Event-Driven Rules to Prophet 21 56
Link Business Rules to User-Defined Menus 59
Link Business Rules to Custom Right Mouse Button Menu Options 63
Set Configuration Options 64
Manage Rule Permissions 71
Edit Business Rule Links 72
Remove Business Rules 75
Share Business Rules 77
Troubleshooting Rules 80
General Troubleshooting 80
Prophet 21 Troubleshooting 82
Developer Troubleshooting 85
Web Visual Rule Troubleshooting 88
Appendix A: Rule Class API 90
Rule Class 91
Properties 91
Methods 93
WebBusinessRule Class 93
RuleResult Class 94
Properties 94
ReposeAttributes 95
ResponseButton 95
ResponseField 96
RuleState Class 97
Properties 97
RulePopupService Class 98
Methods 99
DataField Class 99
Properties 99
LogString Class 102

Properties 103
Methods 103
DataCollection Class 104
Properties 105
Methods 106
DataFieldKey Class 108
Properties 108
DataFieldAttributes Class 109
Properties 109
Session Class 110
Properties 110
XML Datastream Class 111
Properties 111
Methods 111
Tips 113
Appendix B: Sample Rules 115
Sample Rule Project Overview 118
BranchSecurity 119
What This Example Shows 119
Use in Prophet 21 119
DLL File 119
CheckForRequired 119
What This Example Shows 119
Use in Prophet 21 120
DLL File 120
ConvertToCaps 120
What This Example Shows 120
Use in Prophet 21 120
DLL File 120
CountryHarmonizer 120
What This Example Shows 121
Use in Prophet 21 121
DLL File 121
CountryValidator 121
What This Example Shows 121
Use in Prophet 21 121
DLL File 121
NoFreightOnCollect 122
What This Example Shows 122
Use in Prophet 21 122
DLL File 122

PhoneNumberFormatterRule 122
What This Example Shows 122
Use in Prophet 21 122
DLL File 123
PopupServiceTest 123
What This Example Shows 123
Use in Prophet 21 123
DLL File 123
RestrictedItemList 123
What This Example Shows 123
Use in Prophet 21 124
DLL File 124
ZipCodeLookupYahooRule 124
What This Example Shows 124
Use in Prophet 21 124
DLL File 124
OrderCreditCheck 125
What This Example Shows 125
Use in Prophet 21 125
DLL File 125
OrderLineCreditCheck 125
What This Example Shows 125
Use in Prophet 21 125
DLL File 126
OrderUpdatedEvent 126
What This Example Shows 126
Use in Prophet 21 126
DLL File 126
AddNewRowForDiscount 126
What This Example Shows 126
Use in Prophet 21 127
DLL File 127
FormDatastream_SortInvoiceLines 127
What This Example Shows 127
Use in Prophet 21 127
DLL File 127
FormDatastream_AddGroup 128
What This Example Shows 128
Use in Prophet 21 128
DLL File 128
SampleRule1 128

What This Example Shows 128
Use in Prophet 21 128
DLL File 128
OEAddDiscount 129
What This Example Shows 129
Use in Prophet 21 129
DLL File 129
Appendix C: Special Cases 130
Cascading Rules 130
Adding Rows to Multi-Row Rules 131
Form Datastream Created Rule Event 132
Pre-SQL Hooks 133
Simulating Keystrokes 134

About This Manual

The DynaChange Rules Guide helps you understand how to integrate with Prophet 21 DynaChange™ business rules you develop (or rules developed by Epicor or a third party).

This manual includes:

- Setting up Prophet 21 to use business rules
- Creating a "stub" project and business rule class in Visual Studio
- Linking the DynaChange business rules into the Prophet 21 user interface
- Brief overviews of the included Epicor business rules

This manual does *not* include:

- Reference or how-to information on programming the business rule code algorithms
- Help for novice programmers

The Visual Studio screen shots and instructions in this manual use Visual Studio Express 2013 for Windows Desktop. Instructions and screens may be slightly different for Visual Studio's Pro versions, and for other versions of Visual Express. Also, because Visual Studio configuration varies depending on each individual user's settings, these screen shots may not show the same tools you see on your screen.

DynaChange Rules Overview

The Prophet 21 DynaChange Rules enables you to truly extend and customize the system to fit your business perfectly. You can write algorithms using the programming language of your choice, and link those algorithms into the Prophet 21 user interface. For example, you could write a business rule that takes the postal code you enter for an address, looks up the city and state for you, and populates them in the window automatically.

You can create business rules that apply:

- When you tab out of a field
- When you save a record
- When you click a button
- When you open a window
- Before or after Prophet 21 validates data
- In specific business scenarios when an event occurs in Prophet 21.

Glossary

This guide contains a lot of language that may be unfamiliar to many Prophet 21 users; because the Business Rule Engine is a method to add your own programming logic to Prophet 21, programming concepts and language is critical to understanding the engine. The following section addresses some of the terminology used in this manual.

Term:	Definition:
Algorithm	Programming code that gets business work done. For example, if your business rule populates the city and state based on the postal code, the algorithm is the code that looks up the city and state for the postal code and returns those values.
Business Rule	See "What Is a Business Rule?" on the next page for more information.
C#	The programming language used to create the business rule classes. Pronounced "see sharp."
Class	A set of algorithms that work together and, typically, represent some object (such as a business rule).

Term:	Definition:
Class Library	A set of classes that turn into a DLL file when you compile them. The DLL can then be used to access the algorithms inside the classes. Your business rules exist inside class libraries.
Developer	A person who creates software, such as a programmer or software engineer.
Logic	See *Algorithm* above.
Method	In programming, the function that is used to access an algorithm.
Object	A combination of programming code and data that represents something, typically something in the world (like a business rule, a person, a customer, etc.).
Web Visual Rule	A web visual rule is a rule in Prophet 21 that needs to display some data or window or web page outside of application fields.
ASP.NET MVC	Web development methodology used to create visual rules in the web application.

What Is a Business Rule?

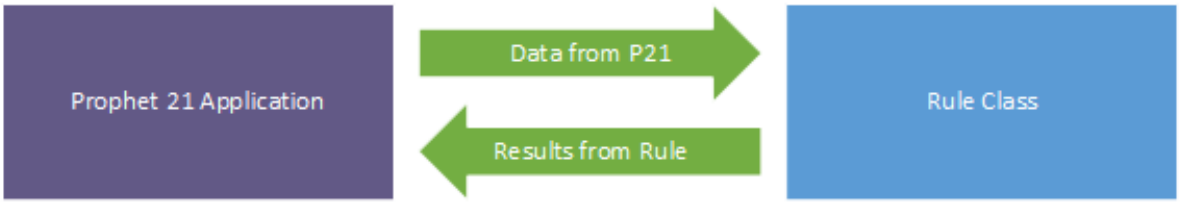
A business rule is a set of logic (programming code) that does some work that serves a business purpose in the system. You can write your own business rules which help your organization streamline processes, prevent errors, and expand capabilities. A business rule can be as simple as requiring a minimum number of characters in a field, or as complicated as creating your own window that will retrieve data from and insert data into your database.

Anatomy of a Business Rule

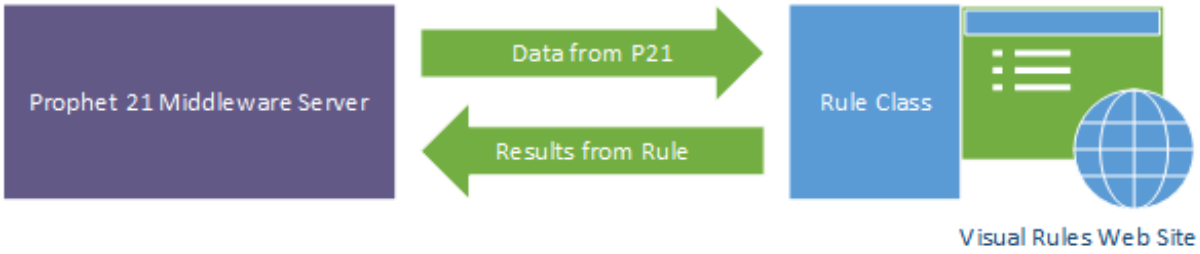
A business rule has several parts:

- "Hooks" in theProphet 21 application that enable the business rule and Prophet 21 to communicate and pass data back and forth
- The business rule class that contains the attributes of your business rule, including the algorithms of your rule that gets your business work done.
- In the case of web visual rules, the rule class is embedded in an asp.net MVC web site

DynaChange Rules



DynaChange Rules – Web Visual Rules



A business rule class is a C# class derived from the Rule class provided by Epicor.

What Business Rules Are Not

While business rules are very flexible and can be used in a lot of ways, there are some things business rules are not, and cannot do.

Business rules are not:

- Able to create a new tab in an existing window
- Able to change existing Prophet 21 logic; they may layer on top of what the system does only

Support

Epicor Software Support supports the Prophet 21 user interface elements of the DynaChange Rules and the link between the Visual Studio business rule class and Prophet 21. Software Support does not support problems in Visual Studio or business rule code or algorithms.

Setup

This chapter explains how to set up your system to use DynaChange Rules.

Prerequisites

Before you can set up the DynaChange Rules and create business rules, you must satisfy the following prerequisites.

Required Software

- Visual Studio 2015 or better
  You can use the C# Express versions of Visual Studio, but they lack all of the features, tools, and flexibility of Visual Studio Pro or better versions. You can use the following express versions:
  - Visual Studio Express 2015 for Windows Desktop or better
- .NET Framework 4.5 or later on all client PCs that run Prophet 21
- DynaChange Rules software (contact your Epicor salesrep to purchase)
- Web visual rules require asp.net MVC version 5
- Web visual rules require Prophet 21, version 2017.2.3063 or later

Required Skills for Rules Developers

- Basic understanding of Visual Studio
- Basic understanding of the C# programming language
- Working knowledge of the programming language in which you intend to implement your business logic algorithms
- Working knowledge of asp.net MVC for web visual rules

Set Up DynaChange Rules

To set up DynaChange Rules on your system, you must:

- Run the DynaChange Rules licensing script
- Specify a folder for the rules in your database
- Set up users with permission to link and manage rules

- Add sample rules to the DLL folder (optional)
- Test setup

Best Practice: Set Up and Test DynaChange Rules in Play System

Before you setup and use DynaChange Rules in your live system, Epicor recommends setting up DynaChange Rules and testing it on your play system.

Once you have set up and tested DynaChange Rules on your play system, you can run the setup again on your live system. If there are any rules on your play system that you want to transfer from play to live, rather than re-linking the rules in the live system, you can export rules from the play system and import them into the live system. See "Share Business Rules" on page 77 for more information.

Run the Licensing Script

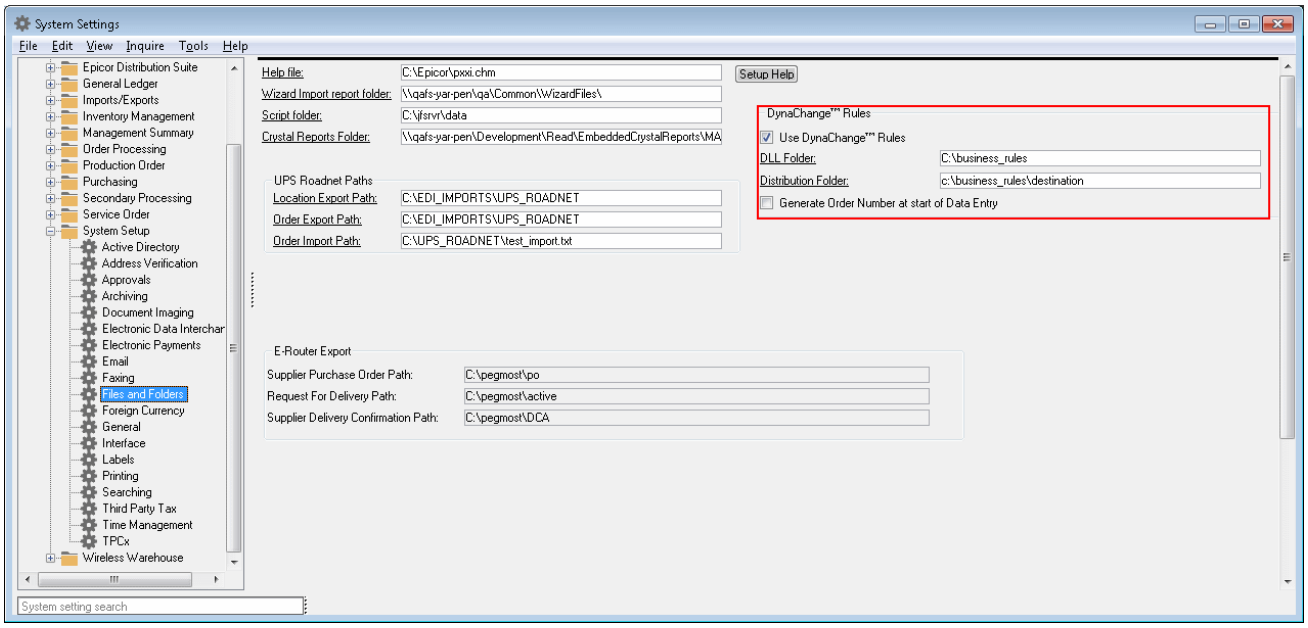
After you purchase DynaChange Rules, you receive a licensing script via email. You must run this script using SQL32.exe. For more information about running scripts, see Support Solution #4022 on the Customer Web Site.

Make sure you exit Prophet 21 prior to running the script.

Set Up System Settings

There are two system settings you must set up to use DynaChange Rules: enable DynaChange Rules, and set up the rules folder. The rules folder is the folder where Prophet 21 can find both your business rules, and the DLL it needs to connect to them. The DLL it needs is provided by the installer you run below.

Navigation Path: Setup > System Setup > System > System Settings > System Setup folder > Files and Folders node



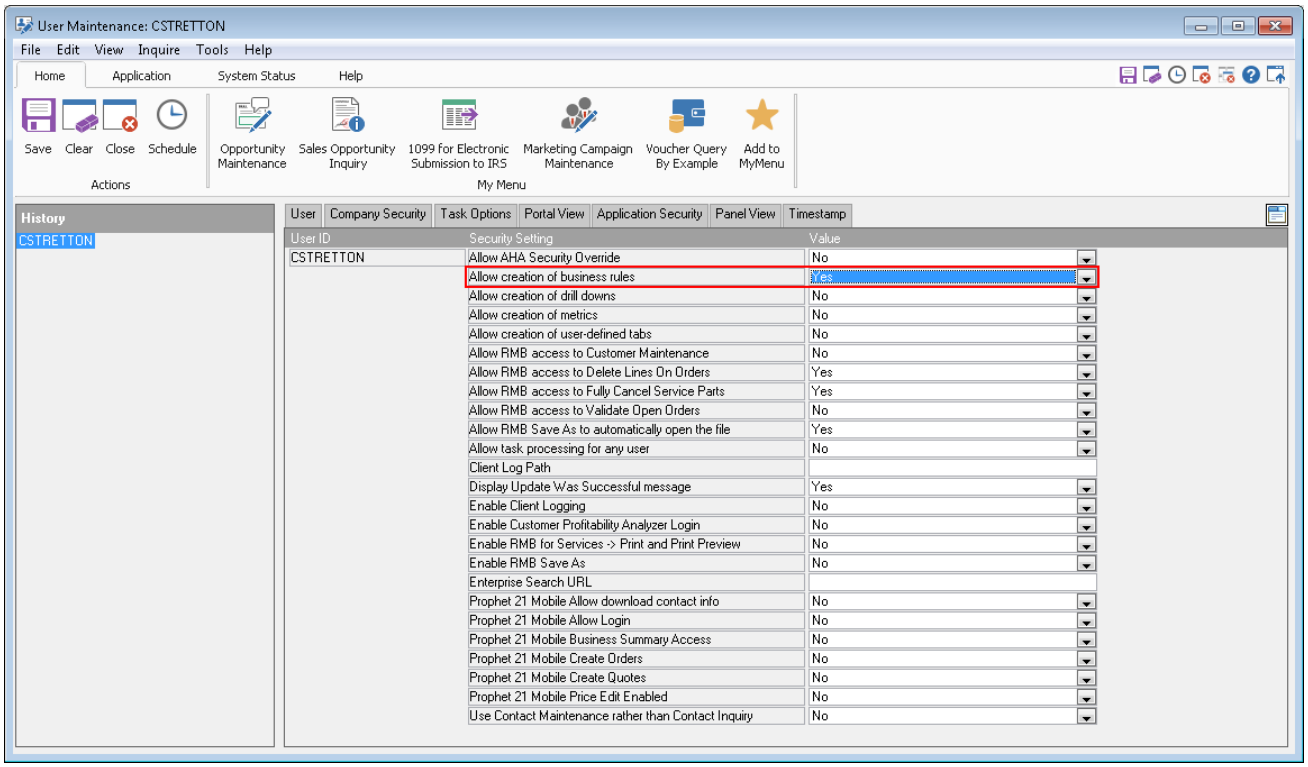
Field:	Description:
Use DynaChange Rules	Determines if they system is currently using any DynaChange Rules functionality. You must enable this check box before DynaChange Rules becomes active. • **On** - You can add and organize business rules, and the rules will trigger off of selected fields. • **Off** - You cannot add or organize business rules, and no rules will trigger.
DLL Folder	You must specify the DLL folder path; Prophet 21 looks there for business rules. This field is required. 1) Every Prophet 21 client (including your Middleware servers) must have access to this folder. 2) This folder must not be a UNC path. If you need to use a network location, you must use a mapped drive letter. If you do use a mapped network drive, make sure you add AllowPartiallyTrustedCallers to your business rules (see "Create a Visual Studio Project" on page 22 for more information). If a business rule references other assemblies (other DLLs), those

Field:	Description:
	assemblies must be in same DLL folder as business rules.
Distribution Folder	The folder into which the system deposits business rules that you export. See "Share Business Rules" on page 77 for more information.

Set Up Users

Before a user can actually create or apply rules in Prophet 21, you must grant the user permission to do so in User Maintenance.

Navigation Path: Setup > System Setup > System > User Maintenance



On the Application Security tab, set the *Allow Creation of Business Rules* security setting to **Yes** to enable a user to link and manage business rules in Prophet 21.

Test That the DynaChange Rules Setup Was Successful

Before you try to create business rules of your own, you should test with sample rules to ensure that the setup was successful.

Add Sample Rules DLL to DLL Folder

To test with the sample rules, you must copy the sample rules DLL files to the DLL folder you specified above. Using Windows Explorer, browse to the location where you installed Prophet 21. In that folder, you will find two files: **Activant.P21.Extensions.Examples.dll** and **P21.Extensions.Examples.dll**. Copy these files to the location you specified in the DLL folder, and you will be able to apply the example rules within Prophet 21. See "Appendix B: Sample Rules" on page 115 for more information on the sample rules.

Link ConvertToCaps Rule in Terms Maintenance

In Terms Maintenance, right-click the Terms Description field and select Services > DynaChange > DynaChange Rules > Create Business Rule. For the rule named ConvertToCaps, select Converter as the rule type. Click Save. See "Link Business Rules to Prophet 21" on page 50 for more information on linking a rule to Prophet 21.

In the Terms Description field, type "abcd" (without quotes) and tab off of the field. The field should now contain "ABCD" (in capital letters). If it does not, there has been some problem with the setup process, see "Troubleshooting Rules" on page 80 to try to resolve the problem. If you cannot resolve the problem, contact Software Support.

Once you have verified the setup works, go to Application > Rules, select the ConvertToCaps rule you just created, and click Delete Selected Rule. See "Remove Business Rules" on page 75 for more information.

Designing Business Rules

Before you create a business rule, it is best to design it. This chapter explains one simple method for designing business rules. Understanding how to design a business rule is especially critical if you outsource your programming efforts, as you must communicate your requirements clearly to someone outside your organization.

While you can create and code a business rule without performing any explicit design up-front (in effect, designing as you develop), investing some time and effort to create at least a basic design makes it much easier to develop the rules later (or, to hand the design off to an outside developer if you are not developing the rules yourself). Approaching design carefully helps you avoid spending money on lots of changes to a rule after it is finished because the requirements were not clear at the start.

There are many methods and philosophies for designing software, and it is beyond the scope of this manual to explore these; instead, this chapter presents one simple method that should be effective for most business rules.

1. Research the business need with stakeholders
2. Describe what business need you want the software to address
3. From that description, create a set of requirements

Research the Business Need **17**

Describe the Business Need **18**

Define the Requirements **19**

Example 19

Research the Business Need

The first step in designing a business rule is to understand the business need. To do so, you must discuss the subject with stakeholders involved, interview end users, and possibly watch them perform the task in question to ensure you understand the issue as fully as you reasonably can. The key things to get from the stakeholders:

- **The context surrounding the problem or opportunity.** This context may be critical to understanding the best way to address the business need. For example, if you want to increase efficiency in customer service, it helps to understand the entire process currently used to service

customers. Otherwise, you may create a business rule that does not work because of some process that occurs later, or because of an influencing factor you did not account for initially.

- **The problem or opportunity itself.** For example, you want to speed up the process of identifying the customer when they call.
- **The business value of the rule.** Business value is important because it can affect how you design and implement the rule. The need and the value are frequently interrelated. For example, the value of sought in the examples above is to make customer service more efficient, so whatever means you use to speed up identifying the customer must overall contribute to more efficient customer service. If the method you use to speed up customer identification does not result in more efficient service, it's not worth doing.

Describe the Business Need

The next step to designing a business rule is to state the business need. This may be as simple as a couple of sentences for a very simple business rule, or it may be several paragraphs for a more complicated rule. It does not have to be (and indeed, should not be) complete to the last detail; the objective is to make the basic business need clear so that you can create requirements. The best way to make sure you understand it is to articulate it and, ideally, share that description with any stakeholders, to ensure that it meets their approval.

A good description typically includes:

- Who will use the rule
- What business rule needs to be implemented (*without* explaining how to implement it)
- The goal of the business rule (its value to your business)

One simple format to use is "*As a <job_description>, I want to <do something> so that <business_value>*."

The following is an example of a simple business need description. For a complicated business need, you might want to separate the context, need, and value, stating each in its own section.

Example

Salesreps currently discount order lines too deeply. As a sales manager, I want to limit salesreps to a minimum profit percent so that we improve our ability to forecast profit, set goals, and execute against our plan.

The description starts with the context (salesreps are discounting orders too heavily). Notice that the business description does not necessarily specify how to limit discounting; it merely identifies the need to do so. Finally, the description identifies the value of the business rule to the organization.

Now that we have described the business need, we must turn that need into requirements.

Define the Requirements

The requirements define how the software must change to meet the business need. You want to examine the business need and determine how you can meet the need and fulfill the goals. After you create the requirement, it is a good idea to share them with the stakeholders in your project, to ensure that the requirements are complete and satisfactory.

Typically, you want to keep your requirements only as detailed as they have to be; for example, if you need to store data, you may not.

The following example shows how you might create requirements from the business need in the previous example.

Example

Armed with our business need, we create our requirements.

1. Store the minimum profit % in the database.
2. Create a business rule that gets the minimum profit % from the database and compares it to the profit % from the order line.
    - If the profit % is less than the minimum, do not let the order save.
    - If the profit % is less than the minimum, display a message letting the user know that the order will not save until profit % is at least the minimum %.
3. Add the business rule to the Unit Price and Extended Price fields in on the Items tab in Order Entry and Front Counter Order.

Now we share these requirements with our sales management team, and the managers say that they do not want a single hard-coded value; they want to be able to specify the minimum on a per-salesrep basis. So we must modify our business need and requirements.

Revised Business Need

Salesreps currently discount order lines too deeply. As a sales manager, I want to limit salesreps to a minimum profit percent so that we improve our ability to forecast profit, set goals, and execute against our plan. Each salesrep may have a different minimum limit.

Revised Requirements

1. Store records for the user ID and minimum profit % in the database.
2. Provide a method for sales managers to edit the minimum profit % for each salesrep.

3. Create a business rule that gets the minimum profit % for the current user from the database and compares it to the profit % from the order line.
    - If the profit % is less than the minimum, do not let the order save.
    - If the profit % is less than the minimum, display a message letting the user know that the order will not save until profit % is at least the minimum %.
4. Add the business rule to the Unit Price and Extended Price fields in on the Items tab in Order Entry and Front Counter Order.

At this point, we are ready to begin programming our business rule.

A Developer's View of Business Rules

This chapter helps developers hook the business rule algorithms they create into Prophet 21's DynaChange Rules (which will display them for users to select in Prophet 21). This chapter covers “standard” non-web visual rules. In the next chapter, A Developer’s View of Web Visual Rules, you will learn how to create web visual rules. In a subsequent chapter, *Managing Business R"Appendix A: Rule Class API" on page 90*, you will learn how to manage business rules you create, and link them to the Prophet 21 user interface.

> The Visual Studio screen shots and instructions in this manual use Visual Studio Express 2013 for Windows Desktop. Instructions and screens may be slightly different for other versions of Visual Studio. Also, because Visual Studio configuration varies depending on each individual user's settings, these screen shots may not show the same tools you see on your screen.

To hook your algorithms into DynaChange Rules in Prophet 21, you must:

- Create a library project in Visual Studio
- Add a new class to the project; the class must be derived from the Rule class
- Populate the three required methods for the class
- Deploy the library file (the DLL file) to the proper location so Prophet 21 can find and use it

This chapter shows some code, but it does not explain the purpose of most of the code. It explains only the aspects of the code that are key parts of the DynaChange Rules.

To fully understand the parts of the Rule class and its public methods and fields, see the API for the DynaChange Rules, see "Appendix A: Rule Class API" on page 90 for more information. It is a good idea to print out the API and have it on hand for reference as you build your business rules.

Create a Visual Studio Project **22**

Best Practice: One Rule Per Project 22

Adding a New Rule Class **29**

Creating the Required Class Methods 31

Build and Deploy to Prophet 21 ..... **34**

Sample Rules **34**

Create a Visual Studio Project

> The Visual Studio screen shots and instructions in this manual use Visual Studio Express 2013 for Windows Desktop. Instructions and screens may be slightly different for Visual Studio's Pro versions, and for other versions of Visual Express. Also, because Visual Studio configuration varies depending on each individual user's settings, these screen shots may not show the same tools you see on your screen.

To create and store your business rule classes, you need to create projects in Visual Studio. Project enables you to build your rules into one or more DLL files that you can access from Prophet 21. While Prophet 21 can use multiple DLLs from multiple Visual Studio projects; it is simplest to use a single solution to store many projects for business rules. See "Managing Business Rules" on page 47 for more information on organizing your rules and projects.

Best Practice: One Rule Per Project

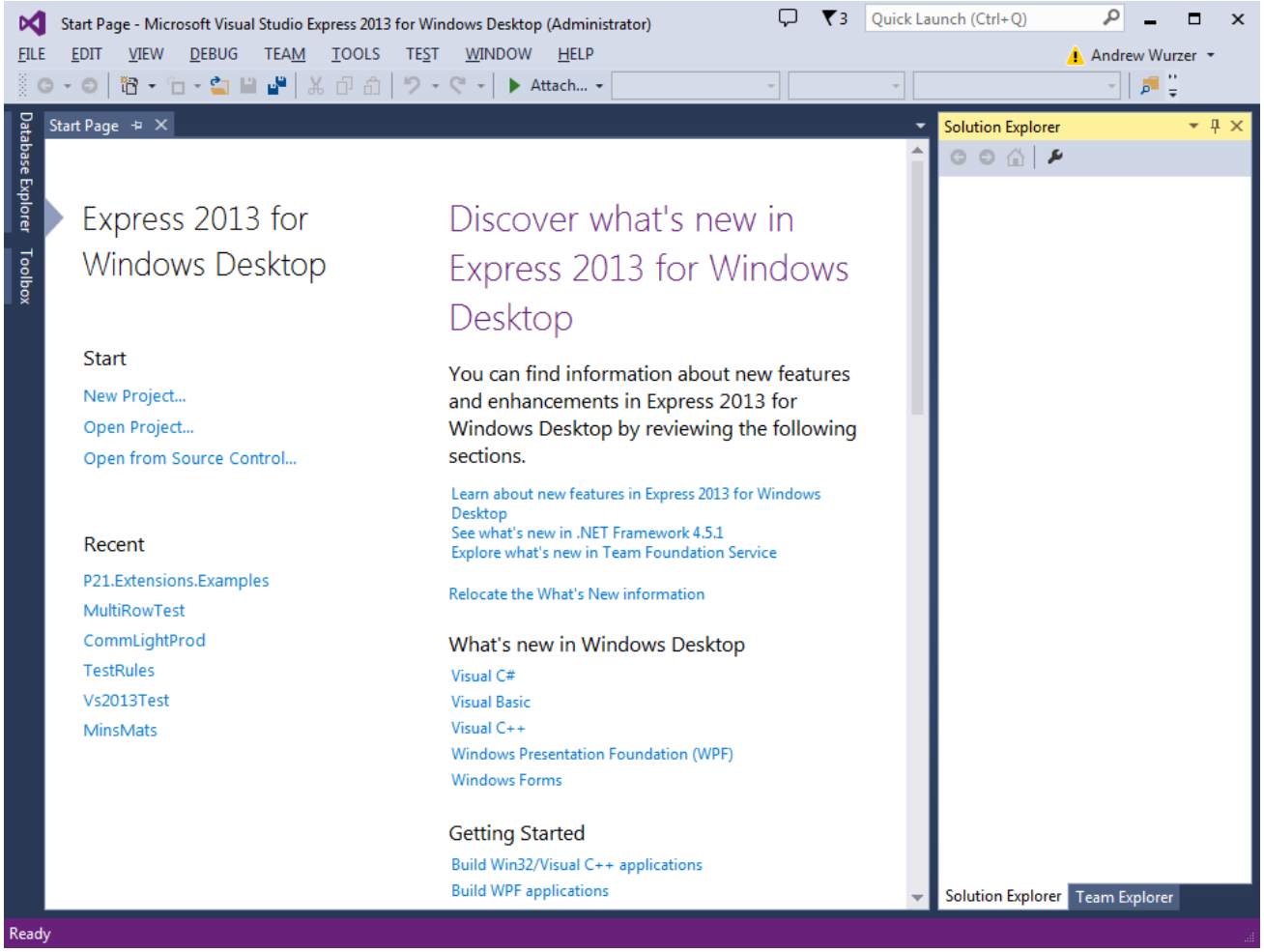
In many cases, you may want to create one project for every business rule. This is a good practice because you can develop and deploy business rules separately or together, depending on your needs. While you can have multiple business rules in a single project by creating many Rule classes in the project, the drawback to this is that you cannot build the classes in one project separately; when you build the project, all the classes are built.

There are two problems that can arise if you group many rules into a single project:

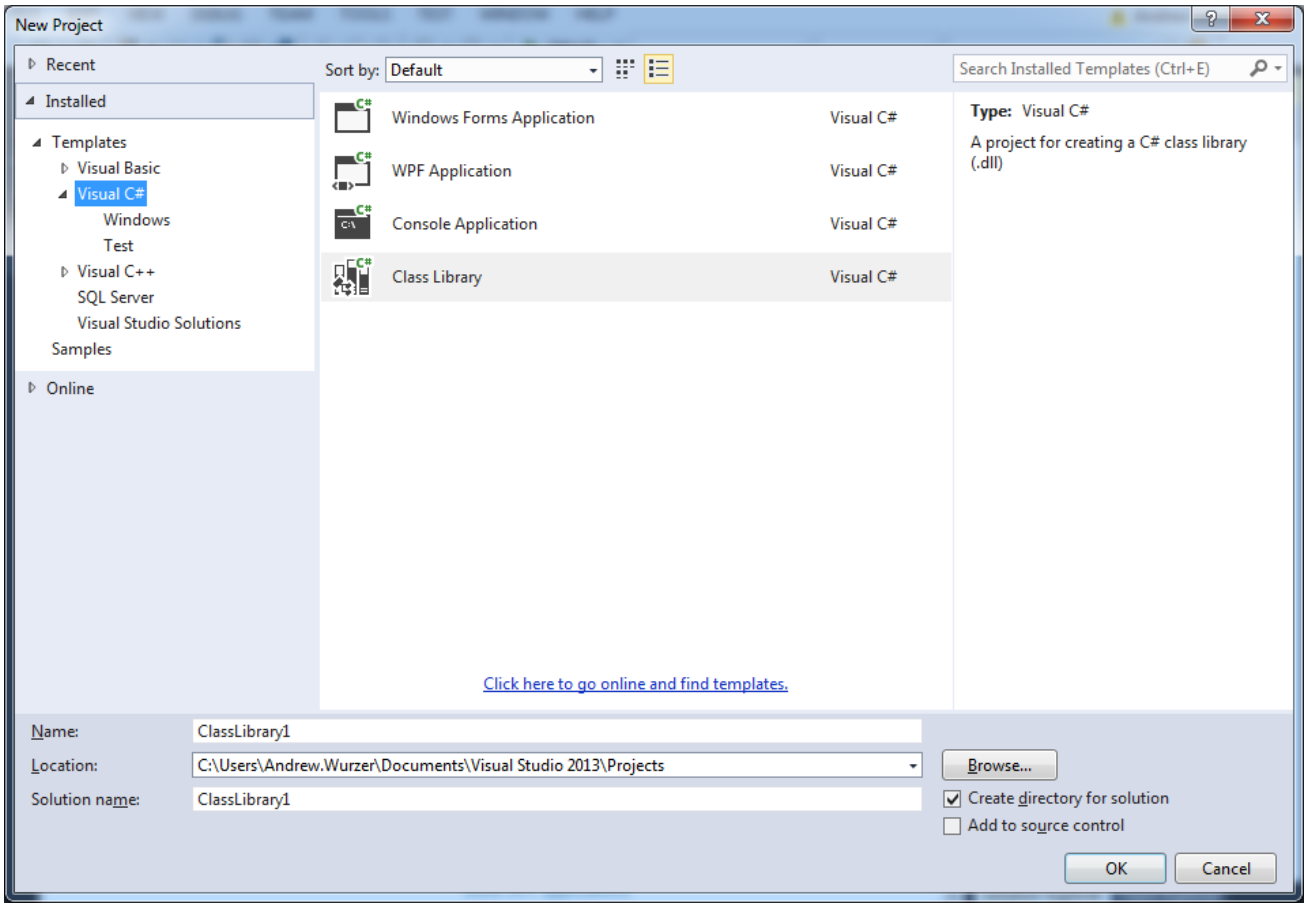
- **It is difficult to adjust to changing priorities.** If you are making major changes to one rule, and you need to quickly make a fix to another rule, you cannot build just the fixed rule into a DLL without the half-finished changes to the other rule. Those half-finished changes may break that business rule. So you have to either wait until both rules are finished, or undo your changes on the original rule.
- **Sharing a business rule shares the entire DLL file.** If you use Prophet 21's sharing business rule feature, the entire DLL ships with any business rule you share, even if there are many unrelated business rules in it. If you have a separate DLL file for each rule, you only share the rule you intended to share.

In Visual Studio Professional (or Visual Studio Express 2013 for Windows Desktop or better), you can create multiple projects in one solution, so you can keep all of your rules in one solution, but in separate projects within the solution. For more information on projects versus solutions, see the Microsoft Developer Network (MSDN) documentation for Visual Studio at msdn.microsoft.com.

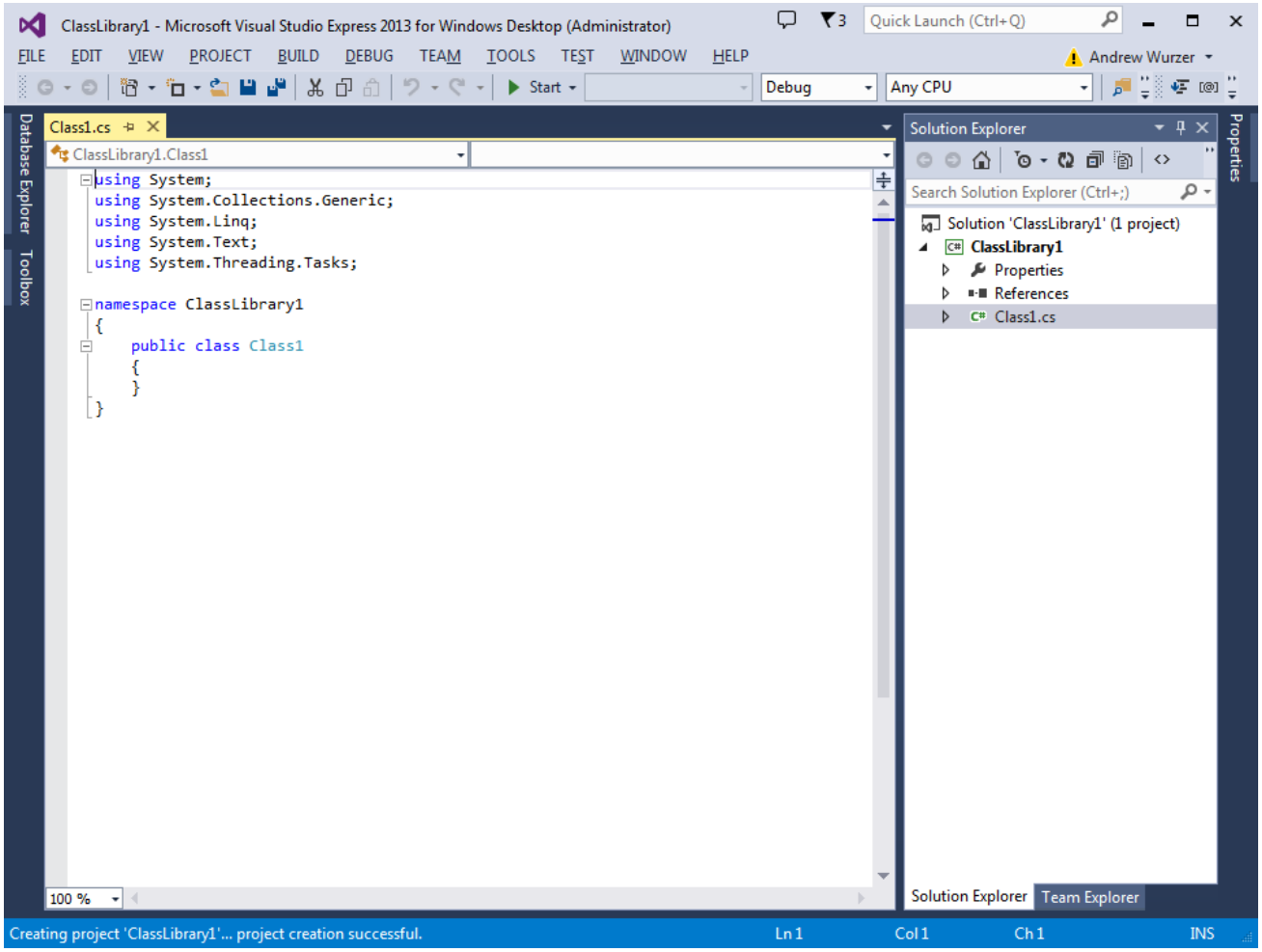
1. Open Visual Studio Express 2013 for Windows Desktop (Start > All Programs > Visual Studio 2013 > VS Express 2013 for Desktop).



2. Click **New Project...**. The New Project dialog opens.
3. Select **Class Library**.
4. Name your project.



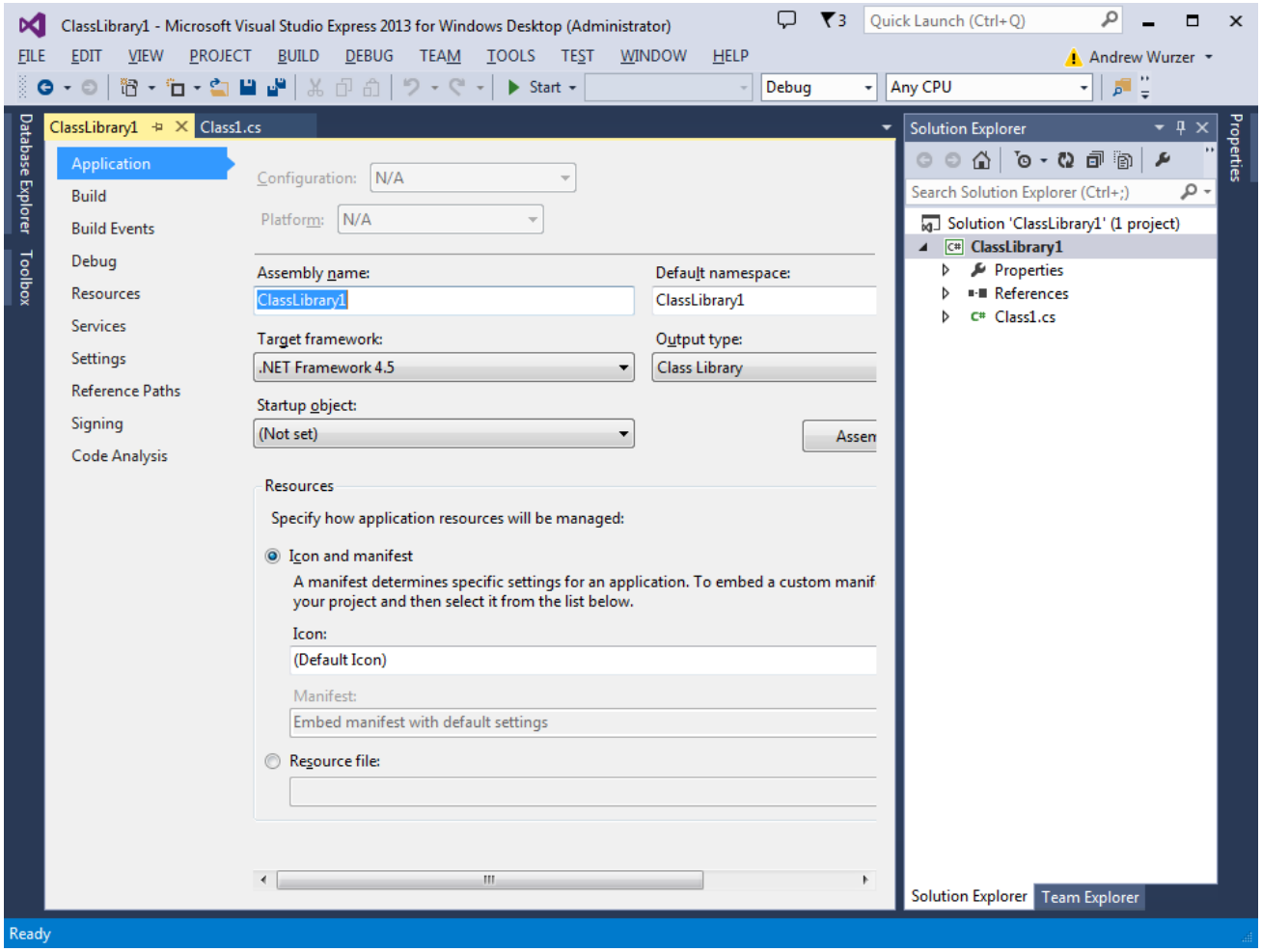
5. Click **OK**. Visual Studio creates the project with all of the basic parts of a C# class library.



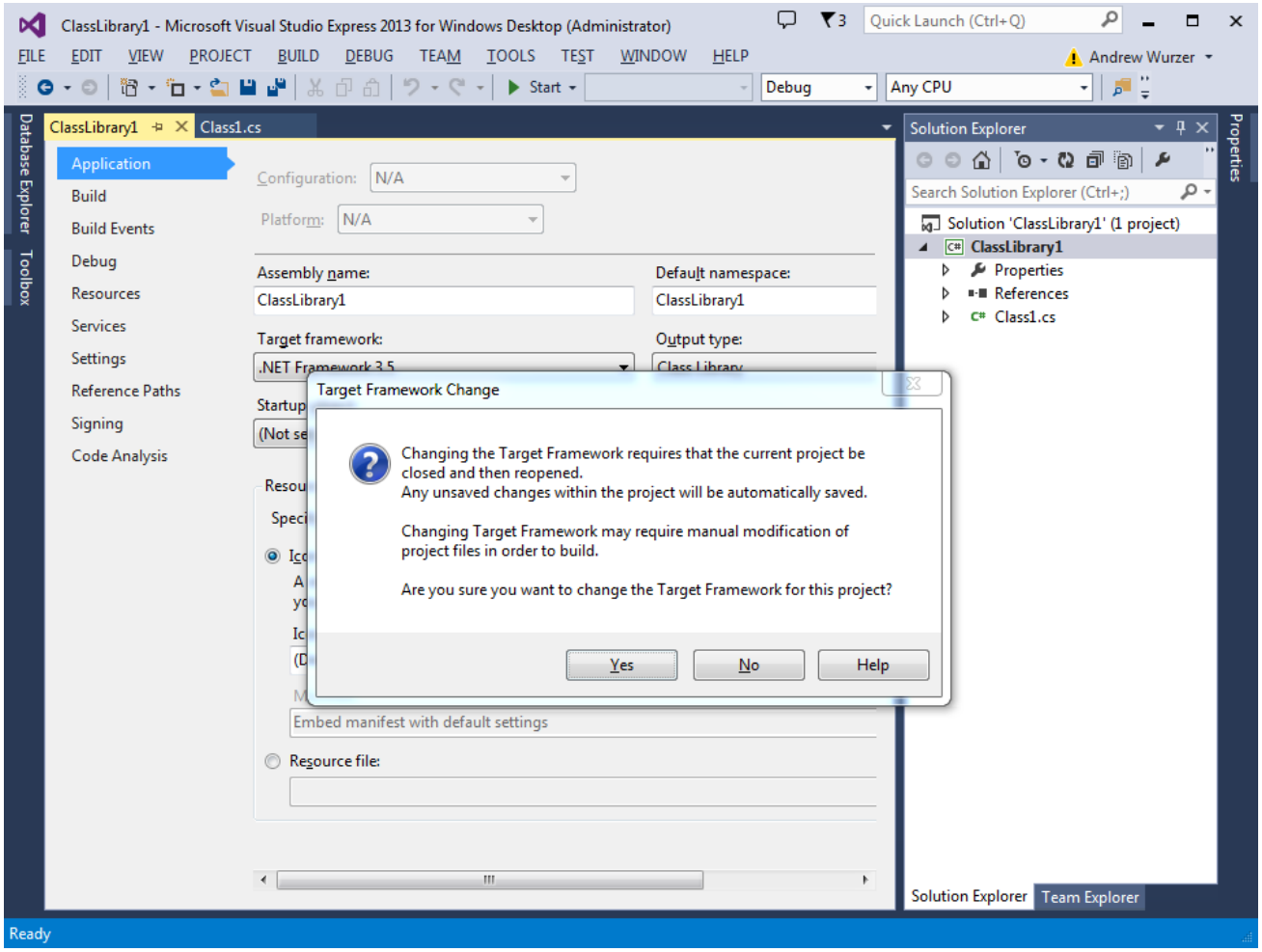
6. Click the **Save All** button to save the project (or go to File > Save All) to save the project.

7. In the Solution Explorer, right-click the name of your project and select **Properties**. The properties window for your project opens.

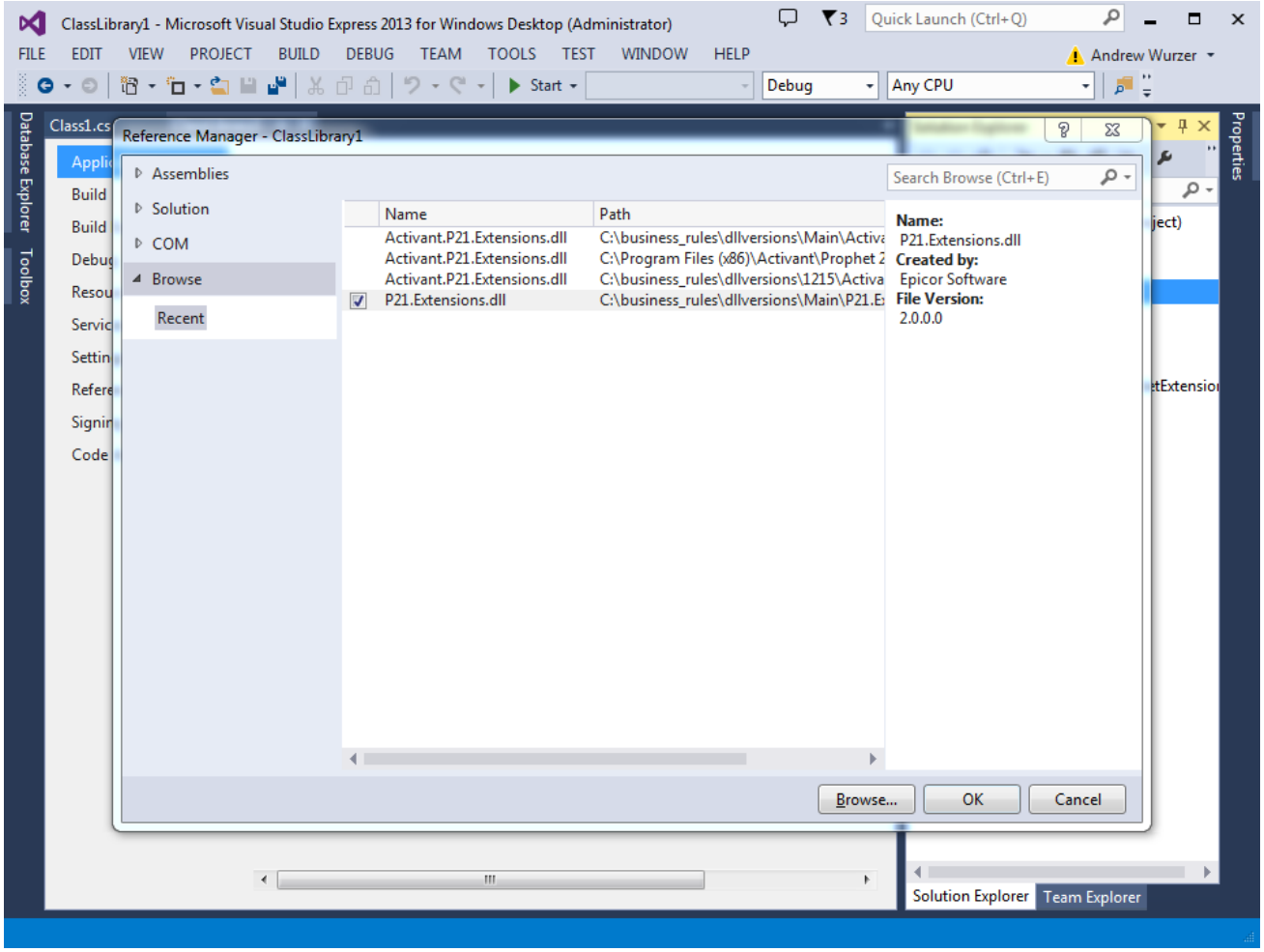
> You only have to change the framework version in Properties if you are using a version of Visual Studio higher than 2008. Visual Studio 2008 versions default to using the .NET Framework version 3.5, while Visual Studio 2010 and above default to version 4.0 or 4.5.



8. Set the Target Framework to **.NET Framework 4.0**. The Target Framework Change dialog pops up.



9. Click **Yes**. Visual Studio Express closes and reopens the project.
10. In the Solution Explorer, open the References folder.
11. Right-click the Microsoft.CSharp reference and select **Remove**.
12. Right-click the References folder and select **Add Reference**.
13. Open the **Browse** option. This shows a list of recently accessed DLLs.



14. From the list, select **P21.Extensions**.

> If you do not see Activant.P21.Extensions in the list, you can click the Browse button, go to the folder that contains your Prophet 21 installation (e.g., C:\Program Files\Activant\Prophet 21 12.15), and find the file P21.Extensions.dll, and highlight it. Continue with the next step.

15. Click **OK**. The P21.Extensions DLL is added to your project as a reference. This reference enables you to use the classes, methods, and fields you need to create business rules.

> In the past, you might have used Activant.P21.Extensions.dll; this DLL is used for the older non-multi-row version of DynaChange Rules and is no longer necessary for new rules. See *DLL File Change* on page 1 for more information.

16. If your DLL folder system setting points to a mapped network drive (see "Set Up System Settings" on page 13 for more information), you must make two modifications to your AssemblyInfo.cs file.

a. In the Solutions Explorer, expand the **Properties** node.
b. Double-click the **AssemblyInfo.cs** file.
c. At the top, add the statement `using System.Security;` if it does not exist.
d. Add the line `[assembly: AllowPartiallyTrustedCallers]` to the end of the file.

> Adding this line does not affect anything if you use rules locally; you can add it without causing any problems for local rules.

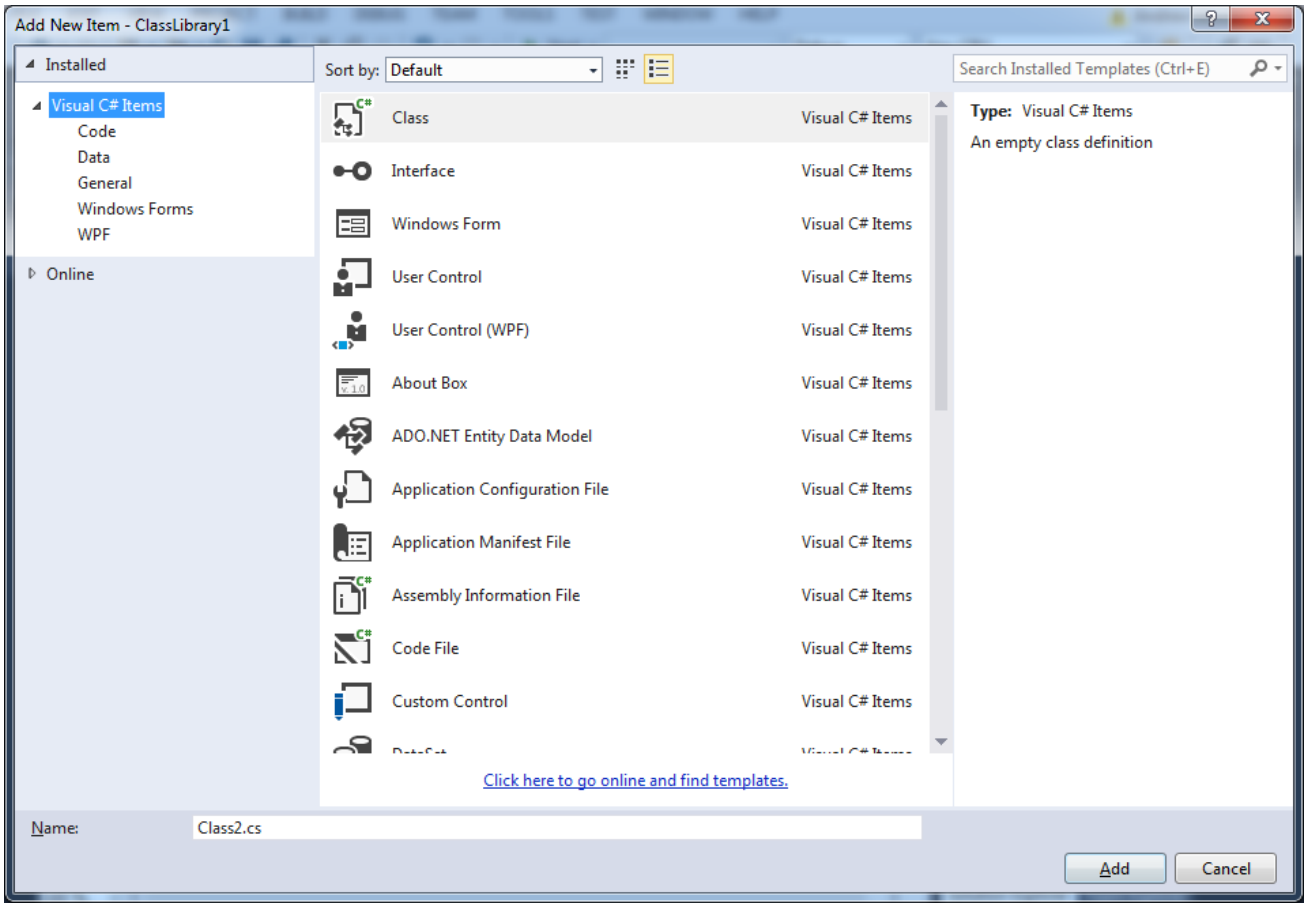
17. Save the project.

You now have a project to which you can add new business rule classes.

Adding a New Rule Class

All business rules are contained by a class derived from the Rule class. Each time you want to create a new business rule, you must add a new class. For more information on the methods and properties available to your business rule, see page 92 for more information.

1. Open the project you created above in "Create a Visual Studio Project" on page 22.
2. Highlight your project.
3. From the Project menu, select **Add Class**. The Add New Item dialog opens with Class selected as the item to add.



4. Change the name of the class to something that represents your business rule. For example, if you are creating a converter that provides the state and city when you enter the ZIP code, you might call it *ZipCodeLookup.cs*.
5. Click **Add**. You return to the project with the class file you just created open.
6. At the top, where you see other "using" statements, add the following statement:
   `using P21.Extensions.BusinessRule;`
7. Change the class declaration from `class ZipCodeLookup` to:
   `public class ZipCodeLookup : Rule`

   The code should now look like this (note your namespace and class name may be different):

```
using System;
using System.Collections.Generic;
using System.Linq;
using System.Text;
using P21.Extensions.BusinessRule;

namespace ClassLibrary1
{
    public class ZipCodeLookup : Rule
    {
    }
}
```

8. Save the project.

Creating the Required Class Methods

Every rule class need to have at least three particular methods defined to work properly: GetName(), GetDescription(), and Execute(). If it lacks GetName() or GetDescription(), the rule will not display in Prophet 21. If it lacks Execute(), no code will run and you will get an error message in Prophet 21.

1. In the Rule-based class you just created above, enter the first method:
   `public override string GetName()`
   Visual Studio will likely autocomplete the method by creating code that will generate an error that the method has not yet been implemented.
2. Create another method:
   `public override string GetDescription()`
3. Create the last required method:
   `public override RuleResult Execute()`

   The code should now look like this:

```
namespace ClassLibrary1
{
    public class ZipCodeLookup : Rule
    {
        public override RuleResult Execute()
        {
            throw new NotImplementedException();
        }

        public override string GetDescription()
        {
            throw new NotImplementedException();
        }

        public override string GetName()
        {
            throw new NotImplementedException();
        }
    }
}
```

4. Change the code in the GetName() and GetDescription() methods to return the name and description for the rule that you want to appear in Prophet 21. GetName() returns the name of the rule, and GetDescription() returns a description for the rule. Following is an example:

```
namespace ClassLibrary1
{
    public class ZipCodeLookup : Rule
    {
        public override RuleResult Execute()
        {
            throw new NotImplementedException();
        }

        public override string GetDescription()
        {
            return "Returns the state and city for a ZIP code.";
        }

        public override string GetName()
        {
            return "ZIP Code Lookup";
        }
    }
}
```

5. Replace `throw new NotImplementedException();` with:
   `RuleResult result = new RuleResult();`
   This code instantiates the RuleResult object that you will return to the Execute method. The

Success property of the RuleResult class defaults to true so that you only return a failure to Prophet 21 if your code actually fails.

6. You now add the algorithm that does the work of your business rule (or the code that calls another program that contains the algorithm). A very simple example follows. If you have questions about any of the code used, you can find more information in the API (see "Appendix A: Rule Class API" on page 90).

> This example is not working code, and is not how you would implement this functionality; it has been abbreviated and simplified for the purpose of example, abridging or removing important elements like error handling, comments, and validation.

```
namespace ClassLibrary1
{
    public class ZipCodeLookup : Rule
    {
        public override RuleResult Execute()
        {
            RuleResult result = new RuleResult();
            String _zip = Data.Fields.GetFieldByAlias
                ("postal_code").FieldValue;
            String _zipState = "";
            String _zipCity = "";
            // Do logic here that passes ZIP to some Internet
            // web service that provides city / state resolution
            // for ZIP codes; assume it returns the variables
            // _zipState and _zipCity.
            Data.Fields.GetFieldByAlias("state").FieldValue = _zipState;
            Data.Fields.GetFieldByAlias("city").FieldValue = _zipCity;
            return result;
        }
```

7. Save the project.

Creating a Workflow Rule Class

When you create a workflow business rule, you must use an additional method in your class definition: ExecuteAsync(). Code executed in ExecuteAsync() does not return a value to Prophet 21.

This code snippet shows the class and required methods. The rule simply pops up a message box that shows the ExecuteAsync() method successfully executed code. See the ExecuteAsync() method of the Rule class in Appendix A: Rule Class API for more information.

Build and Deploy to Prophet 21

Once you have created your business rule, you must build and deploy the class library to the location you specified when you set up Prophet 21 so the system can "see" the rule.

> Make sure the Prophet 21 client is not running when you do this.

1. Press [F6] to build the library (or go to Debug > Build Solution). As long as there are no compiler errors in your code, Visual Studio builds the class library into a DLL file (with the same name as your solution).
2. Find the DLL file in Windows Explorer. By default, this is located in your My Documents folder (or Documents folder in Windows Vista or higher). From My Documents, find the path: `Visual Studio 2013\Projects\<solution>\<project>\bin\Release\`
3. Copy the DLL file.
4. Find the file path you set up in the DLL Folder field in the Files and Folders node of the System Setup system settings.
5. In Windows Explorer, open that folder.
6. Paste your DLL file into the folder.
7. Next time you open Prophet 21, you will have access to your new business rule. See "Link Business Rules to Prophet 21" on page 50 for more information on hooking the business rule into a Prophet 21 field or button.

Sample Rules

Epicor provides a few sample business rules to demonstrate various types of rules. The rules are provided both as a DLL file you can hook directly to Prophet 21 and as a Visual Studio solution that contains all the source code for the rules.

See "Appendix B: Sample Rules" on page 115 for more information on the sample rules.

Developer's View of Web Visual Rules

This chapter helps developers understand and create the projects and sites for Prophet 21's web visual rules. In "Appendix A: Rule Class API" on page 90, you will learn how to manage business rules you create, and link them to the Prophet 21 user interface.

To set up web visual rules and make them available in Prophet 21, you must:

- Copy the P21.Visual.Rules solution folder from your Middleware server and set up the solution into a new folder on your development machine
- Add a new MVC area to the project
- Create a controller and view for your rule
- Test your web site using the test harness
- Deploy the web site so that Prophet 21 can find and use it

This chapter shows some code, but it does not explain the purpose of most of the code. It explains only the aspects of the code that are key parts of web visual rules setup.

What Is a Web Visual Rule?

A web visual rule is a rule that uses a web site to display its visual elements, rather than WinForms or WPF or some other visual technology.

You should use web visual rules for the Prophet 21 web application. They are necessary because, unlike the desktop application, the web application cannot open WinForms or other similar technologies; web browsers cannot execute them. So where, in the desktop application, you could use WinForms to show your user a data input screen, or display the results of a query, in Prophet 21, you must open a web page. Web visual rules provide the framework under which you can open that web page from a rule inside of Prophet 21.

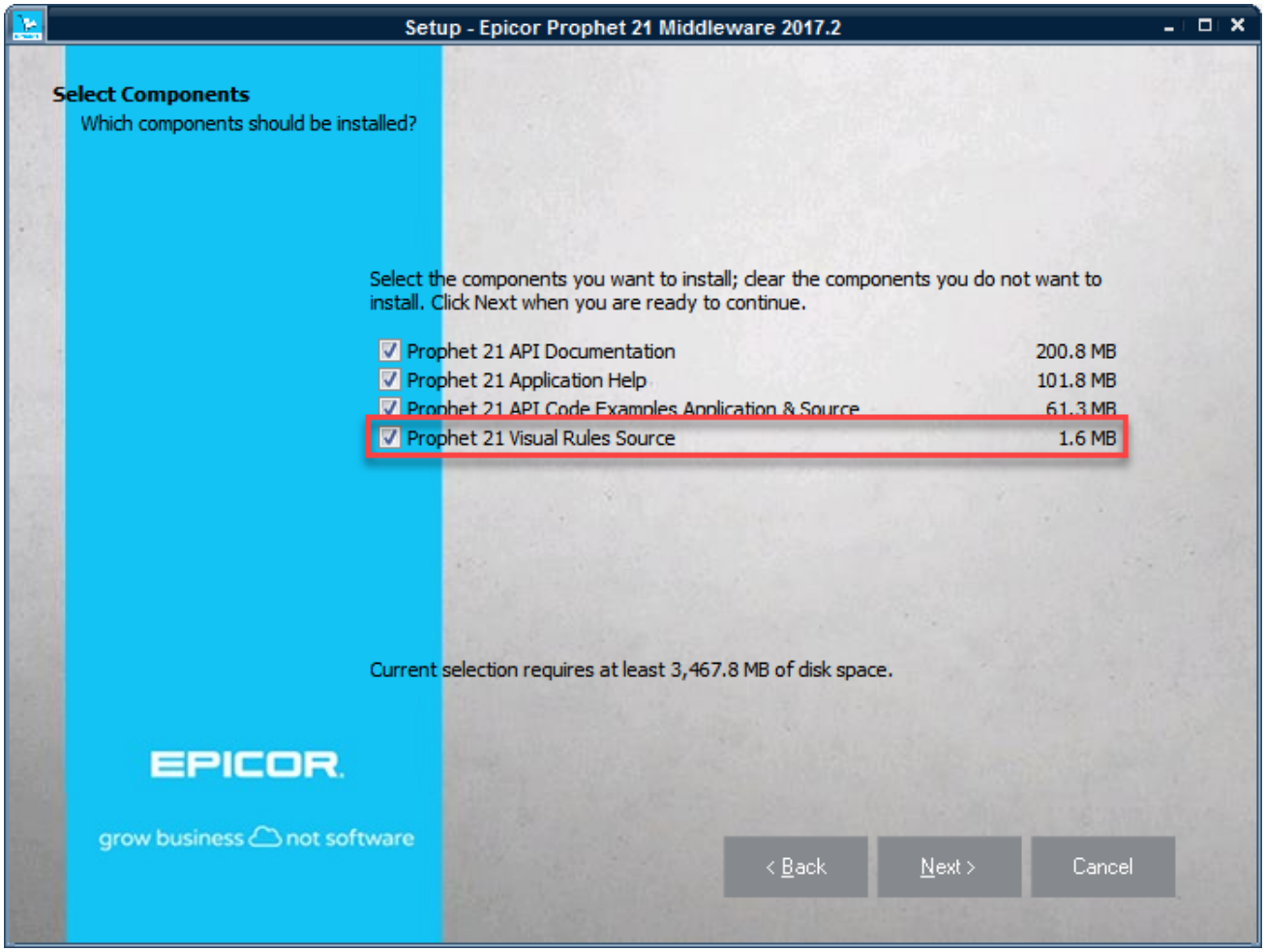
For example, you may have created a rule in the desktop application that, when you press a button on the Aging tab in Customer Master Inquiry, displays all the invoices that go into each of the buckets. You use a WinForm grid to display the invoices. On the web, you can no longer use the WinForm, so instead you can create a web page that will query and display that same invoice information within Prophet 21. Web visual rules is the framework you can use to display the invoices in a grid.

Web visual rules use an ASP.NET MVC web site to handle the visual display of information from a rule in Prophet 21.

Set Up Your Visual Studio Solution

Epicor provides a template solution for you to use to create your web visual rules; this template is installed with the Prophet 21 Middleware server software. Use the following instructions to obtain and set up your Visual Studio solution to create web visual rules.

1. Install the source code for web visual rules. As of version 2017.2, if you install or update the Prophet 21 Middleware server, you will see a screen that prompts you to install the Prophet 21 Visual Rules Source. This option installs the Visual Studio solution that you will need to use to create web visual rules.



2. Once you have installed the source, you can go to where you have installed your Middleware (by default, this is C:\Epicor\Prophet21\Middleware\Default) and find the P21.Rules.Visual folder. This folder contains the solution source code files for web visual rules.

> If you enable the **Web Visual Rules Physical Path**, you can enter a different path from the default where the installer places the **Visual Rules** folder.
>
> Epicor strongly recommends placing visual rules outside the middleware folder. This way, updates do not overwrite them.

3. Copy the folder to the location where you will work on your code (presumably your source control repository).
4. Double-click **P21.Rules.Visual.sln** to open the solution in Visual Studio.
5. Right-click on the project name (**P21.Rules.Visual** by default) and select **Properties**. The Properties file opens.
6. On the Web tab, set up the server you will use to create and test your web visual rules.

> You can use your real middleware server IIS for web visual rules development, or you can use a local server for convenience of testing and deployment; these instructions describe using a local server with a local Middleware instance.

7. Under the Servers area, select Local IIS.

> If you are not using a local server, you will have to use the External Host settings.

8. In the Project URL field, enter **https://<my_server_url>/P21.Rules.Visual** (or whatever you want to call your web visual rules site) where <my_server_url> is the fully-qualified domain name for your web server.

> Your web server must be using SSL, so if you do not have an SSL cert for your local machine, you will have to get one, or use one of your IIS web servers for which you have an SSL certificate.

9. Click **Create Virtual Directory.** This will create a web page on the IIS server and use the local files of your project to drive it.
10. Build your solution (Build > Build Solution, or press F6 or Ctrl + Shift + B).

The solution is now ready to develop web visual rules.

Create a New Rule

As a tutorial, you will recreate one of the sample visual rules, a rule which displays and enables you to change address information. This process will help you understand how a web visual rule is put together.

Add a New Area

You can use areas as a simple way to keep your various web visual rules separated. This cuts down on confusion as to which models, views, and controllers go with which rules. Use the following instructions to add an area to your project for the tutorial rule.

1. In the Solution Explorer, right-click the Areas folder and click Add > Add Area. The Add Area dialog opens.
2. TypeTutorialSample into the Area Name field.
3. Click Add. Visual Studio displays a progress bar while it scaffolds the new area, then you will see the TutorialSample folder under Areas.

You now have a new area ready for you to create a new web visual rule. The scaffold built up the files and structures your rule needs.

Create a Model

A model is an optional step in creating a web visual rule. A model is a simple C# class (or classes) that contains data you will use in your rule. A model can make it easier to work with and bind data in your web visual rule. The sample rule contains a CustomerAddress model class, which we will duplicate, because the sample rule uses a model. Models are not required for web visual rules. Use the following instructions to create the CustomerAddress model.

1. In the Solution Explorer, right-click the Models folder and select Add > Class. The Add New Item dialog opens.
2. Select Class, then name your class CustomerAddress.cs. The class appears in the Solution Explorer, and opens in an editing pane.
3. In the Solution Explorer, go to Areas > SampleRule1 > Models and open the CustomerAddress.cs file there.

4. Copy the contents of CustomerAddress.cs to the new CustomerAddress.cs you just created. Copy only the contents within the braces after “public class CustomerAddress”. Do not copy any of the headers or boilerplate.

You now have a model for CustomerAddress that includes customer ID, address 1, address 2, city, and country.

Create a Controller

The first step of creating a web visual rule is to create your controller. The controller is the class that contains the business logic for your web visual rule. It is roughly analogous to the Execute() method in non-web visual rules. The controller lets you use all of the normal C# methods for your rule. Use the following instructions to create a controller and add code into it.

1. Right-click the Controllers folder under TutorialSample and select Add > Controller. The Add Scaffold dialog opens.
2. Select **MVC5 Controller - Empty**.
3. Click Add. The Add Controller dialog opens.
4. Change the controller’s name from DefaultController to **TutSampController.**
5. Click **Add.** The TutSampController.cs file appears under the Controller folder, and the controller file is open in the editing pane.
6. In the Solution Explorer, go to Areas > SampleRule1 > Controllers and open the SampleRule1Controller.cs file there.
7. Copy the contents between the braces after the line “public class SampleRule1Controller : BaseRuleController” from the SampleRule1Controller.cs file and paste it between the braces after “public class TutSampController : BaseRuleController” in the TutSampeController.cs file, replacing what was there previously. There will be several underlined values.
8. Right-click the underlined CustomerAddress and select **Quick Actions and Refactorings.**
9. Select **using P21.Rules.Visual.Areas.TutorialSample.Models;,** which adds the using statement to the file so the CustomerAddress is now qualified.
10. Right-click the underlined DataRow and select **Quick Actions and Refactorings.**
11. Select **using System.Data;,** which adds the using statement to the file so the DataRow is now qualified.

12. Right-click the underlined SqlCommand and select **Quick Actions and Refactorings.**
13. Select **using System.Data.SqlClient;**, which adds the using statement to the file so the SqlCommand is now qualified.
14. Right-click the underlined List<dynamic> and select **Quick Actions and Refactorings.**
15. Select **using System.Collections.Generic;,** which adds the using statement to the file so the List<dynamic> is now qualified.
16. Press **Ctrl + Shift + S** to save all the files in the project so far.

You have now created your controller. The controller simply instantiates a CustomerAddress class, then loads the values passed from Prophet 21 into it, using the d_customer_maint_address table that was passed. Then, depending on which option the user chose on the page (grid view or form view), the rule returns the results to the appropriate view. Once the user returns the rule, the controller pushes any edited values back into the d_customer_maint_address DataTable and sends the results back to Prophet 21.

Create a View

The last step in reecreating the sample rule is to create the two views it supports (form view and grid view). Use the following instructions to create the two views.

1. Right-click the Views folder under TutorialSample and select Add > View. The Add View dialog opens.
2. Name the view **FormView.**
3. Click **Add.** The FormView.cshtml file appears under the View folder, and the view file is open in the editing pane.
4. In the Solution Explorer, go to Areas > SampleRule1 > Views and open the FormView.cshtml file there.
5. Copy the entire contents.
6. Paste the entire contents into the FormView.cshmtl file you just created, replacing everything in the file.
7. Change the line @model P21.Rules.Visual.Areas.SampleRule1.Models.CustomerAddress to read **@model P21.Rules.Visual.Areas.TutorialSample.Models.CustomerAddress.**

8. Change the line **Html.BeginForm("SaveAndReturn", "SampleRule1", FormMethod.Post) to Html.BeginForm("SaveAndReturn", "TutSamp", FormMethod.Post).**
9. Change the line **@Url.Action("FormView", "SampleRule1") to read @Url.Action ("FormView", "TutSamp").**
10. Change the line **@Url.Action("GridView", "SampleRule1") to @Url.Action("GridView", "TutSamp").**
11. Right-click the Views folder under TutorialSample and select Add > View. The Add View dialog opens.
12. Name the view **GridView.**
13. Click **Add**. The GridView.cshtml file appears under the View folder, and the view file is open in the editing pane.
14. In the Solution Explorer, go to Areas > SampleRule1 > Views and open the GridView.cshtml file there.
15. Copy the entire contents.
16. Paste the entire contents into the GridView.cshmtl file you just created, replacing everything in the file.
17. Change the line **Html.BeginForm("Return", "SampleRule1", FormMethod.Post) to Html.BeginForm("Return", "TutSamp", FormMethod.Post).**
18. Change the line **@Url.Action("FormView", "SampleRule1")** to read **@Url.Action("FormView", "TutSamp").**
19. Change the line **@Url.Action("GridView", "SampleRule1") to @Url.Action("GridView", " TutSamp")**.
20. Press **Ctrl + Shift + S** to save all the files in the project so far.

You now have your two views: a form view and a grid view for your address. These will display the data passed from Prophet 21 in a web page so the user can see and interact with it, then submit it back to the controller once they have finished.

Test Your Rule with the Test Harness

Now you are ready to test your web visual rule using the built-in test harness. The test harness allows you to pass in XML just as Prophet 21 would pass it to your web visual rule and see how the rule performs. This saves time and trouble because you do not have to set up the rule in Prophet 21; you can

simply get the XML from Prophet 21 (to do so, you must set up the rule there; and then Load Selected Fields, then Generate XML for Unit Test to get the XML; see Link Business Rules to Prophet 21 for more information). Use the following instructions to test your new web visual rule in the test harness.

> See the TestHarnessData.txt file right below the hmtl file? That file contains the XML from Prophet 21. Copy XML from into that file if you do not want to copy the same XML over and over into the test harness web page.

1. Build the solution (Build > Build Solution). If you encounter and build errors, you will have to troubleshoot them to continue; review the previous sets of steps to ensure you have not missed any.
2. In Visual Studio, go to Debug > Start Debugging (or you can press F5). Chrome (or the browser you have set up) will start in a debugging mode, and the test harness page opens automatically.
3. Enter a valid username and password for the database connected to the Middleware server you are using for this test.
4. If you were testing your own rule, you could replace data in the Data for Server Request field, which is the data that Prophet 21 would send from a rule.
5. Scroll down and click the **Start Execution** button. The second step in the page displays, showing the editable address information in a form view.
6. Click Grid View to see the address data presented in a grid.
7. Click Form View to see it presented back into a form again.
8. Change the Address1 field to a different value (such as 111 Main).
9. Scroll down and click **Save and Return**. The test harness shows the XML that would be returned to Prophet 21. You can review this XML to see if the changes you intended exist – in this case, see the <fieldValue></fieldValue> tags under the <fieldTitle>Address 1</fieldTitle> to see the changed address field.
10. Click Run Again if you wish to run the rule a second time. Running again may be useful if you want to test multiple different sets of data.

If you run into any issues while testing your rule, review the steps above, or see the Troubleshooting Web Visual Rules section for more information.

Connect Your Web Visual Rule with Prophet 21

Finally, to test your rule, you can connect it to Prophet 21. The process for connecting a web visual rule to Prophet 21 is very similar to any other rule; the main difference is that you need to set up a web URL for the web visual rule.

Use the following instructions to set up your test rule in Prophet 21.

1. Open the Prophet 21 desktop application.
2. Open the Customer Maintenance window.
3. Right-click the Mail Address 1 column and select DynaChange > DynaChange Rules > Create Business Rule. The New Business Rule dialog opens.
4. At the end of the list of rules (if you have any rules at all) you will see the rule with the Rule Name "<New Web Visual Rule>" – enable the Selected checkbox for this rule.
5. Name the rule TestCustomerAddress.
6. On the Field Selector tab, select the following fields: **mail_address2, mail_city, and mail_ country.** The **mail_address1** field should already be selected and marked as the Triggers Rule field.
7. On the Configuration Options tab, set the Rule Type to Converter.
8. Enable the Multi-Row checkbox.
9. Select **Web Browser** from the Enabled for Version drop-down field. This setting prevents Prophet 21 from trying to open the web visual rule in the desktop (where it will not work).
10. In the Web Visual Rule field, enter the URL to your rule as it was listed in the Visual Rule URL field in the test harness. It should look something like the following (without a line break):

    https://myserver/P21.Rules.Visual/Initialize?

    ruleController=TutorialSample/TutSamp&ruleAction=FormView

11. Select **Save**.
12. Open the same Prophet 21 system, but this time in the web application.
13. Open Customer Maintenance.

14. Retrieve a customer record.

> Ensure the Mail Address 2, Mail City, and Mail Country fields all have data. The extremely simple sample rule does not have null-checking code, so if you do not have a value in one of the fields, you will see a null cast exception.

15. Change the data in the Mail Address 1 field. The Visual Business Rule dialog displays.
16. You can switch between the Form View and Grid View, just as you could in the test harness.
17. Change the Mail Address 2 to Suite 200.
18. Click Save and Return. “Suite 200” appears in the Mail Address 2 field.

You have now learned the basic parts of a web visual rule and connected a simple example to Prophet 21. If you received an error message at some point, carefully repeat the steps. If that does not solve the problem, see the Troubleshooting section of the manual.

Visual Response Attributes

You want to make a popup window that asks a user to pick from a few options or to enter a few additional details in fields the popup displays. You could do this by building a web visual rule, but that is a lot of work for a simple response window.

Enter visual response attributes. These tools provide a simpler way to create these response windows using the simpler framework of standard DynaChange Rules. With visual response attributes, you can create two different kinds of response windows:

- One containing multiple buttons that a user can select.
- One containing fields for the user to enter additional data.

> In both cases, the response window has a limit of five options/fields.

What They Cannot Do

Visual response attributes provide a nice shortcut from building a full web visual rule when you want a simple response window, but they do have some limitations.

- You only have limited customization options for the appearance of the popup windows you create with visual response attributes.
- You cannot attach other rules to the fields in a popup and the popup cannot trigger a rule in Prophet 21.
- Prophet 21 performs no validation for data entered in these popups.

If you require any of this, you need to create the popup using web visual rules.

How They Work

Inside a traditional business rule, a visual response attribute defines the text and options in the popup. Once the user interacts with the popup, a callback rule handles the new input. This is all part of the rule with no additional setup or coding required.

> Because of the way visual response attributes work within a rule, you only need to ensure your p21.Extensions.dll is up to date and to set up a Callback Rule class. There is no additional setup required beyond the procedure described elsewhere in this manual for business rules overall.
>
> If you want an example of these rules in action, the Prophet 21 SDK, available within

the Prophet 21 Middleware site, contains a zipped Visual Studio solution called VisualResponseAttributesExamples. This solution contains two example projects, one illustrating how to use ResponseAttributes to generate a multi-option message and another that generates a simple response window. These code examples may be downloaded and experimented with as needed. Please see the example solution for more information.

Coding Callback Rules

Within the CallbackRule, there are a few key new additions that are useful for coding the handling logic. Two new relevant properties have been added to the RuleState class, an instance of which is available to all rules. Both properties can be used in the callback rule to verify the context and confirm that the expected data from the message/response should be available for additional processing.

Property:	Description:
IsCallbackRule	Boolean value indicating whether the rule being executed is a callback rule triggered after a message or simple response window generated via ResponseAttributes has been displayed and interacted with by the user.
CallbackParentRule	The name of the triggering rule. This is the name of the rule class that intitially returned ResponseAttributes causing a message or simple response window to be displayed.

Though you can set CallbackRule as the initially triggered rule class, but if you do you need to use callback rule properties in code to differentiate between the contexts and separate pre-display and post-display logic to avoid errors.

You can find all of the rules used in these popups in the Rule API chapter.

Managing Business Rules

Prophet 21 enables you to manage your business rules: you can link and unlink them to the user interface, configure them, manage permissions for them, and share them with other distributors. After you create your business rules, you must link them into the Prophet 21 user interface.

How Managing Business Rules Works **47**

Types of Business Rule Links 48

Link Business Rules to Prophet 21 50

Link Business Rules to Imports 55

Link Event-Driven Rules to Prophet 21 56

Link Business Rules to User-Defined Menus 59

Link Business Rules to Custom Right Mouse Button Menu Options 63

Set Configuration Options 64

Manage Rule Permissions 71

Edit Business Rule Links 72

Remove Business Rules 75

Share Business Rules 77

How Managing Business Rules Works

Once you have created and compiled your business rules and placed them in the DLL folder, Prophet 21 gains access to them. There are tools built into Prophet 21 that help you to manage your business rules. Those tools expose the business rule classes contained in your class libraries (DLL files), so that you can hook them into Prophet 21.

Managing business rules comprises:

- Managing the "link" between the Prophet 21 user interface and business rules
- Configuring how Prophet 21 handles business rules you link
- Setting permissions on business rules you link

- Sharing business rules
- Removing business rules

Types of Business Rule Links

There are several types of business rule links you can use to hook your business rule into Prophet 21. These types of links define *when* a business rule runs and if it returns a value to Prophet 21.

- **Converter** - Use to change data before Prophet 21 validation. Cannot be applied at save.
- **Validator** - Use to validate data after Prophet 21 validation. However, when you apply a validator at save time, it validates before Prophet 21 validations.
- **On Demand** - Use for buttons. Cannot be applied at save.
- **Workflow** - Use for business rules that execute asynchronously and return no value to Prophet 21. Only applies at save time.

  When you use the workflow link type, the business rule executes code in the ExecuteAsync() method, not the Execute() method. Ensure your business rule can access your logic through the ExecuteAsync() method.

- **On Event** - Use for event-driven rules that trigger when some event arises in the application (such as the creation of a pick ticket). Note that you cannot select this rule type in the Rule Type drop-down; it is set automatically when you set up a rule while listening for an event. See "Link Event-Driven Rules to Prophet 21" on page 56 for more information on linking On Event rules.
- **User-Defined Menu** - This type of rule does not appear as an option on the Rule Type drop-down list; rather, this link type is always used for rules linked to user-defined menus. See "Link Business Rules to User-Defined Menus" on page 59 for more information on linking rules to user-defined menus.

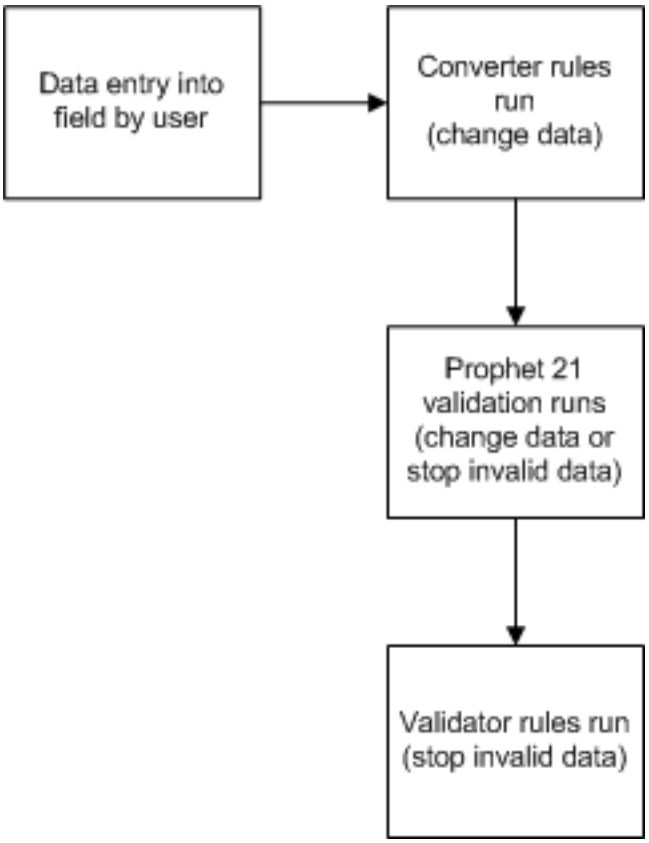
  When you use the User-Defined Menu link type, the business rule executes code in the ExecuteAsync() method, not the Execute() method. Ensure your business rule can access your logic through the ExecuteAsync() method.

- **RMB** - This link type is always used for rules linked to custom right mouse button menu options. See "Link Business Rules to Custom Right Mouse Button Menu Options" on page 63
- **Pre-SQL** – This rule type enables you to change SQL that Prophet 21 is about to execute before it executes.

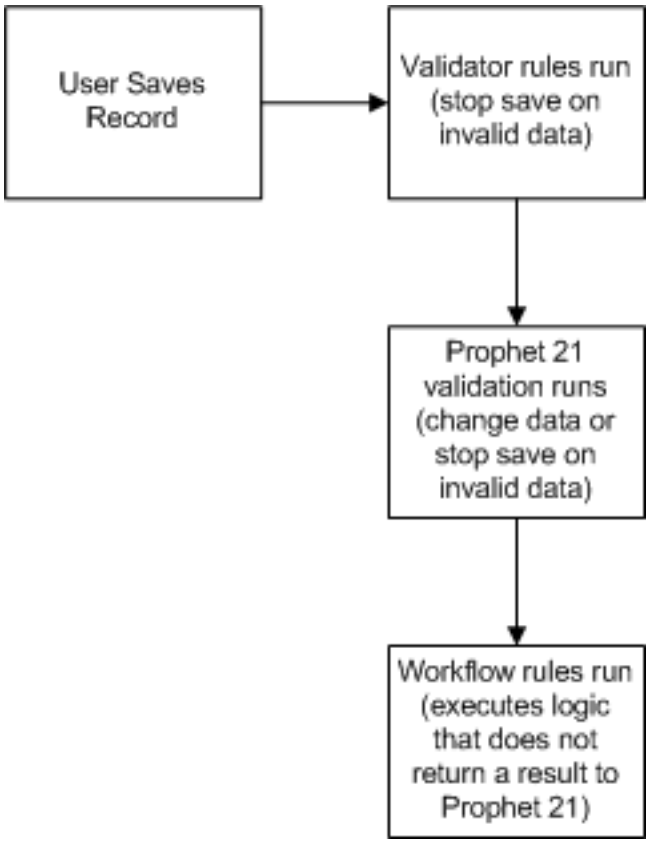
Rule Flow Based on Type of Rule Link

On Demand and On Event rules are not included in the diagrams below because On Demand only apply when the user clicks a button, and On Event apply immediately upon the event being triggered.

The following graphic shows the process data goes through when you use either a converter or a validator at the field level (rather than the applying the rule at save).



The following graphic shows the process data goes through when you save a record and have validator or workflow business rules applied on save.



Best Practice: Test Business Rules in Play System

Before you link a business rule to your live system, Epicor recommends that you first test the rule in your play system. After you have set up and tested the rule in your play system, you can avoid re-creating the setup in your live system by exporting the rule from your play system, and then importing it into your live system. The export / import process carries all of the associated fields and the rule DLL with it. See "Share Business Rules" on page 77 for more information.

Link Business Rules to Prophet 21

After you create your business rules, you must link your business rules to the Prophet 21 user interface.

Linking business rules to imports involve extra steps. See "Link Business Rules to Imports" on page 55 for more information on linking business rules to imports.

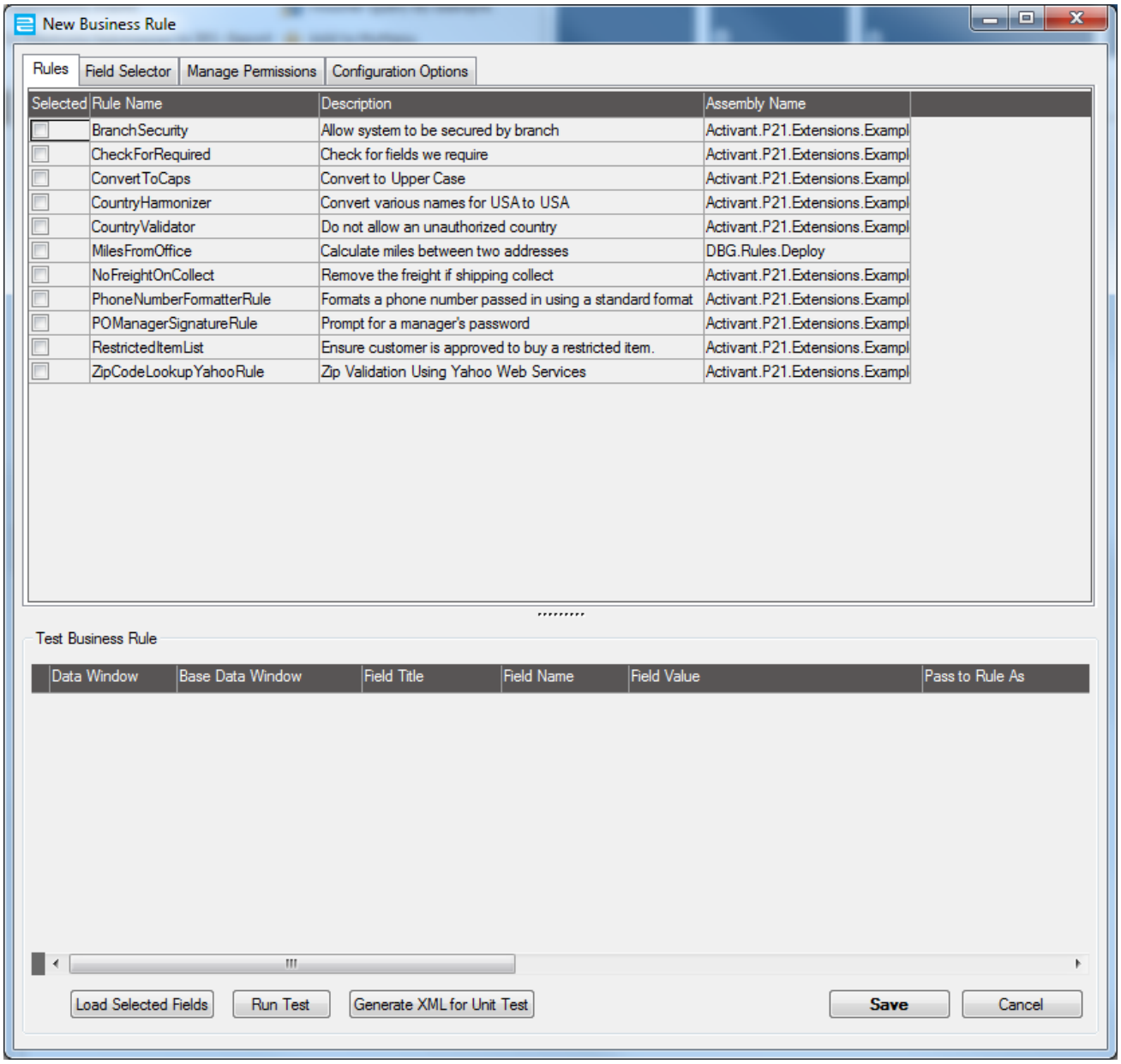
Event-driven rules have a different way of initiating a link, but the rest of the process is the same. See "Link Event-Driven Rules to Prophet 21" on page 56 for more information on linking event-driven rules in Prophet 21.

Rules attached to user-defined menu options and custom right mouse button menu

options have a different way of initiating a link in Prophet 21. See "Link Business Rules to User-Defined Menus" on page 59 for more information on linking rules to user-defined menu options. See "Link Business Rules to Custom Right Mouse Button Menu Options" on page 63 for more information on linking rules to custom right mouse button menu options.

1. Open Prophet 21.
2. Navigate to the window and tab where you want to link your business rule.
3. Right-click the field or button to which you want to link your business rule and select DynaChange > DynaChange Rules > **Create Business Rule**. The New Business Rule dialog opens, listing your available business rules.
4. Find the rule you want to apply, and enable the **Selected** check box next to it.

If you are linking a web visual rule, scroll to the bottom of the list and enable the Selected checkbox next to <New Web Visual Rule>. You can then name the rule by editing the Rule Name field.



5. Open the **Field Selector** tab and select the fields you want to send to your business rule. You see a treeview of tab names.

> In many windows, you will not see all the tabs in the window. The dialog can only show tab names and objects that have been loaded. For performance reasons, in some windows not all tabs load when the window opens. If you do not see the tab you want, make sure you visit that tab in the application before you click Create Business Rule or Edit Business Rule from the right mouse button menu.

6. Expand the tab name that contains the field or fields you want to send to the business rule. A list of objects appears.

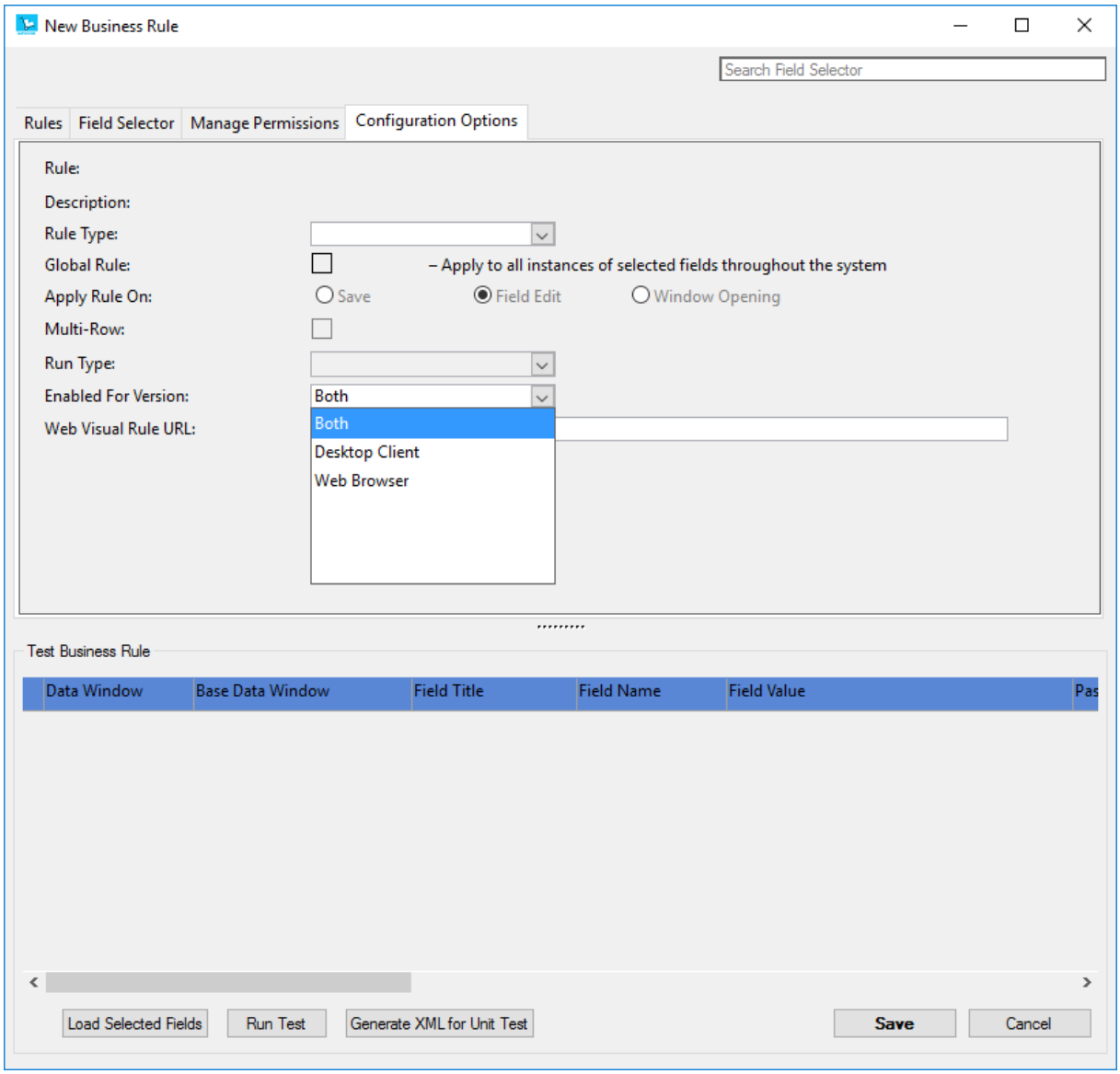
> Usually, there is one object per tab, but some tabs contain more than one object. An object is typically a part of the window; for example, if a window contains both form-type and grid-type elements these are frequently divided into two objects.

7. Expand the desired object. A list of fields appears.

> Rather than scrolling through each of the tabs, use the Search Field Selector field to search the treeview for a field. To clear the search and return to the original treeview, delete the value from the Search Field Selector field.

8. From the list of fields, enabled the **Selected** check box for all the fields you want to be sent to the business rule.
9. If necessary, set the field's alias by populating the **Pass to Rule As** field.
10. Repeat Steps 6 - 9 for any other tabs that contain fields you want to include in your rule.
11. On the **Configuration Options** tab, select the rule type you want to use from the **Rule Type** drop-down field. See "Types of Business Rule Links" on page 48 for more information on rule types.

> You can only select a rule type for one rule. If you want to select a rule type for a different rule, simply select a rule type for that rule, and the system automatically removes the rule type from the current rule.



12. From the Enabled For Version list, select the system for which the rule is used.

> This setting allows you to designate the rule only to the Prophet 21, only to the desktop application, or to both. The **Enabled for Version** list box also appears on the Configuration Options tab on the Edit Business Rule window.

> Web visual rules should always be set to Web Browser.

13. For web visual rules, in the Web Visual Rule URL field, add the URL that leads to your web visual rule. This should follow this convention:

https://<YourWebVisualRuleServer>/Initialize?ruleController=<MyRuleArea>/<MyController>&RuleAction=<MyView>

For example:

https://PH0816.americas.epicor.net/P21.Rules.Visual.3063/Initialize?ruleController=TutorialSample/TutSamp&ruleAction=FormView

14. Click **Load Selected Fields** to load your fields into the Test Business Rule pane.
15. The Field Value field contains the text that is sent to the rule. Change that data to represent your test data.
16. Click **Run Test**. This button passes the data from your fields to the business rule, runs the algorithm, and passes back the results (if any).

> The Run Test feature does not work with web visual rules.

17. If you are finished with the rule, save the rule. This closes the New Business Rule dialog. If you plan to modify permissions or configuration for the rule, you can leave the rule open and continue to the sections on that.

> You can view the XML that is sent to the rule by clicking the Generate XML for Unit Test button. You can copy this text to your clipboard by clicking the appropriate button. Displaying the XML makes it easier for the rules writer to create unit tests in Visual Studio or use the test harness for web visual rules.

You have now built the data set that Prophet 21 provides to your business rule. Continue to one or more of the following sections.

- "Set Configuration Options" on page 64
- "Manage Rule Permissions" on page 71

Link Business Rules to Imports

Linking business rules to imports begins similarly to linking business rules to Prophet 21; however, it does include additional steps.

> In some situations, convertor rules work with imports; however, because import processing occurs after saving, validator rules are the most appropriate rule type.

1. Open Prophet 21.
2. Navigate to the import window where you want to link your business rule.

3. On the Setup tab, select a header file and a line file.

> Do not import these files. You only need to select them to enable the Header and Line tabs.

4. Open the tab that contains the column to which you want to link the business rule.
5. Right-click the field to which you want to link your business rule and select DynaChange > DynaChange Rules > **Create Business Rule**. The New Business Rule dialog opens, listing your available business rules.
6. Find the rule you want to apply, and enable the **Selected** check box next to it.
7. On the **Field Selector** tab, select the fields you want to send to your business rule.
8. On the Configuration Options tab, select the rule type you want to use from the Rule Type drop-down field. See "Types of Business Rule Links" on page 48 for more information on rule types.
9. From the Apply Rule On field, select **Import Validator**.
10. Click **Load Selected Fields** to load your fields into the Test Business Rule pane.
11. The Field Value field contains the text that is sent to the rule. Change that data to represent your test data.
12. Click **Run Test**. This button passes the data from your fields to the business rule, runs the algorithm, and passes back the results (if any).
13. If you are finished with the rule, save the rule. This closes the New Business Rule dialog. If you plan to modify permissions or configuration for the rule, you can leave the rule open and continue to the sections on that.

> Test import rules against your test database prior to importing it to your live database.

Link Event-Driven Rules to Prophet 21

Standard rules in Prophet 21 only work when a user interacts with the screen in some way (saves a record, changes text and tabs off a field, opens a window, etc.). The new event-driven rules enable you to trigger a rule when some triggering event occurs in Prophet 21. For example, a rule can be created that triggers when a sales order pick ticket is created.

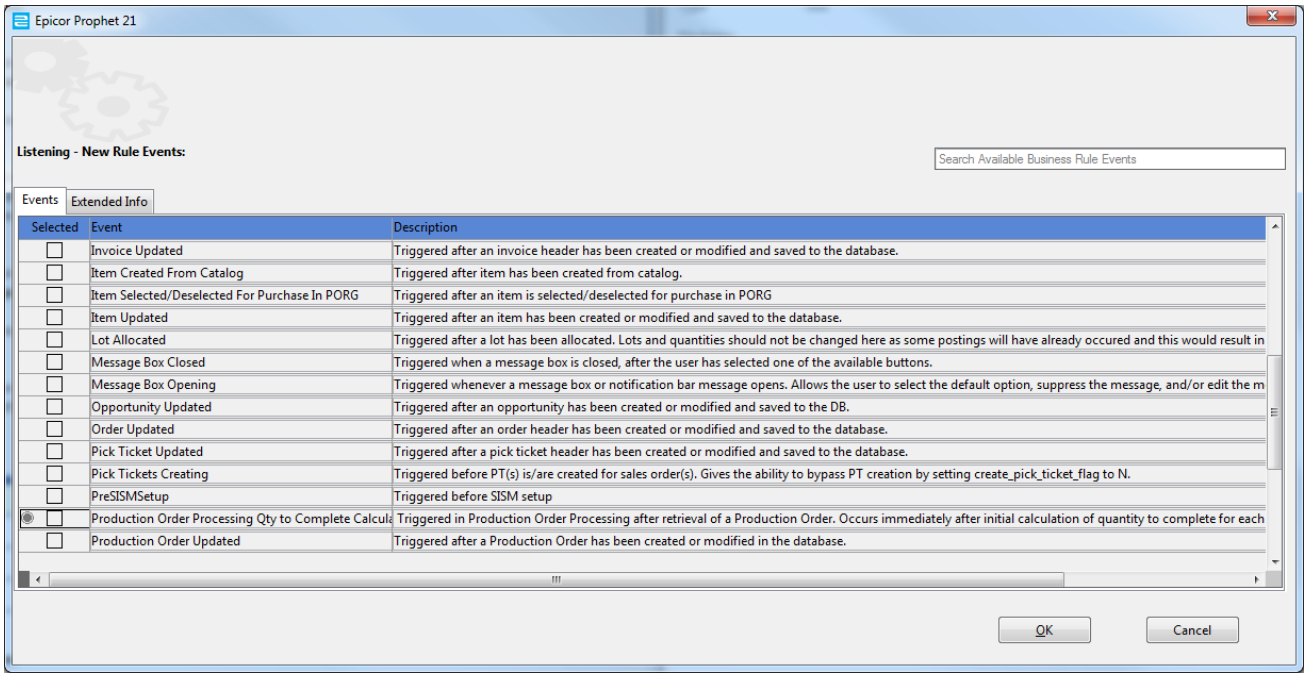
The beginning of the process of linking an event rule to Prophet 21 is of necessity different from that of a normal visual rule; once the initial start has finished, the rest of the process works the same as a visual business rule.

1. Open Prophet 21.
2. Start listening for a rule (Tools > DynaChange > DynaChange Rules > Listen...). There are two options.

   Listen for New Rule - Enables you to set up a new rule (analogous to Create Business Rule for visual rules).

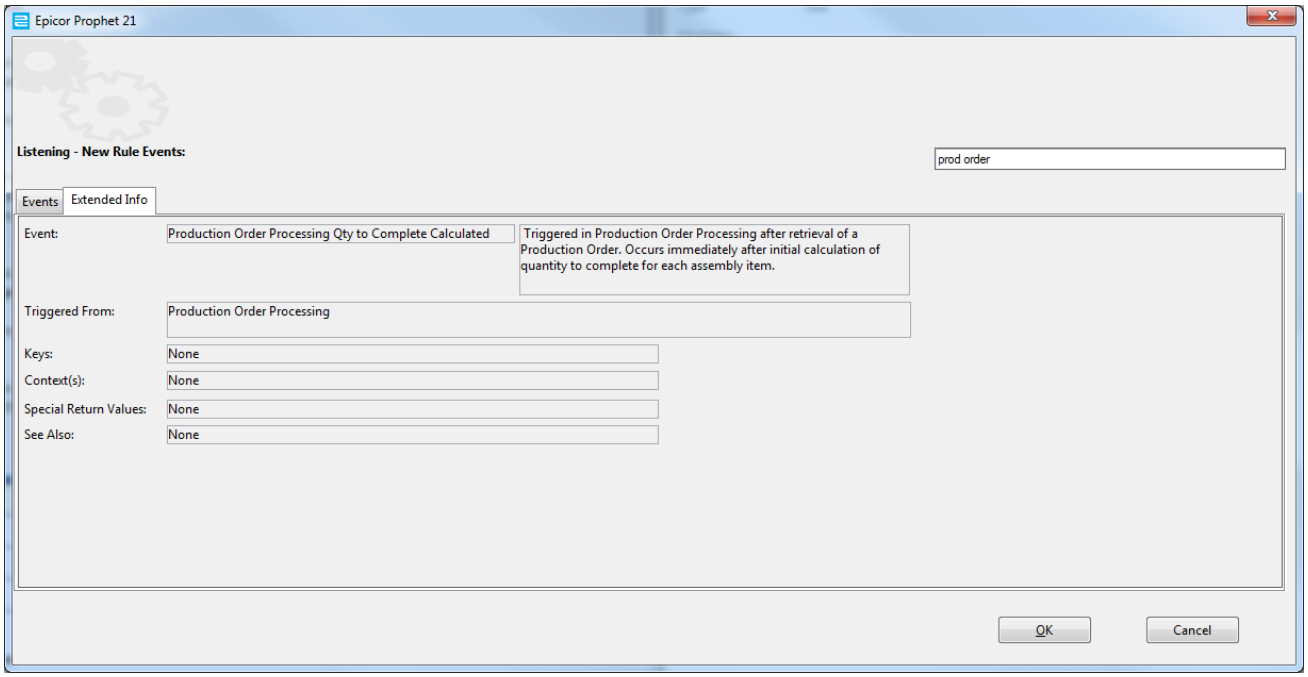
   Listen for Existing Rule - Enables you to modify existing event-driven rules (analogous to Edit Business Rules for visual rules).

   The system then shows a dialog that lets you select the event you want to trigger the rule.



> Use the Search box to search for available rules. Prophet 21 searches the event and description fields to find the rule.

3. Use the Extended Info tab to view more information about the selected rule. Acess the extended tab by either selecting a rule and opening the Extended Info tab, or simply double click the rule description.



4. Enable the **Selected** check box for the event you want to listen for, and then click OK.

> If you enable the Form Datastream Created event, the Import Preprocessing event, or any event that requires and event type, upon clicking OK, Prophet 21 prompts you to select the event type. Select the type and click OK to continue.

5. Within Prophet 21, you must now cause that event. So if the event is creating a pick ticket, you must create a pick ticket. When you do, the system will pop open the normal Create or Edit Business Rule dialog, enabling you to set up your rule as desired.
6. From this point on, linking the rule is the same as it is for visual rules, with the following exceptions. See "Link Business Rules to Prophet 21" on page 50 for more information on the rest of the process of linking rules to Prophet 21.
   - In some case, you see "action" and "key_value" as the triggers on the Field Selector tab. The system automatically populates these values and passes them to the rule so you can determine what you rule needs to do based on them.

     action - The action Prophet 21 is taking: UPDATE or ADD. These values are passed to the rule in case you want to take a different action when updating a record versus inserting a new record.

     key_value - The value that uniquely identifies the record. For example, for an order, it would be the order number.

   - You cannot select the rule type or where the rule applies.

- The rule is always multi-row.
- You can select if the rule is Synchronous or Asynchronous.
    - **Synchronous** - Prophet 21 waits for the rule to respond before it continues processing. If you want the rule to pass data back to the Prophet 21 application after processing, you must use a Synchronous rule. This option works similar to each rule type except for Workflows.
    - **Asynchronous** - Prophet 21 does not wait for or accept any response from the rule; it simply activates the rule then continues normal processing. This type of event rule cannot pass data back to Prophet 21 (though it can of course still manually update the database after the fact using SQL). This option works similar to a Workflow rule type.

7. Once you have finished setting up your rule, the system automatically stops listening.

> If you cancel out of the setup prior to completing it, the system will continue listening until you set it to stop listening (Tools > DynaChange > DynaChange Rules > **Stop Listening**); otherwise, the application will pop up the Create or Edit rule dialog every time you hit that event in Prophet 21.

Link Business Rules to User-Defined Menus

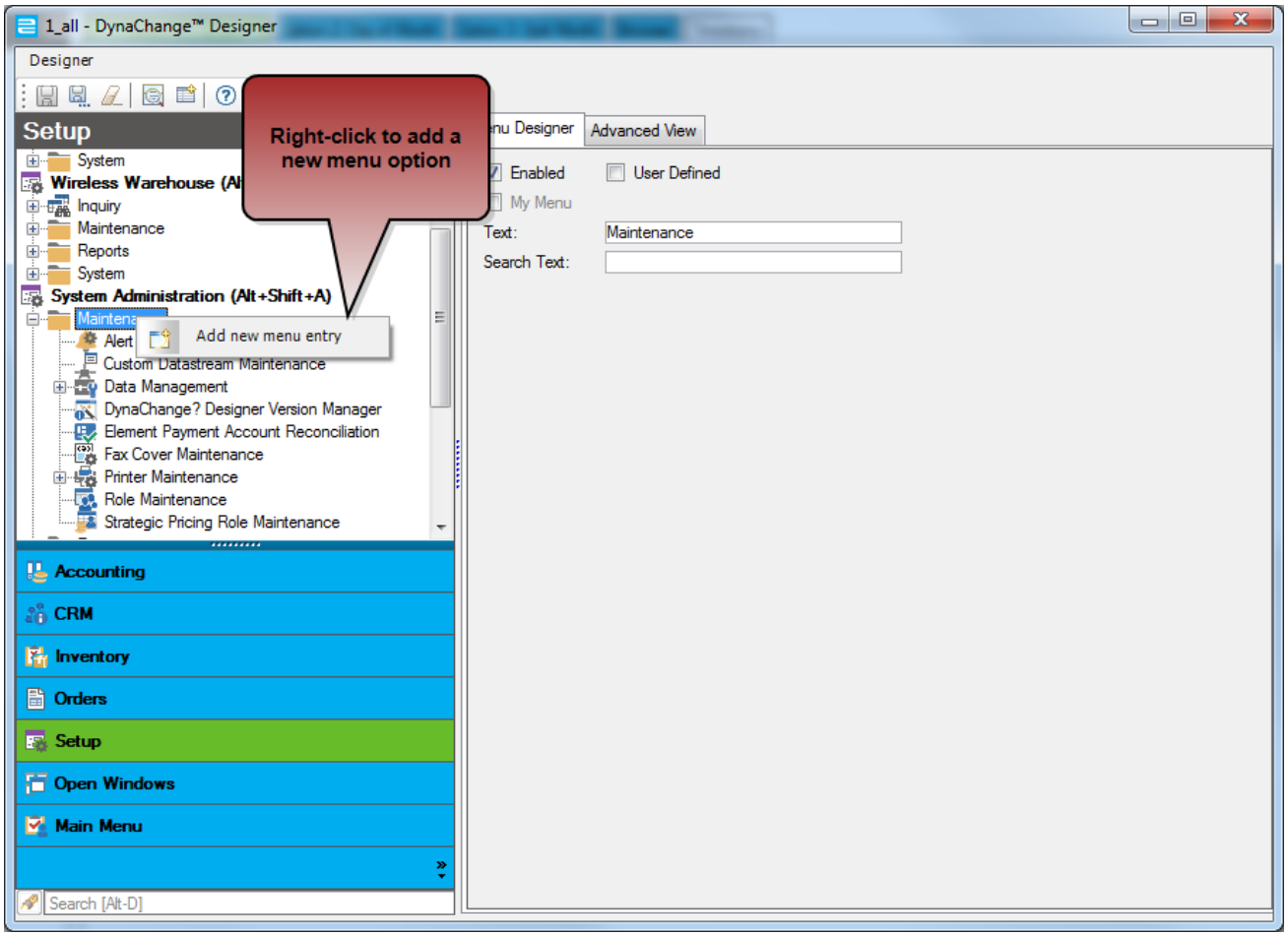
You can use DynaChange ™ Menu Designer to create custom menu options that launch business rules. Creating customer menu options is a multi-step process.

> You cannot currently connect a web visual rule to a menu option.

Create the Option

Create the menu option.

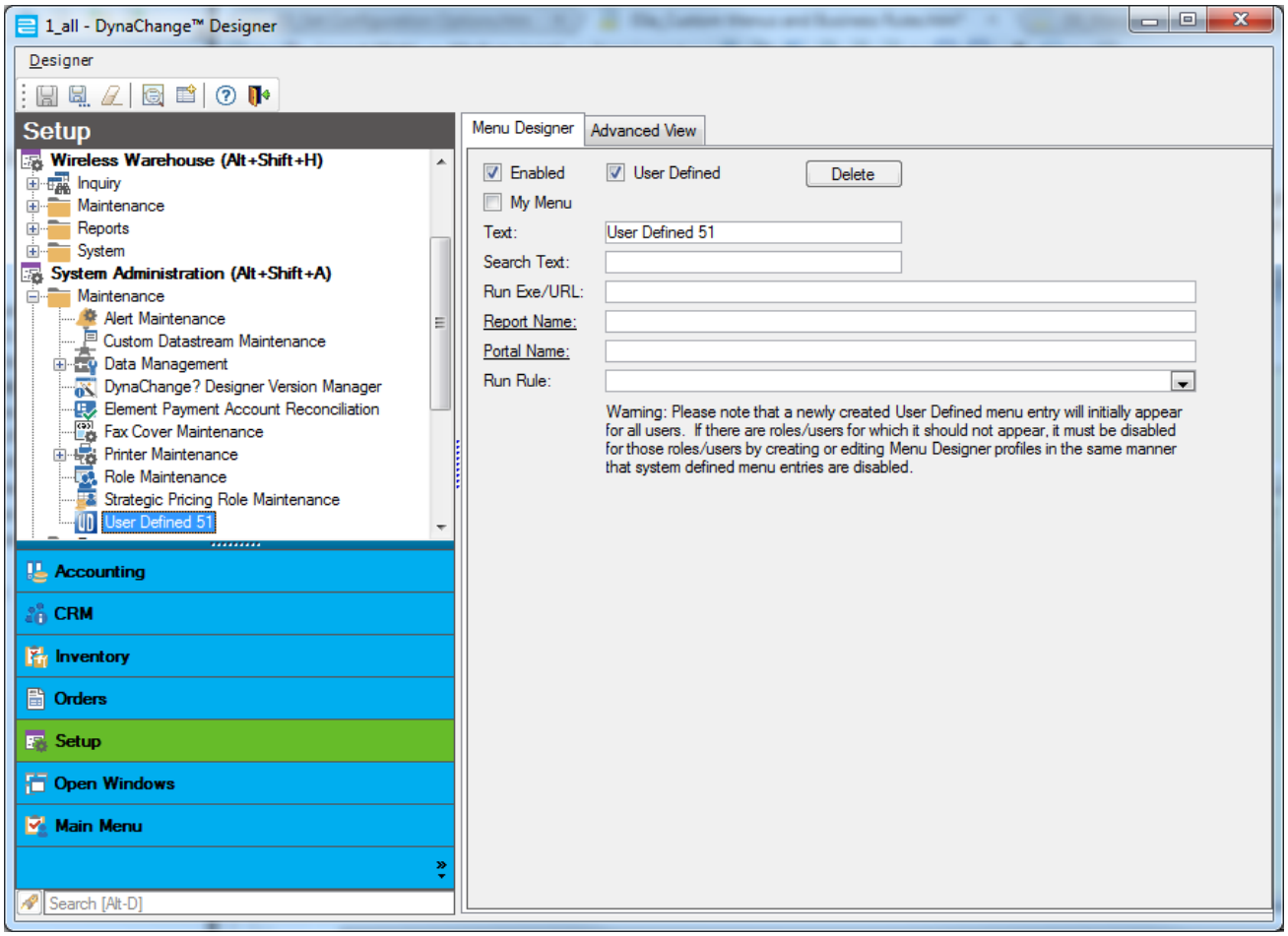
1. In Prophet 21, navigate to the window and tab where you want to link your business rule.
2. Right-click and select DynaChange > Menu Designer.
3. Right click the folder in which you want to add the option and select **Add New Menu Entry.**



The menu options you create this way are “leaf nodes” of the menu; they are options that you click on to run a command or rule. You cannot create folders this way.

Check Permissions

Once you have the menu option added, ensure that the you apply any DynaChange Menu Designer security options you want to it; by default the menu option is added for all users (except users who do not have access to one of the parent menus which contain the new option).

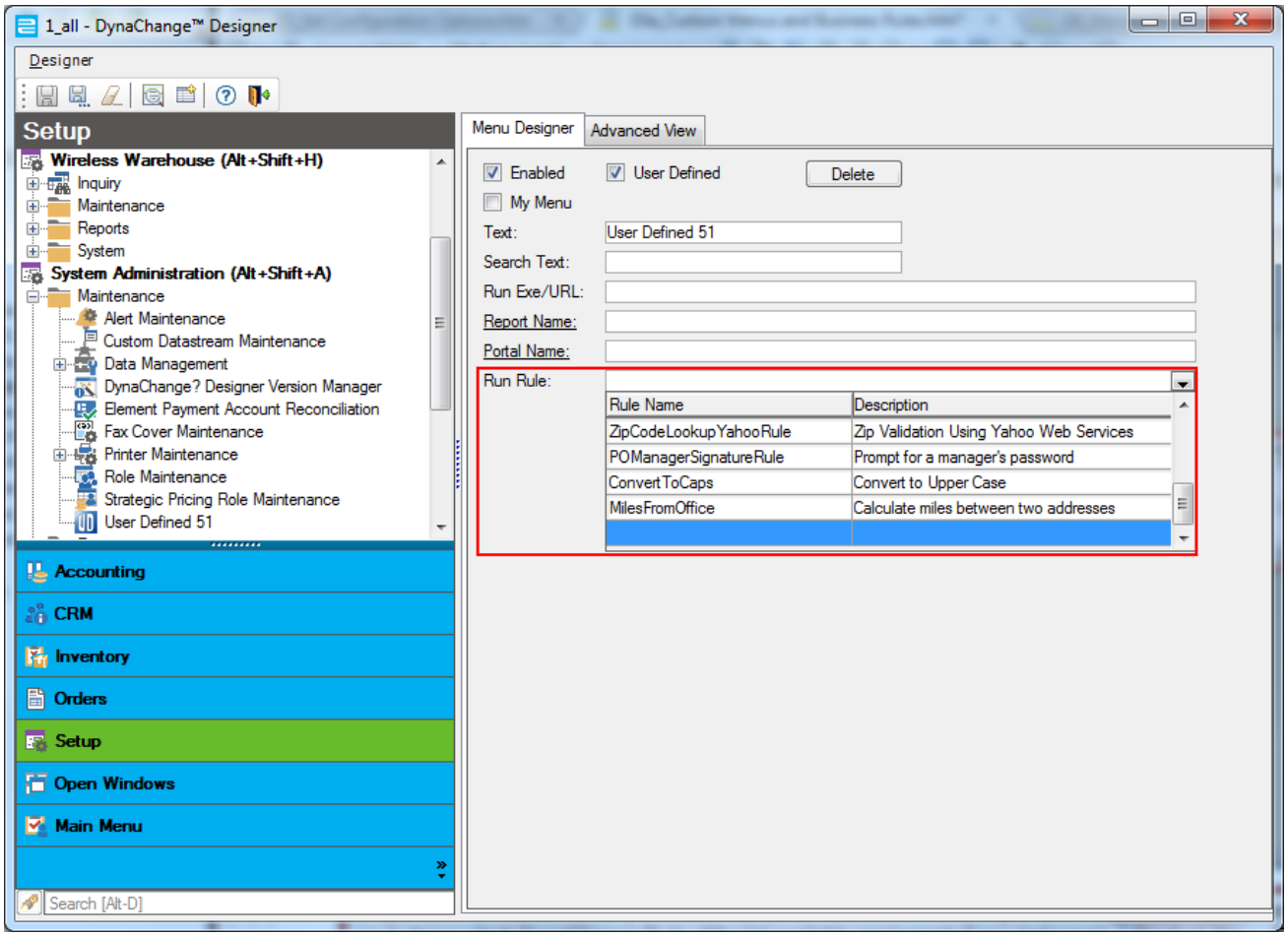


New menu options added are added for ALL roles and users can have access to the parent menu set. Thus if you add a menu option to Accounting > Accounts Receivable > Transaction, any user that has access to the Transaction menu in Accounts Receivable can access the option. If you want to limit access to the new option, you can use DynaChange Menu Designer to remove access.

Link the Rule to the Menu Option

Once you have the menu option, you must link it to the rule you want it to launch. Link a business rule by selecting it from the Run Rule drop down menu.

The information that you enter in the Text and Search Text fields get passed into the rule, allowing the you to identify which menu item triggered the rule.



You can use any workflow rule (which are the ones that implement the ExecuteAsync() method), but it is up to you to know which DynaChange Rule of the list available implements ExecuteAsync(); Prophet 21 performs no validations; it will not stop you from selecting a rule that does not implement ExecuteAsync().

Once you select a rule, you can remove it by selecting the blank option on the drop down (the blank option is at the bottom).

When you click the new menu option, the system fires the DynaChange Rule. It sends the DynaChange Rules global fields (or populated the Session class if you are using P21.Extensions.dll) to the rule and no other fields. See "Session Class" on page 110 for more information on global fields and the Session class.

When the rule opens, the system displays status bar text that indicates that the rule has been launched. The rule cannot return any result to Prophet 21; once the menu option is engaged; the rule is entirely separate from Prophet 21.

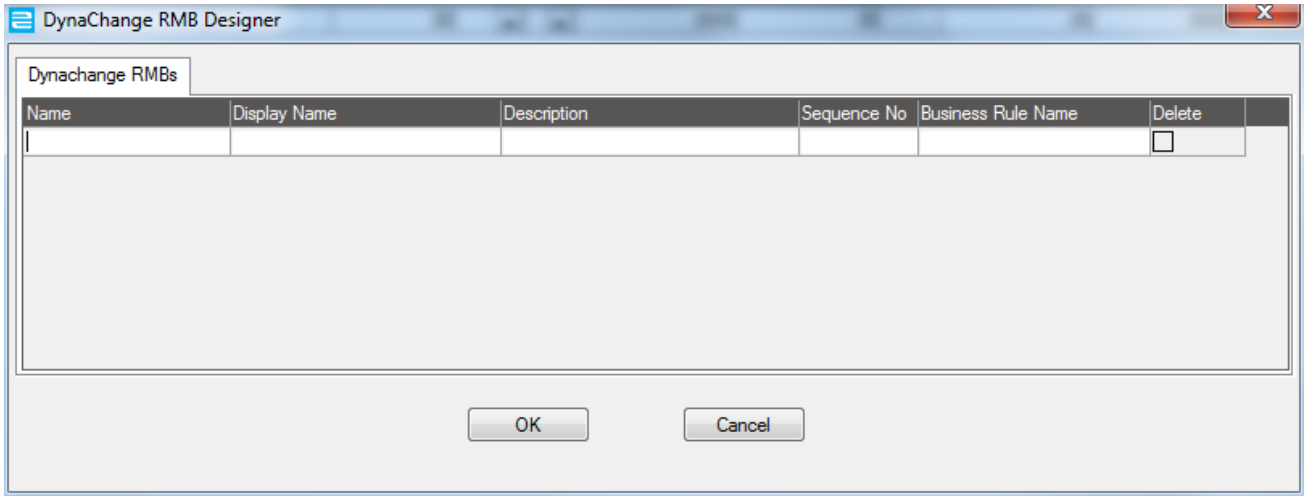
Link Business Rules to Custom Right Mouse Button Menu Options

Use DynaChange™ Create RMB Rules to create custom right mouse button (RMB) menu options that launch business rules.

You can create custom RMB options for any window. That said, you can only have up to five options per window/tab/datawindow combination. This means that if you are working with the Order Entry window, you can create five RMB options for the Order tab, and five more options o the Items tab.

> The limit of five custom right mouse button menu options per window and tab combination is per user, rather it is per window, tab, and datawindow. You can, however, restrict user access to RMB options.

1. Open Prophet 21.
2. Navigate to the window and tab where you want to link your business rule and select DynaChange > DynaChange Rules > **Create RMB Rules**. The DynaChange RMB Designer window opens.



3. In **Name**, enter a name for the right mouse button option rule. This should be something that is easily recognizable and describes which rule or function the right mouse button option performs.
4. In **Display Name**, enter the name of the RMB option as it should appear on the right mouse button menu.
5. In **Description**, enter a brief description of the right mouse button menu option and rule.
6. In **Sequence No**, enter the order in which this option should appear in the right mouse button menu. Remember that you are limited to five options per window/tab/datawindow combination.
7. Double click the **Business Rule Name** field to open the New Business Rule window. Use this window to select a rule to associate with the RMB option, select fields, and configure the rule as

needed. For more information on using the New Business Rule window, see "Link Business Rules to Prophet 21" on page 50.

8. Save the business rule to return to the DynaChange ™ RMB Designer. The selected business rule populates the Business Rule Name field.
9. Click **OK** save the right mouse button menu option and close the DynaChange™ RMB Designer. Now, when a user accesses the right mouse button menu, the new option lists at the top of the menu.

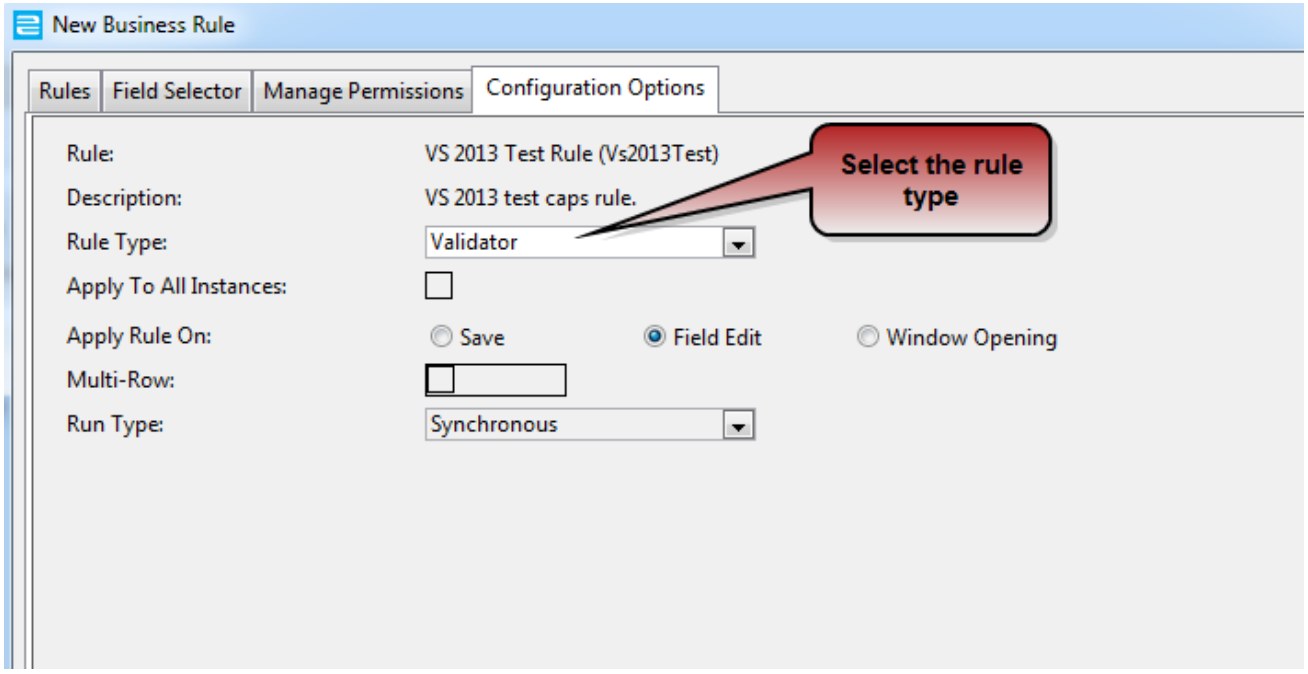
Set Configuration Options

You have several options for configuring your rules.

Set Rule Type

There are several types of business rule links you can use to hook your business rule into Prophet 21. These types of links define *when* a business rule runs and if it returns a value to Prophet 21. See "Types of Business Rule Links" on page 48 for more information on the types.

1. If you are creating a new business rule, create the rule (see "Link Business Rules to Prophet 21" on page 50 for more information). If you are editing an existing business rule, select the rule to edit (see "Edit Business Rule Links" on page 72 for more information). The business rule appears in either the New Business Rule or Edit Business Rule dialog.
2. Click the **Configuration Options** tab.

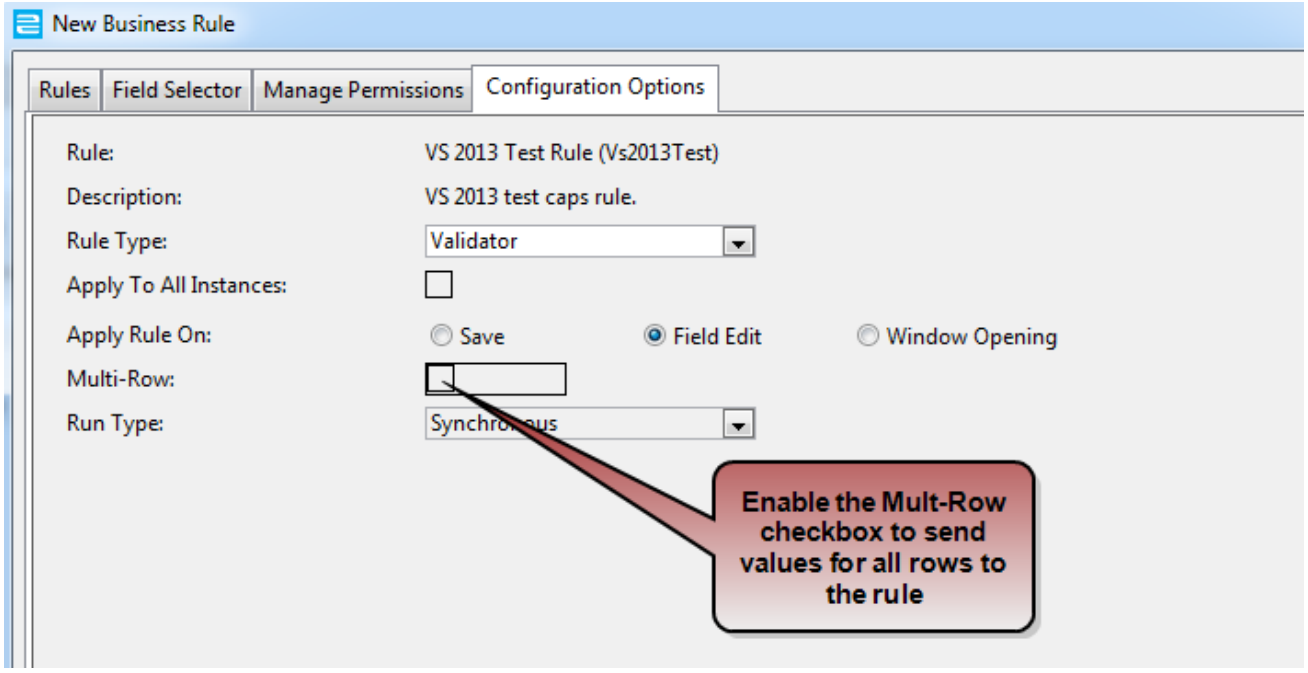


3. Select the rule type from the **Rule Type** drop-down.
4. Save the rule.

Set a Rule as Multi-Row

You can allow a rule to act on and pass multiple rows of data. For example, you may want to have a rule that sets the allocated quantity of all order lines to zero and sets their disposition to hold. Sending multiple rows will pass all rows to the corresponding data fields you choose to your business rule.

1. If you are creating a new business rule, create the rule (see "Link Business Rules to Prophet 21" on page 50 for more information). If you are editing an existing business rule, select the rule to edit (see "Edit Business Rule Links" on page 72 for more information). The business rule appears in either the New Business Rule or Edit Business Rule dialog.
2. Click the **Configuration Options** tab.



3. Enable the **Multi-Row** check box.
4. Save the rule.

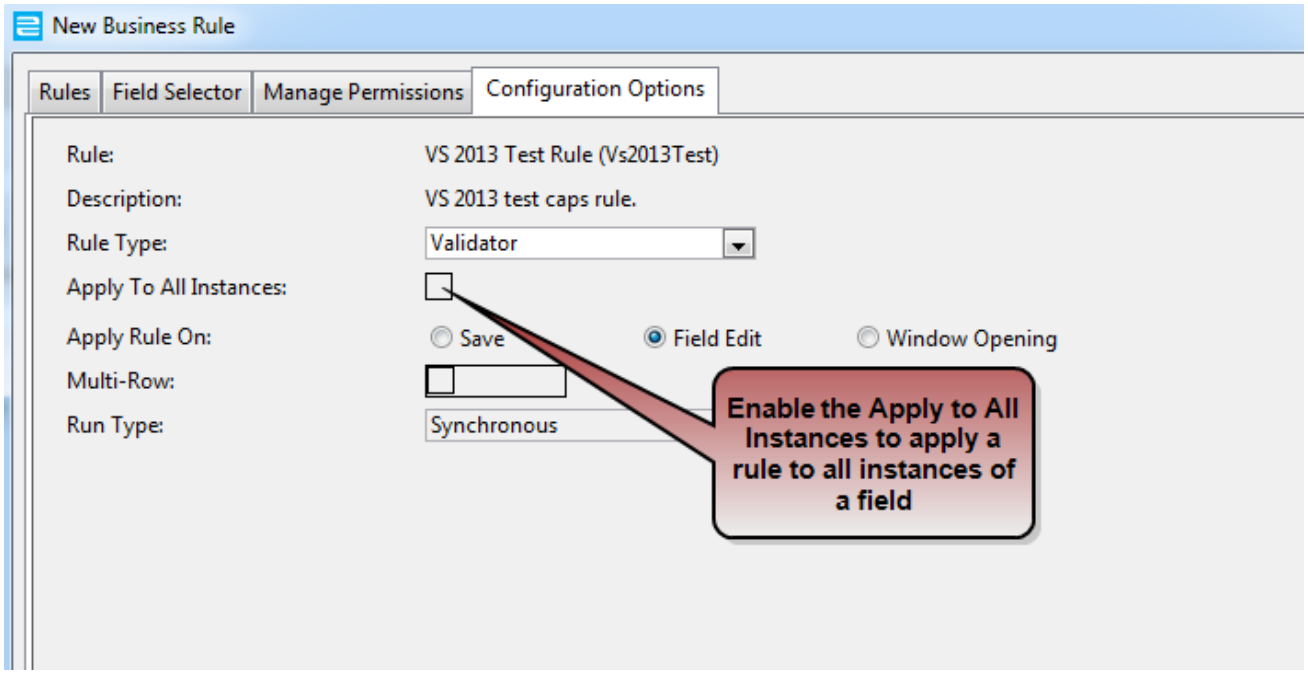
Apply a Rule to All Instances of a Field

You can apply a business rule to all instances of a field in Prophet 21. For example, you may want to have a rule that formats phone numbers as (xxx) xxx-xxxx, and not xxxxxxxxxx or xxx.xxx.xxxx so all phone number formats are consistent. Rather than manually applying the rule to every phone number field in Prophet 21, you can apply a rule to all fields with the same field name as the current field (the

field you right-clicked to create the business rule; you can find the field name for this field by right-clicking the field, selecting Help > SQL Information; the name of the field is listed at the bottom of the popup).

> Particularly for fields like phone numbers and addresses, not all fields are named the same in Prophet 21, because there are different types of phone numbers and addresses. If you want the rule to work for all those fields, you must repeat the following process for each type of the field.

1. If you are creating a new business rule, create the rule (see "Link Business Rules to Prophet 21" on page 50 for more information). If you are editing an existing business rule, select the rule to edit (see "Edit Business Rule Links" on page 72 for more information). The business rule appears in either the New Business Rule or Edit Business Rule dialog.
2. Click the **Configuration Options** tab.



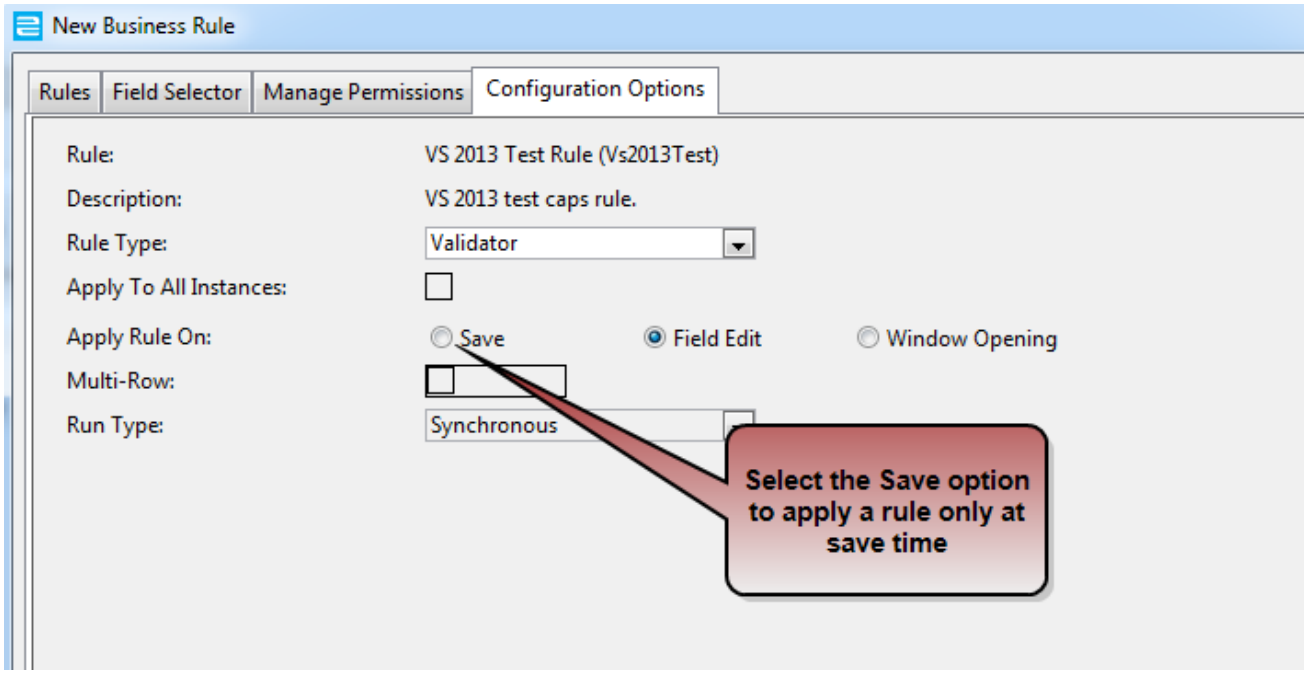
3. Enable the **Apply To All Instances** check box.
4. Save the rule.

Apply a Rule to the Save Process

You can apply a business rule to the save process, so that instead of running the business rule when the field loses focus (i.e., you tab off of the field, or click on a different field), Prophet 21 runs the business rule when you save the record. For example, if the payment type for an order is a credit card, and the user did not enter the full amount of the invoice for payment, prompt the user if they want to enter the

full amount or not. You only want to run this validation during the save process, so you must apply the rule to save.

1. If you are creating a new business rule, create the rule (see "Link Business Rules to Prophet 21" on page 50 for more information). If you are editing an existing business rule, select the rule to edit (see "Edit Business Rule Links" on page 72 for more information). The business rule appears in either the New Business Rule or Edit Business Rule dialog.
2. Click the **Configuration Options** tab.



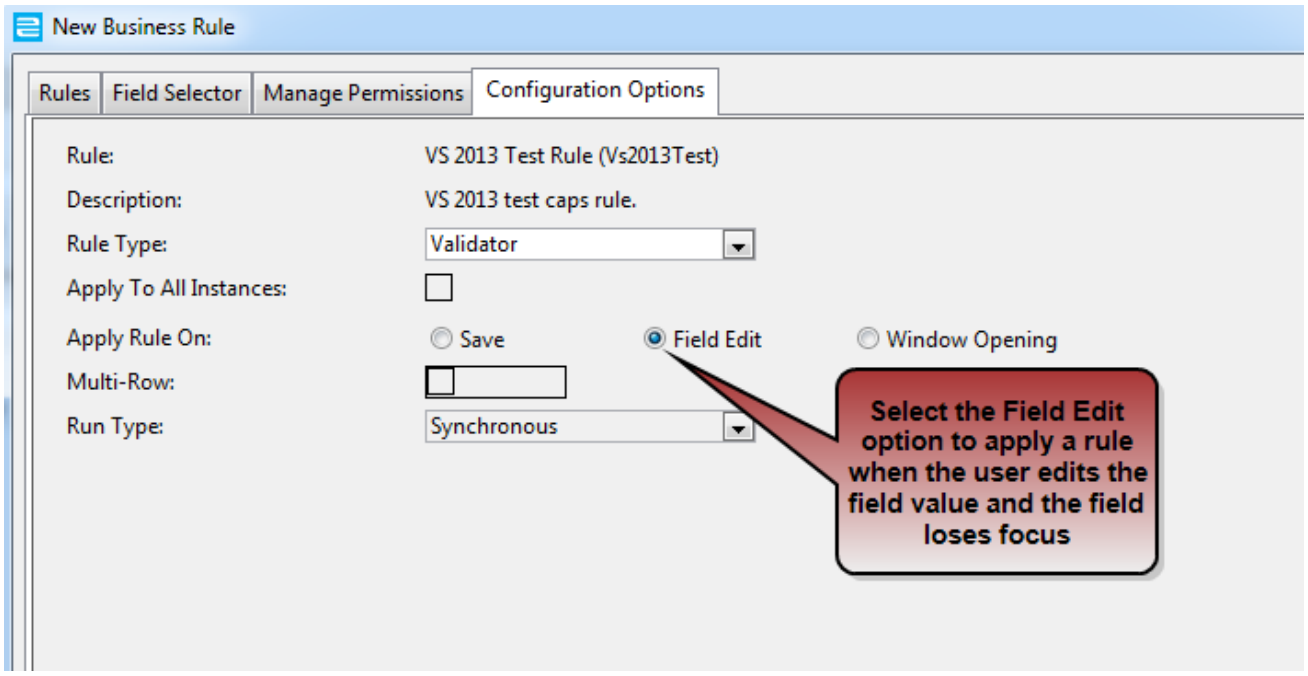
3. Select the **Save** option.
4. Save the rule.

Apply a Rule to the Field Edit Process

You can apply a business rule to the field edit process, so that instead of waiting for the window to save, Prophet 21 runs the business rule when the field loses focus (i.e., you tab off of the field, or click on a different field). For example, you want all customer names recorded in all caps, so you create a rule to convert text entered in these fields to all capital letters as soon as someone tabs off the field. Since this happens as soon as someone changes a field value, you must apply the rule to field edit.

1. If you are creating a new business rule, create the rule (see "Link Business Rules to Prophet 21" on page 50 for more information). If you are editing an existing business rule, select the rule to edit (see "Edit Business Rule Links" on page 72 for more information). The business rule appears in either the New Business Rule or Edit Business Rule dialog.

2. Click the **Configuration Options** tab.

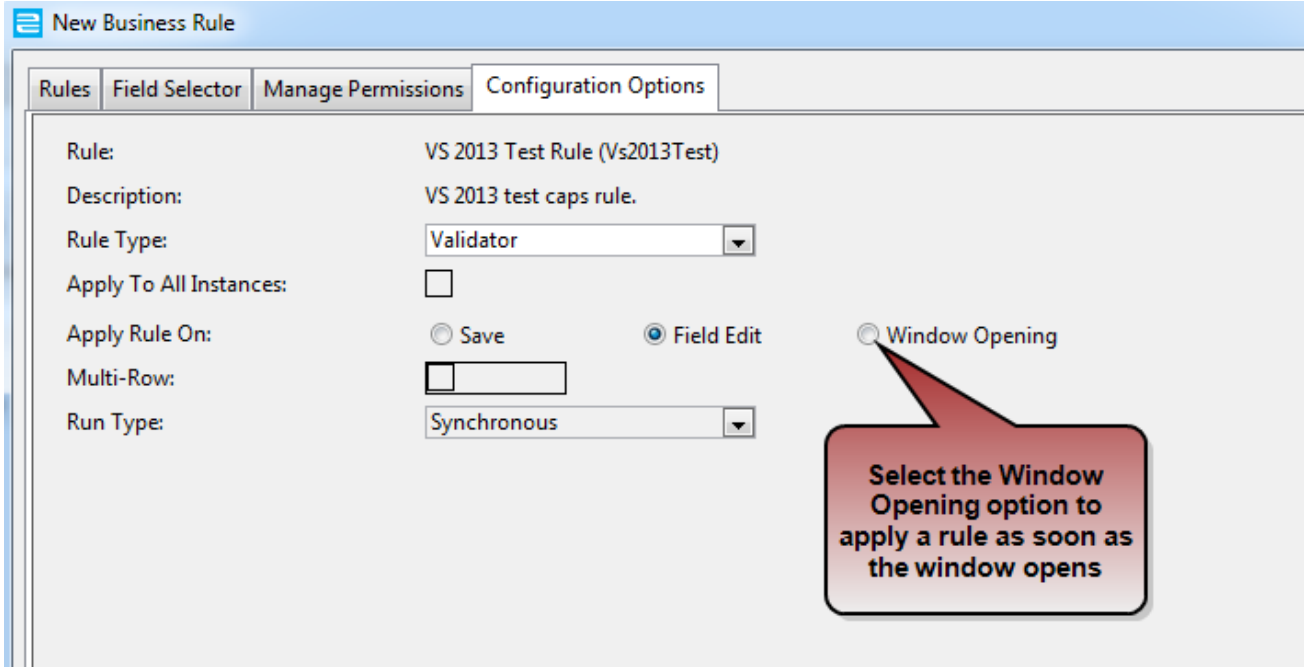


3. Select the **Field Edit** option.
4. Save the rule.

Apply a Rule at Window Opening

You can apply a business rule to the opening of a new window, so that Prophet 21 runs the business rule as soon as you open a particular window. For example, you want a report window to default certain criteria values as soon as you open it so that you don't have to key them in every time, so you create a rule to enter these values in the fields every time the window opens.

1. If you are creating a new business rule, create the rule (see "Link Business Rules to Prophet 21" on page 50 for more information). If you are editing an existing business rule, select the rule to edit (see "Edit Business Rule Links" on page 72 for more information). The business rule appears in either the New Business Rule or Edit Business Rule dialog.
2. Click the **Configuration Options** tab.



3. Select the **Window Opening** option.
4. Save the rule.

Apply a Rule On Demand

An on demand rule only triggers when you tell it to. In this case, you use DynaChange to create a button on the screen, and tie the rule to that button. The rule triggers whenever you click the new button, but only then. Without the manual trigger, the rule never fires. There are two parts to this procedure.

First, you must create the picture button in Prophet 21. For this, you use the DynaChange Screen Designer.

1. Open the Screen Designer.
2. From the Insert menu, select **Picture Button**.
3. A popup appears asking you to identify the image that will be used for the button. Enter the file name or browse for it and click **Open**.

> Images must be bitmaps.

You can view the image button in Screen Designer, but it appears as an area with the text Bitmap(file name). This is because the image is not actually inserted into the application; it is a computed field.

> Because the picture button is a linked image, not an inserted one, be sure to place the image in a network location that all machines with this screen edit have access to. In addition, be aware that moving the storage location of the image will break these links.

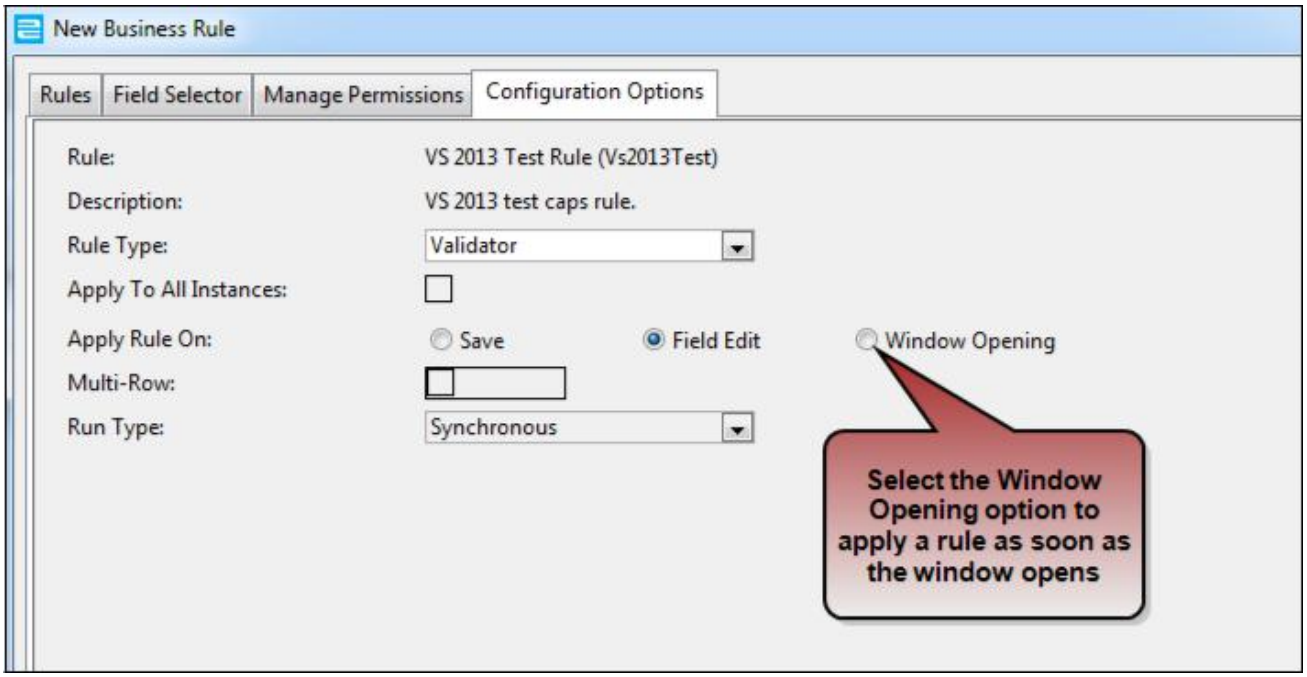
Once you have created the button, you must link the business rule you want it to execute to it.

1. Right click the image button and select **DynaChange > DynaChange Rules > Create Business Rule**.
2. If you are creating a new business rule, create the rule (see "Link Business Rules to Prophet 21" on page 50 for more information). If you are editing an existing business rule, select the rule to edit (see "Edit Business Rule Links" on page 72 for more information). The business rule appears in either the New Business Rule or Edit Business Rule dialog.
3. Click the **Configuration Options** tab.
4. Configure the rule to be **On Demand** in the Rule Type drop-down field.
5. Save the rule.

Enabled for Version

You can determine if a rule will trigger in the web application, in the desktop application, or in both. Use this setting if you are creating web visual rules and you do not want them to trigger from the desktop, or if you have some visual rules using Windows Forms or other technologies that are not compatible with the web application and you do not want the rule to trigger on the web application.

1. If you are creating a new business rule, create the rule (see Link Business Rules to Prophet 21 on page 41 for more information). If you are editing an existing business rule, select the rule to edit (see Edit Business Rule Links on page 63 for more information). The business rule appears in either the New Business Rule or Edit Business Rule dialog.
2. Click the Configuration Options tab.

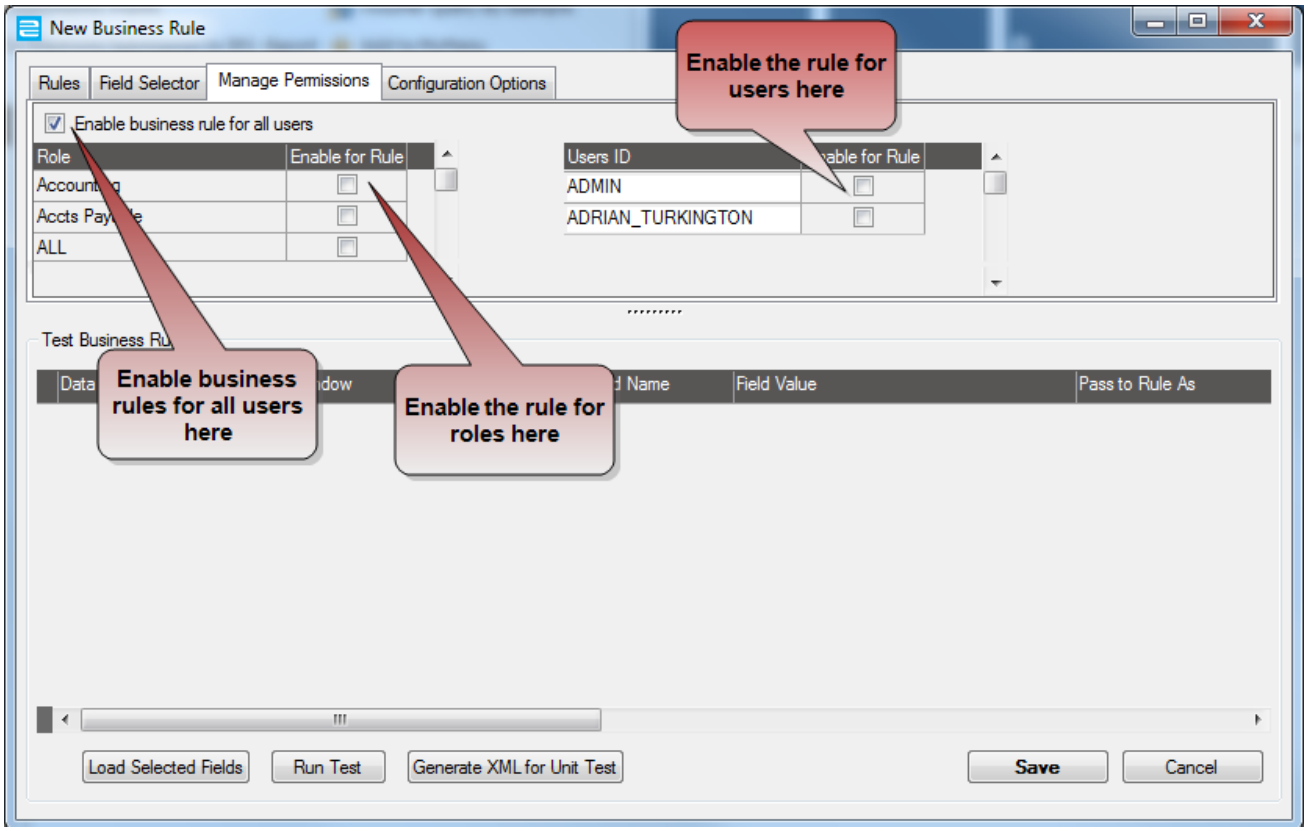


3. Select the desired option from the Enabled for Version drop-down field.

Manage Rule Permissions

You can manage permissions for a rule so that the rule only applies to some users and roles.

1. If you are creating a new business rule, create the rule (see "Link Business Rules to Prophet 21" on page 50 for more information). If you are editing an existing business rule, select the rule to edit (see "Edit Business Rule Links" on the next page for more information). The business rule appears in either the New Business Rule or Edit Business Rule dialog.
2. Click the **Manage Permissions** tab. By default, the Enable Business Rule for All Users check box is enabled, so your rule applies to all users.



3. If you want to set the rule to work for only roles and users you specify, disable the **Enable Business Rule for All Users** check box.
4. Enable the **Enable for Rule** check box for any role or user who should use the business rule.
5. Save the rule.

Edit Business Rule Links

You can edit existing links to business rules. There are two approaches; one lets you edit only the rule type and configuration options, and the second lets you edit everything about a business rule.

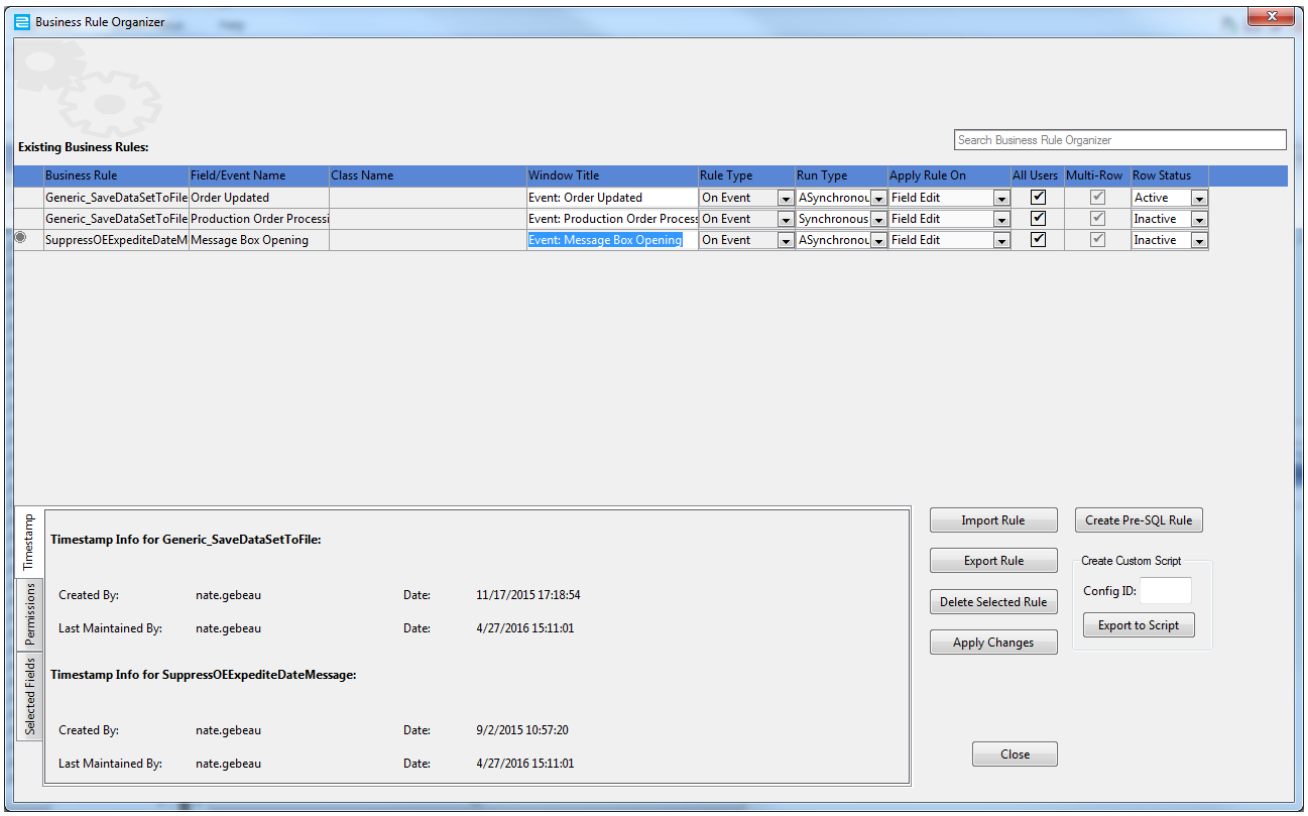
Edit Rule Type and Configuration Options

You can edit the rule type and configuration options for your rule using the Business Rule Organizer. You can also right-click a field that contains a rule to edit it (see "Edit Rule Link, Including Selected Fields" on page 74 for more information).

Business rules linked to imports cannot be edited using the following procedure. Please refer to "Edit Business Rules Linked to Imports" on page 75 for more information.

1. Open the Business Rule Organizer (Application > Rules). The dialog shows you all of the business rules that are currently applied in your system. The Field Name is the field to which the rule is applied, and the Class Name is the object on the tab to which it has been applied.

If the rule has the Apply to All Instances check box enabled, the class name is blank, as the rule applies to the listed field name in all objects.



2. You can change:
   - **Rule Type** (see "Types of Business Rule Links" on page 48)
   - **Apply During Save** (see "Apply a Rule to All Instances of a Field" on page 65)
   - **Apply to All Instances** (see "Apply a Rule to the Save Process" on page 66)
   - **Row Status** (see "Remove Business Rules" on page 75)
   - **Permissions** by clicking the Permissions tab (see "Manage Rule Permissions" on page 71)
3. You can also review the fields currently selected for your rule by clicking the Selected Fields tab.

4. Once your changes are complete, click **Apply Changes**.
5. Click **Close** to close the dialog.

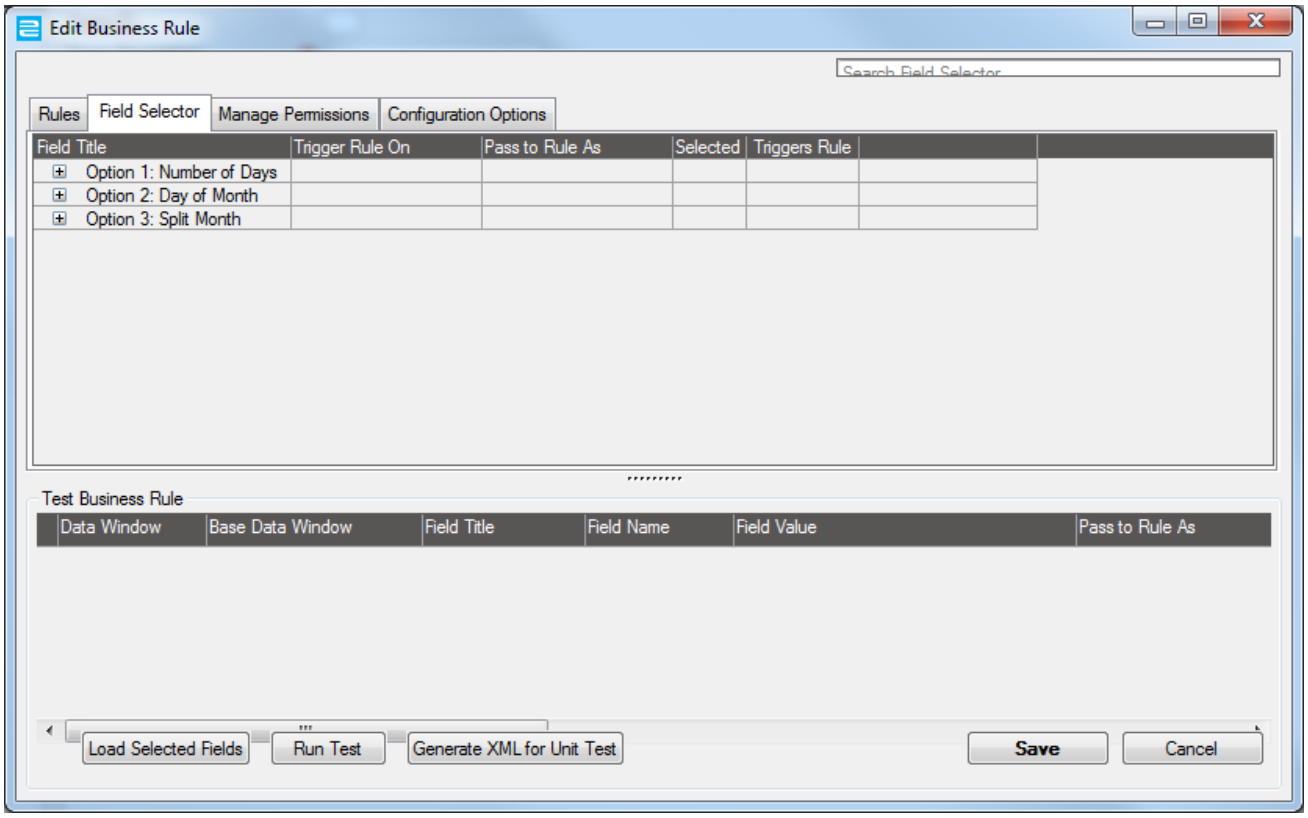
Edit Rule Link, Including Selected Fields

You can edit all information about a business rule link by right-clicking the field the rule is attached to.

> You cannot edit a business rule that has been applied to all instances using this method. Use the method detailed above, or create a new rule link if you need to edit the selected fields.

1. Open the window and tab that contains the field to which the rule has been applied.
2. Right-click the field the rule was applied to and select DynaChange > DynaChange Rules > **Edit Business Rule**. The Business Rule Editor opens, listing only the rule applied to this field.

> If there are multiple rules applied to the field, the Business Rule Organizer opens, and you must select the rule you want to edit and click Edit Selected Rule.



3. Edit the rule:
    - Change the rule type (see "Types of Business Rule Links" on page 48)
    - Select additional or different fields (see "Link Business Rules to Prophet 21" on page 50)
    - Manage rule permissions (see "Manage Rule Permissions" on page 71)
    - Manage configuration options (see "Set Configuration Options" on page 64)
4. Click **Load Selected Fields**. The fields load in the Test Business Rule pane.
5. Modify any field values and click **Run Test** to ensure the rule operates correctly.
6. Select **Save**.

Edit Business Rules Linked to Imports

You can edit all information about a business rule link by right-clicking the field the rule is attached to.

1. In Prophet 21, open the import window and tab that contains the field to which the rule has been applied.
2. Right-click the field the rule was applied to and select DynaChange > DynaChange Rules > **Edit Business Rule**. The Business Rule Editor opens, listing only the rule applied to this field.

    If there are multiple rules applied to the field, the Business Rule Organizer opens, and you must select the rule you want to edit and click Edit Selected Rule.

3. Edit the rule:
    a. Change the rule type (see "Types of Business Rule Links" on page 48)
    b. Select additional or different fields (see "Link Business Rules to Prophet 21" on page 50)
    c. Manage rule permissions (see "Manage Rule Permissions" on page 71)
    d. Manage configuration options (see "Set Configuration Options" on page 64)
4. Click **Load Selected Fields**. The fields load in the Test Business Rule pane.
5. Modify any field values and click **Run Test** to ensure the rule operates correctly.
6. Select **Save**.

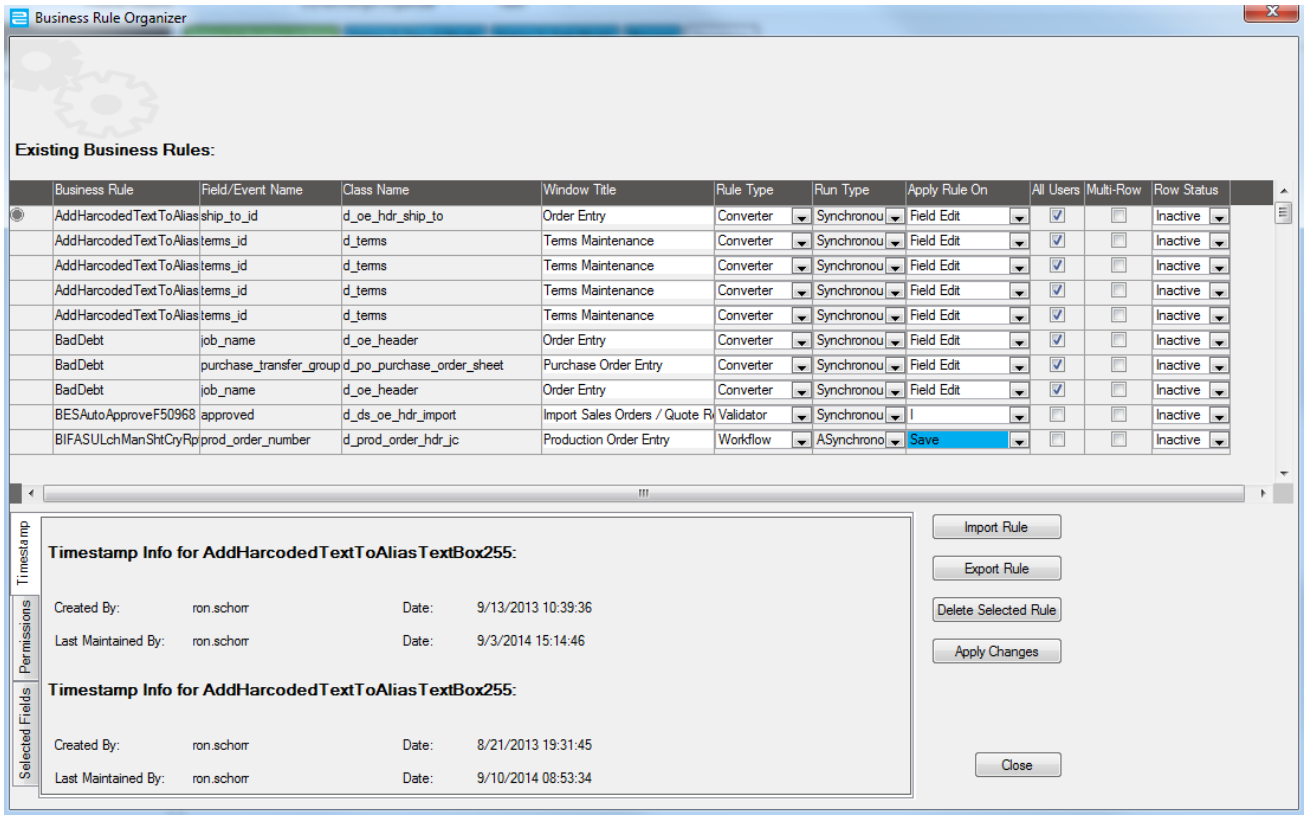
Remove Business Rules

You can remove business rules by either deleting them or deactivating them. Which you choose depends on why you want to remove them.

Deactivate Business Rules

Deactivate a business rule if you want to temporarily remove it from processing in the system. Typically, you would deactivate a rule if it is causing problems (it has a bug that is interfering with your business processes), and you are trying to fix it.

1. Open the Business Rule Organizer (Application > Rules). It lists all of your active and inactive business rules.



2. In the Row Status field, change the status to **Inactive**.
3. Click **Apply Changes**.
4. Click **Close** to close the dialog.

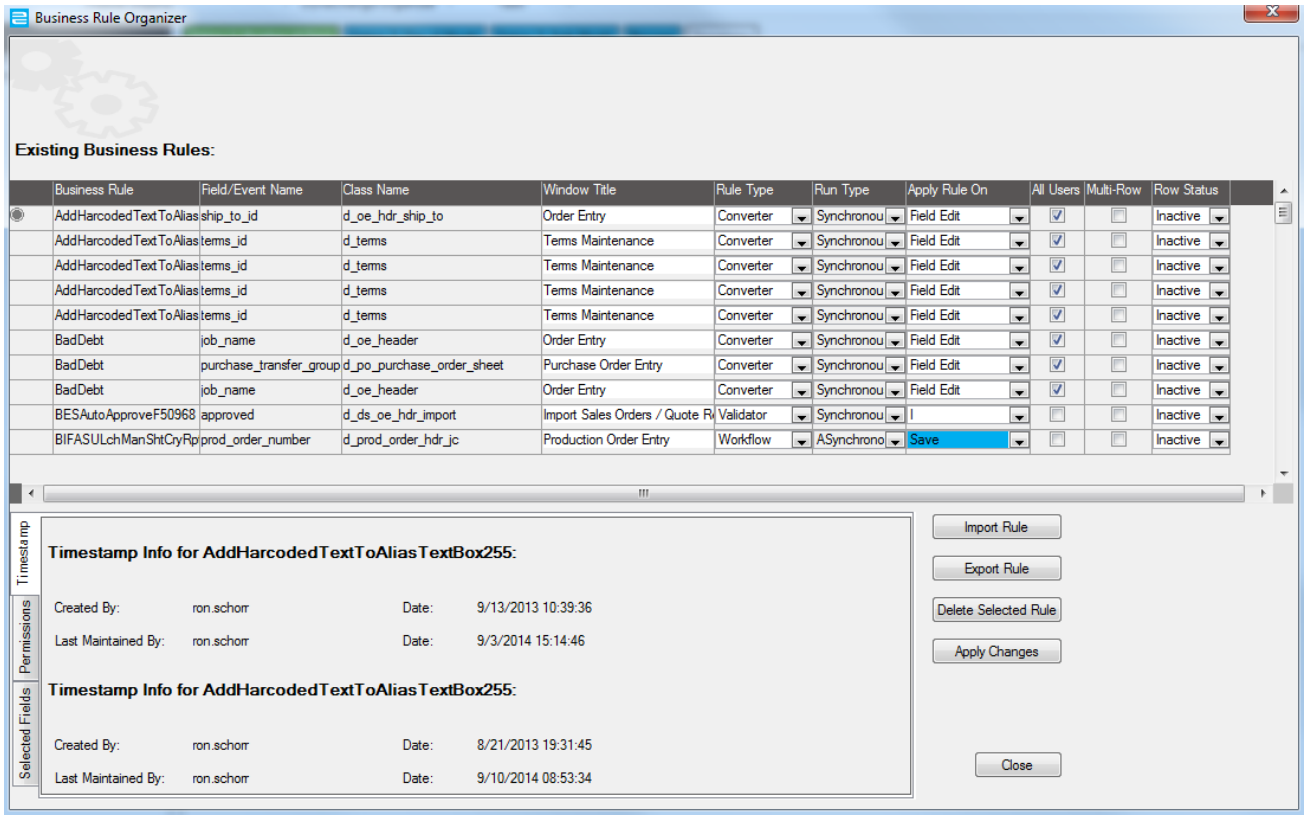
To reactivate the rule, change the Row Status back to Active and click Apply Changes again.

Delete Business Rules

Delete a business rule if you know you no longer need the rule. For example, you changed your business processes, and the rule is obsolete.

> Deleting the business rule removes the link information from your database completely, including all of the settings, permissions, and selected fields. The only way to get that information back is to restore from a backup.

1. Open the Business Rule Organizer (Application > Rules). It lists all of your active and inactive business rules.



2. Enable the **Selected** check box for the rule you want to delete.
3. Click **Delete Selected Rule**. The rule is deleted from the database immediately; you not have to click Apply Changes. Once you click this button, the rule is gone.
4. Click **Close** to close the dialog.

Share Business Rules

You can share business rules among databases, or among other distributors who have DynaChange Rules. Sharing a rule means sharing both the .dll file that contains the rule, and sharing the link between the rule and Prophet 21. To share a rule, you must export it from system where it currently resides, and import it into the system you want to apply it to.

There are some limitations to rule import / export:

- When you export a rule, the system includes the entire DLL file that contains the rule, and any other rules that the DLL may contain. You cannot export one rule at a time unless you confine your rules to one DLL each.
- If you link a rule to a user-defined column, the system importing the rule must also contain the same column, or the rule will not work properly.
- Permissions are not exported with rules. You must set new permissions on any imported rule.
- You cannot share web visual rules using this process.

Export DynaChange Rules

You can export a rule using the Business Rule Organizer.

1. Open the Business Rule Organizer (Application > Rules).
2. Select the rule you want to export.
3. Click the **Export Rule** button. The system exports the files to a subfolder of your export directory (the export folder you created in System Settings). A Windows Explorer window opens, displaying the exported files (the DLL file, and one or more XML files).
4. Send the files to where they need to go. In the case of moving between databases on the same system, you can leave them where they are and import them. In the case of moving them between systems (such as sharing with another distributor), zip them up before sending.
5. Click **Close** to exit the Business Rule Organizer.

> You may want to delete the exported rule folders when you are finished with them; otherwise, they remain in the export folder. The exported rule folders are named "Distribution_X" where X is a number.

Import DynaChange Rules

You can import a rule using the Business Rule Organizer.

1. If you received a zipped copy of the rules files, unzip it to a location where Prophet 21 can find it.
2. Open the Business Rule Organizer (Application > Rules).
3. Click the **Import Rule** button. The Specify XML Files for Business Rule dialog opens.
4. Click the browse button (**...**) for the Business Rule File field and find the businessRule.xml file.
5. Highlight the file and click **Save**.
6. Repeat Steps 4 and 5 for the Business Rule Detail File field (and instead of businessRule.xml, use businessRuleDetail.xml).

7. Repeat Steps 4 and 5 for the Custom DLL File field (and instead of businessRule.xml, use the DLL file for the rule).
8. Click **OK** to import the rule. Prophet 21 copies the DLL file to your rules DLL folder, and updates the business rule and business rule detail information in the database.

> If a DLL file of the same name already exists in your DLL folder, a command prompt window prompts you to overwrite it.

9. The imported rule is now visible in the Business Rule Organizer. Select the rule and add any necessary permissions on the Permissions tab.
10. Click **Close** to exit the Business Rule Organizer.

Troubleshooting Rules

This chapter can help you resolve some common problems you may encounter when using the DynaChange Rules.

General Troubleshooting **80**

Prophet 21 Troubleshooting **82**

Developer Troubleshooting **85**

Web Visual Rule Troubleshooting **88**

General Troubleshooting

There are two tools you can use for general troubleshooting tasks: the LogString class and the business_rule_log table. Use the LogString class to write log files from a rule to pinpoint issues that may occur within the rule (see "LogString Class" on page 102 for more information). Use the business_rule_log table to determine if there is a problem with the data getting sent to the rule from Prophet 21, or if there is a problem with the data returned to Prophet 21 from the rule.

To use the business_rule_log table, you must be logged into Prophet 21 as a programmer.

1. Open the login popup as you normally would to log into Prophet 21.
2. Instead of clicking OK to log in, press **alt - T**. The Testing Options tab appears.
3. Enable the **Run as Programmer** check box.
4. Click **OK**. The system will prompt you for the login override password. Once you have entered that, you will log in as normal, but you will be in programmer mode, which turns on business_rule_log table in Prophet 21.
5. Complete the task that causes the rule to trigger. When the rule triggers, the system writes logging information to the business_rule_log table.

The following table explains the fields available in business_rule_log and how to use them.

Field:	Description:
business_rule_log_uid	The ID column for the log. As each rule trigger and return happens in Prophet 21, the system will write new rows to the table and each has a UID. Order by this column if you need to see the chronological order in which fields were updated by the rule.

Field:	Description:
user_id	The user who triggered the rule.
log_action	This is what type of logging was done. There are several possible values: • **Invoke** - This action occurs when the rule is first triggered. • **Return** - This action occurs when results are returned from the rule. • **Update** - This action occurs for each field the rule updates. For each field updated, the system adds a row to the table.
rule_name	The name of the rule from its GetName() method.
rule_manager_name	The name of the managing DLL governing this rule. The DLL will be either Activant.P21.Extensions.dll or P21.Extensions.dll.
rule_assembly_name	The name of the DLL file that contains the rule's logic.
run_type	Determines if a rule is Synchronous (the system waits for the rule to respond before it continues processing), or Asynchronous (the system does not wait for or accept any response from the rule and simply activates the rule then continues processing).
xml	The XML string sent from Prophet 21 to the rule or sent from the rule back to Prophet 21. If the log_action is Invoke, then xml contains the XML data sent to the rule from Prophet 21. If the log_action is Return, then xml contains the XML data sent from the rule back to Prophet 21.
return_value	Shows if the rule returned as Success or Fail (the RuleResult Success property).
return_message	The message returned from the rule (using the RuleResult Message property).
update_class_name	The Data Window that contains the field being updated. Only populated for the "Update" log_action, and there will be a row for each field updated.

Field:	Description:
update_row_id	The ID for the row being updated, when updating multiple rows in a multi-row rule. Only populated for the "Update" log_action, and there will be a row for each field updated.
update_row_number	Row number when updating multiple rows in a multi-row rule. Only populated for the "Update" log_action, and there will be a row for each field updated. Note that this field is intended for Epicor use and is not intended for end user troubleshooting.
update_field_name	The name of the field getting updated. Only populated for the "Update" log_action, and there will be a row for each field updated.
update_field_value	The value being updated into the field listed in update_field_name. Only populated for the "Update" log_action, and there will be a row for each field updated.

Prophet 21 Troubleshooting

One of the first Prophet 21 troubleshooting techniques if you are encountering problems and you are not sure if they may be caused by a business rule, or by DynaChange Rules in general, you can disable it for a session by pressing Alt - T at the login dialog box and enabling the Disable DynaChange Designer check box.

When I right-click and select DynaChange > DynaChange Rules, there is no option to create a business rule.

There are a few possible problems:

1) Your user is not allowed to create business rules. Set your user to allow creation of business rules on the Application Security tab in User Maintenance (if you do not have access, contact your system administrator). See "Set Up Users" on page 15 for more information.

2) If you lack an option for business rules on the Application Security tab, you have not run the script to enable the DynaChange Rules on the current database, or you might have the system setting disabled. You must run the licensing script to enable the DynaChange Rules, and then enable the Use DynaChange Rules system setting. See "Set Up DynaChange Rules" on page 12 for more information.

3) You may be missing files in the folder where Prophet 21 is installed (e.g., C:\Program Files\Activant\Prophet 21 12.5). You must have **Activant.P21.Extensions.dll** and the **pxxi.exe.manifest** files in the same folder as the pxxi.exe file, or DynaChange Rules will not work.

4) Your client may not be updated to .NET Framework version 3.5 or better. To find out what .NET Framework version you have installed, go to the following path in Windows Explorer: *C:\WINDOWS\Microsoft.NET\Framework*; in this path you will find a list of folders with version names on them. Make sure the v3.5 (or higher) folder is present.

I cannot find the DataWindow for my tab listed in the Field Selector tab.

The Field Selector tab can only load DataWindows for tabs that are instantiated in Prophet 21. To save on memory and startup time, some tabs are not instantiated when the window opens, and are instead only instantiated when you visit them. If you cannot find your tab listed in the Field Selector tab, make sure you open that tab before you create or edit the business rule.

My business rule does not work. I see it in the Business Rule Organizer, but nothing happens when I use the field.

There are several possible reasons for this.

1) The business rule applies to the field you right-click on when you create the business rule, and not the field that currently has focus (the field with the blinking cursor). Make sure you apply your business rule to the correct field.

2) If your business rule was one you applied to all instances of a field, there are some cases where field names are not consistent across the system. For example, there are mail_city and phys_city, which are both cities, but in two different types of address. You want your business rule to work for all addresses. In this case, you must use an alias in your business rule, and ensure you use the Pass Field to Rule As field to specify the alias when you link the rule to Prophet 21. You must also find instances of both mail_city and phys_city and link the business rule and apply to all instances for each. See "Link Business Rules to Prophet 21" on page 50 for more information on linking rules to Prophet 21.

3) The logic of the business rule may result in no changes to the data. This may be due to an unhandled exception, or it may be due to the rule running properly, but the current state of the application did not result in any changes in Prophet 21. Talk to the person who developed your rule (or review the code yourself) to ensure that there are no unhandled exceptions, and to ensure that the current situation should result in some change.

I do not see my business rule in the New Business Rule dialog.

This can occur if the business rule class created by the developer is not declared as a public class. When you create the business rule class for your business rule, it must be public, or Prophet 21 cannot "see"

the rule. The rule developer must make the rule's class public.

I get an error something like "Alias X not found" or "Field Name X not found" (where X is some field name or alias) when I use a field that has a business rule attached.

This means that the business rule is trying to access or set data with the alias or field name specified in the error message, but that the alias or field name does not exist in the data you are sending. To fix this, you must ensure you send the correct data field, or that you set the correct alias for your field. If you are not sure what that is, consult the business rule's source code, or talk to the developer of the business rule.

I get an error "The Custom DLL Folder for the DynaChange Rules needs to be entered in the System Settings window."

The DLL Folder field in the Files and Folders node of the System Setup system settings is blank. Enter the location of the DLLs that contain your business rules. This location must be accessible to all Prophet 21 users. This message appears for any user who does not have access to the folder.

I get an error "Unable to load the Business Rule Manager DLL."

This error is caused in one of the following cases:

1) You may be missing files in the folder where Prophet 21 is installed (e.g., C:\Program Files\Activant\Prophet 21 12.5). You must have **Activant.P21.Extensions.dll** and the **pxxi.exe.manifest** files in the same folder as the pxxi.exe file, or DynaChange Rules will not work.

2) Your client may not be updated to .NET Framework version 3.5 or better. To find out what .NET Framework version you have installed, go to the following path in Windows Explorer: *C:\WINDOWS\Microsoft.NET\Framework*; in this path you will find a list of folders with version names on them. Make sure the v3.5 (or higher) folder is present.

I get an error "Error: That assembly does not allow partially trusted callers."

You are using a mapped network drive as your DLL folder, and you did not add the AllowPartiallyTrustedCallers property to your rule project. See "Create a Visual Studio Project" on page 22 for more information.

I get an error "Data.Fields cannot be accessed in a Multi-Row rule. Please use Data.Set or Session."

Your rule is setup to pass multiple rows into the rule (the Multi-Row check box is enabled). Rules setup in this manner require the developer to use the .NET DataSet object available on the rule. This is accessed by using the "Data.Set" in the rule code in order to access the data that you are passing (see "DataCollection Class" on page 104 for more information). If you are trying to access global data (such

as user id, server name, etc.) you should use the "Session" object available on the rule (see "Session Class" on page 110 for more information).

I get an error "Data.Set cannot be accessed in a non-multi-row rule. Please use Data.Fields."

Your rule is setup to pass only a single row into a rule (the Multi-Row check box is not enabled). Rules setup in this manner require the developer to use the .NET List object available on the rule. This is accessed by using "Data.Fields" in the rule code in order to access the data that you are passing.

I get an error "Cannot use SetFieldUpdateOrder with MultiRow rule. Set Data.UpdateByOrderCoded instead."

You have attempted to access the method "SetFieldUpdateOrder" when your rule is setup to pass multiple rows of data. This method can only be used for single row rules. In order to set the order in which fields update in the Prophet 21 system after the rule executes, you must set the Boolean "Data.UpdateByOrderCoded" to TRUE in your rule code (see "DataCollection Class" on page 104 for more information).

I get an error "SetFocus(String field) cannot be used with a multi-row rule. Use SetFocus(int RowID, String field)."

You have attempted to access the method "SetFocus(String field)" when your rule is setup to pass multiple rows of data. This method can only be used for single row rules. In order to set focus to a specific field after your rule is executed, you must pass the row number on which you want the field to have focus. This can be accomplished by using the overloaded method "SetFocus(int RowID, String field)".

I get an error "SetFocus(int RowID, String field) cannot be used with a single row rule. Use SetFocus (String field)."

You have attempted to access the method "SetFocus(int RowID, String field)" when your rule is setup to pass only a single row of data. This method can only be used for multi-row rules. In this case, focus can be set to a specific field, but not a specific row. The row on which the field will have focus will be the row on which the rule is operating. To set focus in this manner you can use the "SetFocus(String field)".

Developer Troubleshooting

When I build my rule, I get the error "Unable to copy file..." and then the object it cannot copy.

This error appears when the DLL file the build is trying to copy is locked by another process. Typically you see this error while you are testing your application and you build rules directly to the folder Prophet 21 uses to find your business rules. The error appears because Prophet 21 has a lock on the DLL file. Close Prophet 21 and rebuild your project, and the error should not occur.

When my business rule runs, it does not change the drop-down field or radio button to the option I specified, but the value appears when I test the rule.

In Prophet 21, the database stores most drop-down field and radio button options as a numeric value (which Prophet 21 resolves to the word you see at runtime). However, the DynaChange Rules translates those numbers to the correct text for you, so you do not have to write any logic to find the values, or figure out what the database values for each option is before you can use them in your rule. If you need to access or set the value of a drop-down field or radio button, make sure you use for the value the name of the option as you see it in Prophet 21.

When I try to test my rule in Prophet 21, it crashes with an error like "Error calling external object function initialize."

If you are using Visual Studio 2010 or higher, your project targets .NET Framework 4.0 by default. If you do not change the setting to .NET Framework 3.5, Prophet 21 crashes because it cannot communicate with version 4.0. To set your project to .NET Framework 3.5, right-click your project in Visual Studio and select Properties. On the Application tab, change the Target Framework field to **.NET Framework 3.5**. You may have to also remove a reference to csharp in the assembly folder (if so, you will see an error to that effect when you try to build).

When I build my rule, I get the error "Non-invocable member 'Activant.P21.Extensions.BusinessRule.DataCollection.Fields' cannot be used like a method."

This most commonly appears when you use parentheses with the Data.Fields collection instead of brackets. The code must look like this...

```
Data.Fields["ship_to_id"].FieldValue
```

...and not like this...

```
Data.Fields("ship_to_id").FieldValue
```

When I build my rule, I get the error "Cannot apply indexing with [] to an expression of type 'method group'."

This most commonly appears when you use brackets with GetFieldByAlias() instead of parentheses. The code must look like this...

```
Data.Fields.GetFieldByAlias("ship_to_id").FieldValue
```

...and not like this...

```
Data.Fields.GetFieldByAlias["ship_to_id"].FieldValue
```

My system administrator or users tell me my rule doesn't work, but nothing happens, and there is no error message. The administrator has confirmed that the rule is linked correctly in Prophet 21.

There are several possible problems.

1) The business rule algorithm does not account for the situation or application state the user faces, and you must modify it to account for that, or there is a bug of some kind within the rule's logic. To test this, you can log into Prophet 21 as a programmer (alt - t at the login screen, and on the Testing Options tab, select **Run as Programmer**). Run the rule, which will create entries in the business_rule_log table. You can use that information to ensure that the data traveling from Prophet 21 to the rule is correct, and then that the data traveling from the rule back into Prophet 21 is also correct. See "General Troubleshooting" on page 80 for more information on using the business_rule_log table.

2) The business rule algorithm encountered an unhandled exception, so the business rule reported the RuleResult's Success property back to Prophet 21 as true and the Message property as blank, so Prophet 21 did not know the rule failed, and did not report a message. This most commonly occurs if you do not place your code in try / catch error handling. One good way to troubleshoot this to ensure all of your code is tested in try / catch error handling, and set the Message and Success properties of the RuleResult to the exception thrown. The code looks something like this for the catch block (the "rr" below is the instantiated RuleResult object):

```
try
{
    if (Data.Fields["ship_to_id"].FieldValue != null)
    {
        Data.Fields["payment_type_id"].FieldValue = "Visa";
    }
}
catch (Exception ex)
{

    rr.Message = ex.Message;
    rr.Success = false;
}
```

This causes Prophet 21 to report the error message provided by the .NET Framework.

If that does not do the trick, or if, for whatever reason, you cannot use try / catch logic, use the log recording features of your business rule to record what happens as it happens so you can see where the rule is failing (see "LogString Class" on page 102 for more information on creating and using log files).

I'm referencing a separate assembly (DLL file) from my business rule, but I'm getting an error that it cannot be found.

You must ensure that the assembly DLL referenced by your business rule is in the same folder as your rule (i.e., in the DLL folder you specified in System Settings). See "Set Up System Settings" on page 13 for more information on the DLL folder.

Web Visual Rule Troubleshooting

Most web visual rule troubleshooting is similar to the general troubleshooting above. However, there are some situations that are specific to web visual rules and their setup.

I am seeing a “not secure” message in the test harness or when I go to my web visual rules site.

Web visual rules require valid SSL certificates just like the Prophet 21. If they do not have a valid cert, then you will see the “not secure” message when testing, and likely the web visual rules will not work at all.

I get a 413 error when trying to run my rule.

This is most frequently caused by in incorrectly formatted URL – e.g., one that is missing a / in the wrong place, for example. This can also be caused by bad settings in IIS; if you search the web for this error, you can find information about the problem and how to fix it.

I get an access denied error.

You may see this if you cannot ping your Middleware server from your web visual rules site, or if there are SSL problems with either site.

Appendix A: Rule Class API

This chapter describes the API for the Rule class. All of your business rule classes are derived from the Rule class, and gain access to several methods, classes, and properties that you can use to interact with the data passed to you from Prophet 21.

The end of this section also contains some tips to remember.

Rule Class **91**

Properties 91

Methods 93

WebBusinessRule Class **93**

RuleResult Class **94**

Properties 94

ReposeAttributes 95

ResponseButton 95

ResponseField 96

RuleState Class **97**

Properties 97

RulePopupService Class **98**

Methods 99

DataField Class **99**

Properties 99

LogString Class **102**

Properties 103

Methods 103

DataCollection Class **104**

Properties 105

Methods 106

DataFieldKey Class 108

Properties 108

DataFieldAttributes Class 109

Properties 109

Session Class 110

Properties 110

XML Datastream Class 111

Properties 111

Methods 111

Tips 113

Rule Class

Rule is the class from which you derive all of your business rules. It contains all of the other methods and properties you need to create a business rule. It also enables you to interact with the other classes in the API, and instantiates an instance of the LogString class called Log and a DataCollection class called Data.

Properties

Property:	Description:
LogString Log	The LogString object instantiated with your rule. Used to log progress in your business rule so you can debug errors and check on the progress of your code. You can use the methods and properties of the LogString class to add to, persist, and clear the log. Use this property to handle your logging, rather than instantiating a separate LogString class.

Property:	Description:
	See "LogString Class" on page 102 for more information on the methods and properties available.
DataCollection Data	The DataCollection object instantiated with your rule, consisting of a collection of DataField objects which were passed from Prophet 21. You must use this property to handle the Prophet 21 data, rather than instantiating a separate DataCollection class. If you do not use the Data instance of this class, it will not contain the data sent by Prophet 21. See "DataCollection Class" on page 104 for more information on the methods and properties available.
Session	The Session object instantiated with your rule, consisting of a collection of properties which were passed from Prophet 21. See "Session Class" on page 110 for more information on the methods and properties available.
RuleState	The RuleState object instantiated with your rule, consisting of properties describing the rule and its current state. See "RuleState Class" on page 97 for more information on the methods and properties available.
SqlConnection P21 SqlConnection	The P21SqlConnection object is instantiated with your rule. The connection is opened by the RuleManager and is ready to be used immediately during execution of the rule. Since the RuleManager is handling opening and closing this connection, it does not need to be (and should not be) closed or disposed of within the execution of a business rule.

Methods

Method Signature:	Description:
public string GetName()	Provides the name of your rule to Prophet 21. This method is required for the class you derive from Rule.
public string GetDescription()	Provides the description of your rule to Prophet 21. This method is required for the class you derive from Rule.
public RuleResult Execute()	Code inside this method gets executed when you run your business rule. Prophet 21 waits for the RuleResult to return before continuing. This method cannot be used with a workflow link; workflows use asynchronous execution (see "Types of Business Rule Links" on page 48 for more information).
public void ExecuteAsync()	Code inside this method gets executed when you run your business rule. Prophet 21 does *not* wait for code in this method to execute before continuing. Use this method when you want to do something, but Prophet 21 does not need to know the results. Workflows use this method. You cannot use this method if you want to return values to Prophet 21. The Execute() method must still be defined in your rule class, even if you are using the ExecuteAsync() method for your code.
public void Initialize(string)	The constructor for the rule class; it is called only by Prophet 21. You should not invoke this method.

WebBusinessRule Class

WebBusinessRule inherits from the Rule class (and thus contains all of its features), as well as a method of its own. This class is used to instantiate a web visual rule.

Method Signature:	Description:
public List<dynamic> GetDataTableAsList (DataTable)	Creates a List<dynamic> collection when you pass in a DataTable. This can simplify passing values from a DataTable to something like a WebGrid.

Method Signature:	Description:
public List<dynamic> GetDataTableAsList (int) public List<dynamic> GetDataTableAsList (string)	Creates a List<dynamic> collection when you pass in a rule table's table index. This can simplify passing values from a DataTable to something like a WebGrid. Creates a List<dynamic> collection when you pass in a name of a DataTable in string format. This can simplify passing values from a DataTable to something like a WebGrid.

RuleResult Class

RuleResult contains information on the success or failure of your business rule. RuleResult is the return type of the Execute() method, and must be instantiated for any rules that use the Execute() method. Prophet 21 uses its properties to determine if the rule ran successfully, and if it did not run successfully, what went wrong.

Properties

Property:	Description:
bool Success	Boolean property that Prophet 21 checks to see if the rule ran successfully. You should set this property in such a way that it is only true if your code ran successfully.
string Message	Contains the message Prophet 21 shows when the Execute() method returns. You can manually set the message for this, or you can pass a try / catch message to it, which will then display in Prophet 21. Typically, you will use this to pass error messages back to Prophet 21.
string Keystroke	Contains the string value of the keystroke that should be simulated upon completion of the rule. This is currently only implemented for OnDemand type rules. When the rule returns to Prophet 21, the system will read this value and simulate the user pressing the given key. For more information see "Simulating Keystrokes" on page 134.
ShowResponse	Indicates if the rule result includes response attributes. While this defaults to false, you should set this to true if coding a rule that uses response attributes.
ResponseAttributes	These define the properties of a multi-option message or response window generated by the rule. These are detailed in a

Property:	Description:
	separate section below.

ReposeAttributes

ResponseAttributes are a special kind of rule result that define and configure popups with buttons or fields.

Property:	Description:
ResponseTitle	The title displayed at the top of the response window.
ResponseText	The text displayed in the response window. For multi-option popups, this is the message text. For popups with data entry fields, this text appears above the fields.
CallbackRule	The rule class that processes the selection or entered values from the response window. It triggers when the popup closes. Use a name of a traditional rule class descending from P21.Extensions.BusinessRule.Rule.
CallbackDataTableName	The data table that records user feedback from the popup, meaning it stores what selections the user made or data they entered.
Buttons[]	If your response window contains buttons for the user to select an option, this is the property that defines them. See the ResponseButton class section below.
Fields[]	If your response window contains buttons for the user to enter values in fields, this is the property that defines them. See the ResponseField class section below.

ResponseButton

The ResponseButton class defines the buttons that appear on a response window.

Property:	Description:
ButtonName	How the button is named in the attached data table.

Property:	Description:
ButtonText	How the button is named in the response window.
ButtonValue	Additional value to identify or associate with the button. The value of the button the user clicks gets returned in the “selected_button_value” column of the callback DataTable. Usage is often redundant or a duplicate of ButtonName, but can be set as any desired string.

ResponseField

The ResponseField class defines the fields that appear on a response window.

Property:	Description:
Name	How the field is named in the attached data table.
Label	How the field is named in the response window.
DataType	The data format the field accepts. Your options are: • ResponseFieldType.Alphanumeric • ResponseFieldType.Numeric • ResponseFieldType.Decimal
DataTypeLength	The character limit for data entered in the field. For decimal fields, this is the number of digits allowed after the decimal point.
DropDownListDisplayValues[]	Entering values in this string turns the field into a dropdown menu. What you enter here displays as the menu options.
DropDownListDataValues[]	These are the dropdown menu values that get recorded to the attached data table. If you use the DropDownListDisplayValues property, use this as well and make them match.

Method:	Description:
SetFieldValue<T>(T initialDataValue)	If you want the field to autopopulate with a default value, enter it here.

RuleState Class

The RuleState class contains all of the information about the state of the rule that is currently executing. All properties are read only.

Properties

Property:	Description:
int UID	Contains a unique identifier for the rule in the business_rule table in the Prophet 21 database.
string Name	Contains the name of the rule.
string Type	Contains the type of the rule. Valid values include Converter, Validator, On Demand, On Event, and Workflow.
string ApplyOn	Contains information on when the rule is applied, including Save, Field Edit, or Window Opening.
bool MultiRow	Boolean property that identifies whether or notProphet 21 is passing data to the rule in a multi-row format.
string RunType	Contains information on who the rule is executing, either Synchronous or Asynchronous.
string EventName	For On Event rules, this contains the name of the event.
string EventType	For On Event rules, this contains the type of event. For Form Datastream Created Events, this contains the type of form, such as invoice or packing list. For Import Preprocessing events, this contains the type of import.
bool CascadeInProgress	Boolean property that identifies whether or not the rule is executing as a result of the execution of another rule.
string theme_name	Contains the name of the current theme.
string	The following value indicates the context in which the Prophet 21

Property:	Description:
ApplicationDisplayMode	application is running at the time the rule is executed.: **uiserver** - Prophet 21 web UI or hybrid client application. **mdi** - classic desktop user interface **sdi** - new desktop user interface **wwms** - wireless warehouse **nv** - API
string ClientPlatform	Value that indicates the client is being used for Prophet 21 when run on the web. The possible values are as follows: **web** – This value is returned when running Prophet 21 via a web browser (HTTPS) **hybrid** – Value returned when running the compiled hybrid client application **blank/empty** – Value returned when running Prophet 21 in any non-web context such as desktop and wwms
IsCallbackRule	Boolean value indicating whether the rule being executed is a callback rule triggered after a message or simple response window generated via ResponseAttributes has been displayed and interacted with by the user.
CallbackParentRule	The name of the triggering rule. This is the name of the rule class that intitially returned ResponseAttributes causing a message or simple response window to be displayed.

RulePopupService Class

The RulePopupService class contains information to invoke popups from business rules, enabling you to show Prophet 21 popups from your own .NET windows. For example, you could write your own window to store customer rebate records, and in that window, you could use the Prophet 21 "Find Customer" popup to find and retrieve customer IDs.

Methods

Method Signature:	Description:
ShowPopup	Invokes the popups for a field. This method receives only one parameter – a string with the name of the Prophet 21 field to pop up is for – and returns a string as the value based on the field selected in the popup. The return value is typically the ID for the field.

DataField Class

The DataField Class contains all of the information that identifies a single data field passed from Prophet 21, as well as the value of the field itself.

Properties

Property:	Description:
string ClassName	Contains the name of the DataWindow class or customized class from Prophet 21 from which the data field originated. This property is read-only.
string FieldName	Contains the name of the field in the Prophet 21 DataWindow. This property is read-only.
string FieldTitle	Contains the label of the field in Prophet 21, if the field has a label. The label is the text that displays on screen for the field. This property is read-only.
string FieldAlias	Contains the name of the field as stated in the Pass to Rule As field in Prophet 21. This property is typically used when you want a business rule to work generically for groups of fields that serve a similar purpose, but do not have the same name (for example, if you want the rule to act on all ZIP code fields, some are mail_postal_code, and others may be phys_postal_code, but if you pass them all to the "postal_code" alias, you can use the alias to refer to them, instead of writing extra business logic in your rule to handle differences). This property is read-only.
string FieldValue	Contains the value of the field from Prophet 21. You can read and

Property:	Description:
	write to this property. Writing a new value to this property enables you pass data back to Prophet 21. Check box values in Prophet 21 are *Y* for checked and *N* for unchecked. If you are either reading or setting values for check boxes, make sure you are expecting or setting a Y or N. The value of radio buttons and drop-down fields uses the name of the value in the radio button option or drop-down, not the database value (which is numeric). In your business rules, make sure you use the names of the options and not the numeric, database representation.
string ReadOnly	Contains Y or N and represents if the field is a read only field in Prophet 21. This property is considered informational only and does not prevent the updating of FieldValue. This property is read-only.
string DataType	Contains the PowerBuilder data type in Prophet 21. This property is read-only.
string FieldOriginalValue	Contains the original value of the trigger field prior to the user changing the value in Prophet 21. For example, the user changes order quantity from 10 to 20 which triggers a rule to execute. This field will contain 10 in this example and the property FieldValue will contain 20. This property is read-only. This field is only set for the field that triggered the rule to execute, for all other fields it will be blank. Certain rules will not have a field that triggered the rule, such as on window save. This string only works for converter rules. The original trigger value can be more easily accessed via the Session Class (and must be for multi-row rules).

Property:	Description:
string TriggerColumn	Contains Y or N and represents if the column associated with the field is the same column associated with the field that triggered the rule. This property is read-only. The trigger column name can be more easily accessed via the DataCollection class (and must be for multi-row rules).
string TriggerRow	Contains Y or N and represents if the row associated with the field is the same row associated with the field that triggered the rule. This property is read-only. The trigger row number can be more easily accessed via the DataCollection class (and must be for multi-row rules).
string UpdateSequence	Contains a sequence number representing the order in which the field will be updated in Prophet 21. For multi-row rules, this field will always represent the default order in which the fields were passed into the rule and is read only. To re-sequence multi-row rules, see the property UpdateByOrderCoded in the DataCollection class. For single row rules, this property will default to the order in which the fields were passed into the rule. It can be updated using a call to the DataCollection class method SetFieldUpdateOrder.
string SetFocus	Contains Y or N and represents if focus should be set to this field after the rules has fired, see the SetFocus methods in the DataCollection class for more information. This property is read-only.
string BaseClassName	Contains the name of the DataWindow base class from Prophet 21 from which the data field originated. This is the class from which the class identified by ClassName is derived, if the class had been customized for the user. This property is read-only.
string TableName	Contains either the ClassName or the BaseClassName in this class.

Property:	Description:
	BaseClassName will be used if it is set, otherwise ClassName will be used. This property will represent the name of the table in which the field resides in the Data. Set property in the DataCollection class. This property is read-only.
string ColumnName	Contains either the FieldName property or the FieldAlias property in this class. FieldAlias will be used if it is set, otherwise FieldName will be used. This property will represent the name of the column in which the field resides in the corresponding table in the Data. Set property in the DataCollection class. This property is read-only.
string RowID	Contains the row number of the field in Prophet 21. Most of the Rule methods that return an index of a row return this RowID and not the table row's index in the DataTable. This property is read-only.
bool Modified	Determines if the data field has been modified by your rule. If you change the FieldValue property, this flag is automatically set to true. This tells Prophet 21 when a field has been modified, so Prophet 21 can populate the modified value. This property is read-only.
bool NewRow	Contains Y or N. It will be set to Y if this row was added using the AddNewRow method on the Data object on the Rule.
bool AllowCascade	Contains Y if this rule is executing as a result of the execution of another rule, otherwise contains N. This is a read only property.

LogString Class

The LogString class handles all the logging for business rules. The class enables you to create a log file that records actions and data you specify so that you can effectively debug your business rules, or track progress if the rule engages in some prolonged activity.

> The class is instantiated as the Log property of your rule class when your rule class is instantiated. Use that instantiation to perform all of your logging. Generally, you should not need to instantiate this class yourself.

Properties

Property:	Description:
bool LineTerminate	Determines if the Add() method adds a carriage return to each log string. • **true** - the Add() method adds a carriage return to the end of the log string. • **false** - the Add() method does not add a carriage return.
int MaxFileMb	Determines the maximum size of the log file (in megabytes). When the log file reaches this size, it will delete the oldest log strings to make room for the newest. By default, this value is not set, so there is no limit.
string Name	The name of the class used to create the log file name. This property is read-only.
bool ReverseOrder	Determines if the log file is written with new log entries appearing at the top or the bottom of the log file. • **true** - the log files are written with the newest entries appearing at the top of the file. • **false** - the log files are written with the newest entries appearing at the bottom of the file.

Methods

Method Signature:	Description:
public void Add(string)	Adds the specified string to the log. Using this method, each log string is added to a list, which is then written to file when you call the Persist () method. Log strings added using this method do *not* get written when you call AddAndPersist(). This method is especially useful if you want to create a log of something that happens in a loop, but do not want to incur the disk input / output overhead for each loop. If the LineTerminate property is true, this method adds a carriage return to the end of the string.

Method Signature:	Description:
public void AddAndPersist (string)	Immediately writes the log string to the log file. Use this method when you want to write a log file entry immediately without waiting for the Persist() method to be called. This method does not add strings to the list created by the Add() method, so when you call Persist(), these strings are not added to the file a second time. The file takes the name of the project and the name of your rule as the name of the log file (e.g., AutoZipCodeProject.AutoZipCodeClass.log). The log file is created in a "logs" folder, which is located in the same folder as your Prophet 21 application (where you find pxxi.exe). If the LineTerminate property is true, this method adds a carriage return to the end of the string.
public void Clear()	Deletes the log file for this business rule.
public void Persist()	Persists the collection of log strings to a file. The file takes the name of the project and the name of your rule as the name of the log file (e.g., AutoZipCodeProject.AutoZipCodeClass.log). The log file is created in a "logs" folder, which is located in the same folder as your Prophet 21 application (where you find pxxi.exe). Once you have called Persist(), the list of strings is purged from the object. That way, you do not write again any strings that you have already written to the log file.

DataCollection Class

The DataCollection class handles all the data passed from Prophet 21 to your business rule. It contains a list of all DataField objects passed by Prophet 21.

The class is instantiated as the Data property of your rule class when your rule class is instantiated. Use that instantiation to act on any data passed from Prophet 21. Do not try to instantiate this class manually, as it will not contain any data.

Properties

Property:	Description:
DataFields Fields	The Fields property enables you to access a data field sent by Prophet 21. Use this property to access the data passed from Prophet 21. You can access a DataField using either the field name (e.g. Data.Fields ["mail_city"]) or its position in the collection (e.g., Data.Fields[0]). When using the field name to access a data field, you must use the field name used by Prophet 21, and not the alias or title. If you need to find the field name, you can look in the Field Selector in Prophet 21. See "Link Business Rules to Prophet 21" on page 50 for more information. You can also access fields using the GetFieldByAlias() method. See *DataField GetFieldByAlias()* above for more information. This property cannot be accessed within multi-row rules, you must use Data.Set as described below.
DataSet Set	The Set property enables you to access and update the data tables sent by Prophet 21 in multi-row rules. This property is a standard .NET DataSet, and you can use it as you would any DataSet. Data for the multi-row rules resides in DataTable objects within the DataSet. You can access the data tables using the table name, as per standard .NET conventions (Data.Set.Tables["oe_line"]). The columns in the DataTable objects are typed, so you must use proper data types when getting and setting values in the DataSet. When updating tables in Data.Set, you should not update data in any column other than the values you passed. You may create new rows and columns, but these are ignored by Prophet 21.
string TriggerTable	Contains the name of the table (class) with which the trigger field was associated. This property is read-only.
string TriggerColumn	Contains the name of the column (field name or alias) with which the

Property:	Description:
	trigger field was associated. This property is read-only.
string TriggerOriginalValue	Contains the original value of the trigger field prior to the user changing the value in Prophet 21. For example, the user changes order quantity from 10 to 20 which triggers a rule to execute. This field will contain 10 in this example and the property FieldValue will contain 20. This property is read-only.
int TriggerRow	Contains the row number with which the trigger field was associated. This row number is the RowID, and not the DataTables's row index. This property is read-only.
bool UpdateByOrderCoded	Used for multi-row rules only. You can read and write to this property. If set to TRUE, the fields from the rule will be updated in the order in which they were updated in the rules code. Default is FALSE.

Methods

Method Signature:	Description:
public string ToXML()	Translates the DataCollection class into XML. You may use this method for debugging, so you can see all the data sent to your business rule.
DataField GetFieldByAlias()	This method is available only through the Fields property. The method is associated with a class not described here (DataFields), but the method is accessible to the Fields property by dot notation (e.g., Data.Fields.GetFieldByAlias). This method also has access to all of the properties available to the DataField class (see "Properties" on page 97 for more information on the DataField properties). This method is very powerful because it enables you to write one rule to handle groups of fields that serve a similar purpose. For example, in Prophet 21, address fields (such as City) have different field names, such as mail_city or phys_city, depending on what kind of address it is. However, in some cases, you want the code you write to work for **all** addresses. Normally, you would have to write code to handle each of the different address types, but using the GetFieldByAlias method, you can identify the data you want using an alias you provide in Prophet 21 (you can specify this in the Pass to Rule As field in Prophet

Method Signature:	Description:
	21). So, if you want to access "city" generically, you would select the mail_city or phys_city field in Prophet 21, and then specify *city* in the Pass to Rule As field. Then, in your code, use the GetFieldByAlias() method to access fields using the alias. The syntax is as follows: `string myCity = Data.Fields.GetFieldByAlias ("city").FieldValue;`
public void SetFieldUpdateOrder (List<string> columnNames)	This method is only available for single row rules. Pass this method a list of column names (see property "string ColumnName" on page 102) in the order in which you would like the columns updated.
public void SetFocus(string columnName)	This method is only available for single row rules. Pass this method the column name of the field on which you want focus transferred to after the rule has executed. The column must be associated with the table that triggered the rule.
public void SetFocus(string columnName, int RowID)	This method is only available for multi-row rules. Pass this method the column name and the row number of the field on which you want focus transferred to after the rule has executed. The column must be associated with the table that triggered the rule. You cannot use this property to change tabs or data windows in Prophet 21.
public DataFieldAttributes GetFieldAttributes(string tableName, string columnName, int RowID)	Returns a DataFieldAttributes object for the field specified by the table name, column name and row number passed as parameters.
public bool IsTriggerTable (string tableName)	Returns a boolean indicating if the specified table was the table associated with the field that triggered the rule.
public bool IsTriggerRow (string tableName, int RowID)	Returns a boolean indicating if the specified row in the specified table was the row associated with the field that triggered the rule. The row you pass in must be the RowID and not the DataTables's row index.
public bool IsTriggerField (string tableName, int row, string columnName)	Returns a boolean indicating if the field associated with the specified table, column and row is the field that triggered the rule.

Method Signature:	Description:
public void SetCascade(bool allow)	Sets or clears the attribute that allows cascading of further rules on all fields.
public void SetCascade (String tableName, bool allow)	Sets or clears the attribute that allows cascading of further rules on an entire table in a multi-row rule.
public void SetCascade (String tableName, String columnName, bool allow)	Sets or clears the attribute that allows cascading of further rules on an entire column in a multi-row rule.
public void SetCascade (String tableName, String columnName, int rowID, bool allow)	Sets or clears the attribute that allows cascading of further rules on a single field in a multi-row rule.
public void SetFieldCascade (String columnName, bool allow)	Sets or clears the attribute that allows cascading of further rules on a single field in a single row rule.
public DataRow AddNewRow (string tableName)	Adds a new row to a table in the data collection in a multi-row rule and returns it as a System.Data.DataRow.
public DataRow AddNewRow (DataTable table)	Adds a new row to a table in the data collection in a multi-row rule and returns it as a System.Data.DataRow.

DataFieldKey Class

The DataFieldKey class defines the key fields needed to access a data field.

> The class is currently found on the DataFieldAttributes class and is instantiated as the DataFieldAttributes class is instantiated.

Properties

Property:	Description:
string TableName	Name of the table associated with the field. See the "string

Property:	Description:
	TableName" on page 101 to understand how it is populated. This property is read-only.
string ColumnName	Name of the column associated with the field. See the "string ColumnName" on page 102 to understand how it is populated. This property is read-only.
string RowID	Row number associated with the field. This property is read-only.

DataFieldAttributes Class

The DataFieldAttributes class contains various attributes associated with a field. This class can be used with both single and multi-row rules, and must be in the case of the latter in order to access the attributes of a field. Single row rules can still access the DataField class directly as well.

To instantiate, call the method GetFieldAttributes in the DataCollection class. Do not try to instantiate yourself as it will not contain any data.

Properties

Property:	Description:
DataFieldKey Key	Contains the key fields that describe the associated field. This property is read-only.
string Title	Contains the field title of the field. See the "string FieldTitle" on page 99 to understand how this is populated. This property is read-only.
string Name	Contains the field name of the field. See the "string FieldName" on page 99 to understand how this is populated. This property is read-only.
string Alias	Contains the field alias of the field. See the "string FieldAlias" on page 99 to understand how this is populated. This property is read-only.
string DataType	Contains the PowerBuilder data type of the field. This property is read-

Property:	Description:
	only.
bool ReadOnly	Contains information of the field. See the "string ReadOnly" on page 100 class to understand how this is populated. This property is read-only.

Session Class

The Session class contains various properties that are always passed to a rule. Use these properties to build a connection string for the database, to control rules on a user-by-user basis, or control rules based on Prophet 21 version.

> The class is instantiated as the Session property of the Rule class when your Rule class is instantiated. Use that instantiation to get various properties.

Properties

Property:	Description:
string UserID	Contains the user ID field for the currently-logged-in user. This property is read-only.
string Version	Contains the current Prophet 21 version of the client. This property is read-only.
string Server	Contains The database server name that hosts the database the user is logged into. This property is read-only.
string Database	Contains the name of the database the user is currently logged into. This property is read-only.
string Language	Contains the language description for the currently-logged in user. This property is read-only.
string ID	Contains a unique ID representing the current Prophet 21 user session. This property is read-only.
string rf_location_id	Contains the Location ID chosen at the welcome screen when running

Property:	Description:
	the RF mode. If you are not running in RF mode, the Location ID is empty. This property is available for both event type and rules setup on a field or save of a window.
string rf_bin_id	Contains the Bin ID chosen at the welcome screen when running the RF mode. If you are not running in RF mode, the Bin ID is 0. This property is available for both event type and rules setup on a field or save of a window.
string Configuration ID	Contains the Configuration ID. This property is available for both event type and rules setup on a field or save of a window.

XML Datastream Class

The XML Datastream class will be instantiated only when using the rule event Form Datastream Created. It contains the file path and an XDocument representation of the XML representing the form, including helper methods to access the form information.

Properties

Property:	Description:
string FilePath	Represents the file path where the XML version of the Prophet 21 Datastream resides.
XDocument Document	Represents a .NET Linq to XML XDocument object type. It will be populated with the contents of the Datastream.

Methods

Method Signature:	Description:
public IEnumerable<XElement> GetForms()	Returns all forms in the datastream.
public IEnumerable<XElement>	Returns the headers for all forms in the data stream.

Method Signature:	Description:
GetHeaders()	
public XElement GetHeader (XElement form)	Returns the header for a given form.
public IEnumerable<XElement> GetLines()	Returns all lines for all forms in the data stream.
public IEnumerable<XElement> GetLines(XElement form)	Returns all lines for a given form.
public IEnumerable<XElement> GetLinesBy(string elementName, string elementValue)	Returns all lines based on the existence of an element with a certain value.
public IEnumerable<XElement> GetLinesBy(XElement form, string elementName, string elementValue)	Returns all lines for a given form based on the existence of an element with a certain value.
public IEnumerable<XElement> GetTotals()	Returns all totals for all forms in the data stream.
public XElement GetTotal (XElement form)	Returns the totals for a given form.
public IEnumerable<XElement> GetGroup(string groupName, XElement fromElement)	Returns a group of elements associated with a given line.
public void AddGroup (XElement groupToAdd, XElement addToElement)	Adds a group element to another group element.

Method Signature:	Description:
public void SortLines(string sortElement, bool numeric=false, bool descending = false)	Sort lines for all forms in sortElement sequence.
public void SortLines (XElement form, string sortElement, bool numeric=false, bool descending = false)	Sort lines for the given form in sortElement sequence.
public void SortLineGroup (string groupName, string sortElement, bool descending = false)	Sort a group for all lines on all forms in sortElement sequence.
public void SortLineGroup (XElement form, string groupName, string sortElement, bool numeric=false, bool descending = false)	Sort a group for all lines on the given form in sortElement sequence.
public void SortLineGroup (XElement form, XElement line, string groupName, string sortElement, bool numeric=false, bool descending = false)	Sort a group for the given line on the given form in sortElement sequence.

Tips

The following list contains some useful tips and things to remember when you are creating your business rules.

- Business rule classes must be declared as public classes, and the required methods must also be public.

- The GetName() and GetDescription() methods are required, and then you must have either Execute(), or both Execute() and ExecuteAsync() if you want the business rule to work with workflows as well as other rule types.
- If you are dealing with a check box in Prophet 21, the values to use are **Y** for checked and **N** for unchecked.
- The value of radio buttons and drop-down fields uses the name of the value in the radio button option or drop-down rather than the database value (which is numeric). In your business rules, make sure you use the names of the options and not the numeric, database representation. For example, "Active" might be a value of 704 in the database. You should access and set the value in your business rule as Active.
- If you need to find the name of a field in Prophet 21, right-click the field and select Help > SQL Information. The SQL Information dialog displays the SQL database column, as well as the field's name. Do not use the SQL database column name; use the field name.
- In many cases, methods require a row index, or return a row index. In most cases, this is not the DataRow's row index. It is instead the RowID of the row, which is a column in the DataRow.

Appendix B: Sample Rules

Epicor provides a few sample business rules to demonstrate various types of rules. The rules are provided both as a DLL file you can hook directly to Prophet 21 and as a Visual Studio solution that contains all the source code for the rules.

Sample Rule Project Overview **118**

BranchSecurity **119**

What This Example Shows 119

Use in Prophet 21 119

DLL File 119

CheckForRequired **119**

What This Example Shows 119

Use in Prophet 21 120

DLL File 120

ConvertToCaps **120**

What This Example Shows 120

Use in Prophet 21 120

DLL File 120

CountryHarmonizer **120**

What This Example Shows 121

Use in Prophet 21 121

DLL File 121

CountryValidator **121**

What This Example Shows 121

Use in Prophet 21 121

DLL File 121

NoFreightOnCollect 122

What This Example Shows 122

Use in Prophet 21 122

DLL File 122

PhoneNumberFormatterRule 122

What This Example Shows 122

Use in Prophet 21 122

DLL File 123

PopupServiceTest 123

What This Example Shows 123

Use in Prophet 21 123

DLL File 123

RestrictedItemList 123

What This Example Shows 123

Use in Prophet 21 124

DLL File 124

ZipCodeLookupYahooRule 124

What This Example Shows 124

Use in Prophet 21 124

DLL File 124

OrderCreditCheck 125

What This Example Shows 125

Use in Prophet 21 125

DLL File 125

OrderLineCreditCheck 125

What This Example Shows 125

Use in Prophet 21 125

DLL File 126

OrderUpdatedEvent 126

What This Example Shows 126

Use in Prophet 21 126

DLL File 126

AddNewRowForDiscount 126

What This Example Shows 126

Use in Prophet 21 127

DLL File 127

FormDatastream_SortInvoiceLines 127

What This Example Shows 127

Use in Prophet 21 127

DLL File 127

FormDatastream_AddGroup 128

What This Example Shows 128

Use in Prophet 21 128

DLL File 128

SampleRule1 128

What This Example Shows 128

Use in Prophet 21 128

DLL File 128

OEAddDiscount 129

What This Example Shows 129

Use in Prophet 21 129

DLL File 129

Sample Rule Project Overview

The DynaChange Rules samples source code for non-web visual rules (in the form of a zipped Visual Studio solution) is available on the Customer Web Site (CWS). Two .zip files on the Customer Website contain the samples: Activant.P21.Extensions.Examples.zip and P21.Extensions.Examples.zip. You can find the zip files on the CWS using the following path: Support > File Center > Downloads > 12.x Downloads > Additional Downloads and Updates > Dynachange Rules. Download the files to the PC where you have Visual Studio installed.

When you unzip these files, each creates a folder that contains the solution. If you have Visual Studio 2008 (or Visual C# Express) or better installed, double-click the file **Activant.P21.Extensions.Examples.sln** (for legacy rule examples) or **P21.Extensions.Examples.sln** (for multi-row and other rules based on the P21.Extensions DLL file) to open the solution in Visual Studio.

> If you want to compile the practice rules (for example, after you make changes to them), you must ensure that you set the properties of the project to .NET Framework 3.5, and you must ensure that Activant.P21.Extensions.dll or P21.Extensions.dll is added to the project as a reference. See "Create a Visual Studio Project" on page 22 for more information.

If you are looking for the DLL file that contains pre-compiled sample rules, look in your Prophet 21 application folder for the *Activant.P21.Extensions.Examples.dll* or *P21.Extensions.Examples.dll* file, and copy it into your DLL folder for business rules.

For web visual rules, you can get a copy of the sample rules which is installed when you install or update your Middleware server.

BranchSecurity

The BranchSecurity rule ensures that the current user has permission to view data at the branch associated with the location passed to the rule.

What This Example Shows

This example shows how to connect to the Prophet 21 database, execute a query, and validate based on the query results.

> Before this rule will work, you need to create the users_x_branch table. You can find the SQL statement that creates the table properly commented at the bottom of the rule's code.
>
> You must replace the values in the string variable "sConn" with the connection information for your database (the correct server and database names).

Use in Prophet 21

Associate this rule with location fields. If the user enters a location, and the location is associated with a branch to which the user does not have access, prevent the user from retrieving data.

DLL File

Activant.P21.Extensions.dll

CheckForRequired

CheckForRequired is a generic rule that ensures every field you send to it is populated. If any fields are not populated, it returns an error message.

What This Example Shows

This example shows how to write a rule in a generic manner so that it does not matter what fields you pass to it. It also shows how to loop through all the fields you send using a foreach loop, and how to send a custom message back to Prophet 21.

Use in Prophet 21

Any time you want to ensure that certain data fields are populated, you can use this rule to ensure they are. You can also modify this rule to do something other than simply fail validation for empty fields.

DLL File

Activant.P21.Extensions.dll

ConvertToCaps

ConvertToCaps converts the letters in any field passed to it into capital letters.

What This Example Shows

The example shows how to loop through all the fields passed to the rule and perform an action on each one.

> Prophet 21 treats all the fields sent as text fields, so you can use string operations on numerals (like this rule could if you pass a numeric field) without generating errors. However, this means that you must cast all numerical values to a numerical data type if you want to perform mathematical operations on them.

Use in Prophet 21

You can use this for fields you may want to always use capital letters (for example, you want the country field to always use capital letters).

DLL File

Activant.P21.Extensions.dll

CountryHarmonizer

The CountryHarmonizer enables you to standardize the name of the USA so that you do not have USA, US, U.S.A., United States, etc. all mixed in your system.

What This Example Shows

CountryHarmonizer shows how to use GetFieldByAlias to create a rule that is "generic" so even if two different country fields have different names, you can still use the same rule.

Use in Prophet 21

Use this rule in the system if you want any form of "United States" to convert to "USA." You can modify the code in the rule to work for other countries, or change USA to United States. You must assign the country field an alias that matches the field name specified in the GetFieldByAlias method in the rule's code.

DLL File

Activant.P21.Extensions.dll

CountryValidator

The CountryValidator ensures that any company a user enters in a mail_company field is on a list of valid countries, and shows a message if the country is not valid.

What This Example Shows

The CountryValidator rule shows how to pass a custom message back to Prophet 21 for it to display.

> While the rule in this case only passes a message back if the rule fails the validation, you can send a message back to display even if the rule is successful.

Use in Prophet 21

In some cases, you may want to prevent salesreps from entering, for example, customers which do not have a mailing presence in your country. You can prevent them from entering any country you have not specified in the rule as valid.

DLL File

Activant.P21.Extensions.dll

NoFreightOnCollect

The NoFreightOnCollect rule sets the freight code to a code that uses free freight if the carrier is a collect carrier (based on the name you define for the carrier).

What This Example Shows

NoFreightOnCollect shows how you can combine naming conventions in Prophet 21 with parsing that name in a rule to take action.

> The rule searches for the string "collect" inside the carrier name, and then specifies a freight code to use.

Use in Prophet 21

You must establish a naming convention for collect carriers in Prophet 21, and then modify the rule to look for the correct string (if something other than "collect") and also specify the correct freight code for it to use. You can also modify this rule to work for any number of other situations where a naming convention is used to drive some change to another field.

DLL File

Activant.P21.Extensions.dll

PhoneNumberFormatterRule

PhoneNumberFormatterRule formats a 10-digit phone number to a specific format, so that all phone numbers have a consistent format.

What This Example Shows

PhoneNumberFormatterRule shows how to use GetFieldByAlias to create a rule that is "generic" so even if two different phone number fields have different names, you can still use the same rule. It also shows how to format text in a field using regular expressions.

Use in Prophet 21

You can apply this rule to any phone number field, and it will format the number. You must assign the phone number an alias that matches the field name specified in the GetFieldByAlias method in the

rule's code.

DLL File

Activant.P21.Extensions.dll

PopupServiceTest

The PopupServiceTest rule opens a .NET Winform and displays the Find Customer popup, which then sets the return value from the popup as the label text on the window.

What This Example Shows

PopupServiceTest shows how to invoke a Prophet 21 popup from your own .NET window. It passes a reference to the RulePopupService from the rule class to the Winform so that it can be used in the window. It then takes the return value from the popup and applies it to the label on the window.

Use in Prophet 21

Use this rule any time you want to use an existing Prophet 21 popup within the context of your own forms or fields.

DLL File

P21.Extensions.Examples.dll

RestrictedItemList

The RestrictedItemList rule enables you to restrict items so that only specified customers can buy them.

What This Example Shows

This example shows how to connect to the Prophet 21 database, execute a query, and validate based on the query results.

> Before this rule will work, you need to create and populate the restricted_item and restricted_item_customer_list tables. You can find the SQL statement that properly creates the table commented at the bottom of the rule's code.

You must replace the values in the string variable "sConn" with the connection information for your database (the correct server and database names).

Use in Prophet 21

Associate this rule with the Item ID field in Order Entry, Front Counter Order Entry, etc. to prevent Prophet 21 from adding items to an order if the item is on the restricted_items tab, and the customer / restricted item combination does not exist in the restricted_item_customer_list table, then Prophet 21 will not allow the user to add the item.

DLL File

Activant.P21.Extensions.dll

ZipCodeLookupYahooRule

ZipCodeLookupYahooRule enables you to populate the city and state automatically when you enter a ZIP code in Prophet 21.

What This Example Shows

ZipCodeLookupYahooRule shows how to connect to a third party web service and use the results to populate data in Prophet 21. It also shows the use of the GetFieldByAlias method.

There is a URL for the Yahoo!® web service embedded in the rule's code. This URL requires you to use an application ID from Yahoo!'s Developer Network. The application ID is available for free, but you must sign up for it at the Yahoo! Developer Network. Alternatively, you could use any other web service that provides city and state in exchange for ZIP code, but if you do, you must make significant changes to the business rule's code to make it work.

Use in Prophet 21

Link this rule to any postal code field in Prophet 21 to populate the city and state. You must assign aliases that match the values the rule uses in GetFieldByAlias to the city, state, and postal code fields.

DLL File

Activant.P21.Extensions.dll

OrderCreditCheck

The OrderCreditCheck rule makes sure that orders for $1,000 or more are not automatically allocated and processed. The rule uses the sum of the extended prices of the order to calculate the order cost.

What This Example Shows

OrderCreditCheck shows how to process multi-row rules that are created using the P21.Extensions DLL. This sample includes accessing and setting data for multi-row rule processing, accessing session information, determining the order in which updates will occur upon return, and setting focus after rule execution.

Use in Prophet 21

Use this rule when you do not want orders over a specific dollar amount to process automatically. A flat dollar amount is used in this example to show the functionality; you can modify this rule to use different criteria to trigger putting the order lines on hold.

DLL File

P21.Extensions.dll

OrderLineCreditCheck

The OrderLineCreditCheck rule makes sure that order lines for $1,000 or more are not automatically allocated and processed. Each line is evaluated based on its extended price.

What This Example Shows

OrderLineCreditCheck shows how to process single row rules that are created using the P21.Extensions DLL. This sample includes accessing session information, determining the order in which updates will occur upon return, and setting focus after rule execution.

Use in Prophet 21

Use this rule any time when you do not want order lines over a specific amount to process automatically. A flat dollar amount is used in this example to show the functionality; you can modify this rule to use different criteria to trigger putting the order line on hold.

DLL File

P21.Extensions.dll

OrderUpdatedEvent

The OrderUpdatedEvent rule drops a text file called OrderUpdatedEventRule.txt in the C:\temp directory any time a sales order is updated in Prophet 21.

What This Example Shows

OrderUpdatedEvent shows how to process non-visual event-driven based business rules that are created using the P21.Extensions.DLL. It accesses data the same way as multi-row rules using a .NET DataSet.

It also provides an example of the data that will be available by a business rule for post save (<xxx>Updated) event type business rules in Prophet 21. The data available will be key_value (in this case order number) and action (ADD when the order was created, UPDATE when the order was modified). Also note that these events are triggered after Prophet 21 has committed its save to the database so additional information can be retrieved from the database.

Use in Prophet 21

Use this rule any time you have workflow processing that needs to be done after an order is updated by Prophet 21.

DLL File

P21.Extensions.dll

AddNewRowForDiscount

The AddNewRowsForDiscount rule allows you to specify a flat discount percentage and then adds an other charge item to the order to reflect the discount.

What This Example Shows

AddNewRowForDiscount shows how to add a new row to a Prophet 21 window with the multi-row rule created using the P21.Extensions.DLL. It uses the AddNewRow method found in the Data object

available as of Prophet 21 version 12.16.

Use in Prophet 21

Use this rule any time you want to add an other charge item to an order to reflect a flat discount amount. It uses a specific other charge item id to show the functionality; you can modify this rule to use a different other charge item id.

DLL File

P21.Extensions.Examples.dll

FormDatastream_SortInvoiceLines

The FormDatastream_SortInvoiceLines rule defines the sort order in which invoice lines appear when printed on an invoice form.

What This Example Shows

FormsDatastream_SortInvoiceLines shows how to access the Prophet 21 database from a rule triggered on the Form Datastream Created event. It then retrieves a user-defined field from the database, appends that information to the datastream, and then sorts transaction lines based on the user-defined field's value. This can be performed for any form printed from Prophet 21.

P21SqlConnect is not available for this event type.

Use in Prophet 21

Use this rule any time you want to query additional information in the database and either add that information to a form, modify an existing field already being output on a form, or when you want to sort lines on a form in a certain order.

DLL File

P21.Extensions.Examples.dll

FormDatastream_AddGroup

The FormDatastream_AddGroups rule shorts the order that invoice lines appear when printed on an invoice form.

What This Example Shows

FormDatastream_AddGroup shows how to add a detail group to the header and line levels in a form datastream. For the purpose of this example, the data added to the new group is hardcoded, but this information will likely come from your Prophet 21 database. This can be performed on any form printed from Prophet 21.

Use in Prophet 21

Use this rule any time you have detail information at the header or line level of a transaction that you want to include in a new group on a form printed from Prophet 21.

DLL File

P21.Extensions.Examples.dll

SampleRule1

The SampleRule1 rule is a simple web visual rule that enables you to edit address information.

What This Example Shows

SampleRule1 shows the basics of how to accept data from Prophet 21, work with it in a controller, and pass that to a view.

Use in Prophet 21

This is a test rule that is designed in introduce the concepts of web visual rules.

DLL File

Not applicable.

OEAddDiscount

The OEAddDiscount rule is a web visual rule that enables you to calculate and apply discounts in order entry.

What This Example Shows

OEAddDiscount shows how to add a database query to a web visual rule.

Use in Prophet 21

Use this rule any time you want to apply a discount on an order that updates the unit prices on the line, and can apply discounts by product group.

DLL File

Not applicable.

Appendix C: Special Cases

This chapter describes special cases for DynaChange rules and tips for writing these types of rules.

Cascading Rules **130**

Adding Rows to Multi-Row Rules **131**

Form Datastream Created Rule Event **132**

Pre-SQL Hooks **133**

Simulating Keystrokes **134**

Cascading Rules

Using the CascadeInProgress property found on the RuleSet case, you can determine whether or not your rule allows any other rules to trigger when it updates data. Using the following methods found in the DataCollection object, you can allow or disallow cascading. This set is at the individual field level.

- public void SetCascade(bool allow)
    - Set or clear the attribute that allows cascading of further rules on all fields
- public void SetCascade(String tableName, bool allow)
    - Set or clear the attribute that allows cascading of further rules on an entire table in a multi-row rule
- public void SetCascade(String tableName, String columnName, bool allow)
    - Set or clear the attribute that allows cascading of further rules on an entire column in a multi-row rule
- public void SetCascade(String tableName, String columnName, int rowID, bool allow)
    - Set or clear the attribute that allows cascading of further rules on a single field in a multi-row rule
- public void SetFieldCascade(String columnName, bool allow)

- Set or clear the attribute that allows cascading of further rules on a single field in a single row rule

Using this property, you can also determine whether or not the rule is triggered if its trigger field is updated by another rule. Use the following if clause to prevent the rule from being triggered:

```
 if(RuleState.CascaseInProgress = true)
{
   return rr;
}
```

If a rule updates multiple fields, the fields are updated in Prophet 21 according to the sequence defined in your rule code. In a cascading rule scenario where one rule triggers another, and both rules update multiple fields, all fields for the first rule update in their respective sequence before any fields for the cascaded rule update.

If you have multiple rules that access, update, and are triggered on fields that are common between them, you must carefully consider the update sequence of the fields. For example, if Rule A updates five different fields and the third field to be updated triggers Rule B, Rule B executes as soon its trigger field is updated, but the remaining two fields from Rule A update before any of the fields returned by Rule B are updated.

Because a rule executes immediately when its trigger field is changed, in the example above, Rule B will not have access to the latest values for the two fields that have yet to be updated from Rule A.

Adding Rows to Multi-Row Rules

Use the following methods found in the DataCollection object to add new rows to multi-row rules. Both methods will add the new row and return the associated DataRow object that was added to the DataSet.

Not all rules allow for the adding of new lines. Most non-event based rules support adding new lines if the available underlying data also supports it. Most event based rules do not support adding new lines. Currently the only exception is the Import

Preprocess event. The AddNewRow methods throw an exception if you attempt to add new rows in a rule that does not support them.

- **public DataRow AddNewRow(string tableName)** – Adds a new row base on the string name of the table
- **public DataRow AddNewRow(DataTable table)** – Adds a new row based on the actual DataTable object.

Adding new rows is a two-step process: You must first invoke one of the AddNew Row methods to return a DataRow object. Then set the fields in the row. The update sequence operates as it does with existing rows – it updates the fields in Prophet 21 in the sequence in which they are updated in the code.

Update all fields in a row together directly after the row is added.

Add new rows after all other existing fields in your data set update.

In cases where no data exists in the source Prophet 21 data window, if a blank row exists, it is sent to the rule. In this case, update the fields in the blank row instead of adding a new row. Check for this situation by looking for the condition where only one row exists in the table and it is blank.

Form Datastream Created Rule Event

The Form Datastream Created rule event allows you to intercept Prophet 21 datastreams as forms output from the system. This allows you to manipulate the contents of the datastream, add new columns, copy the datastream to create additional forms, modify the Crystal form, change the printer, and so on.

This event only applies if you use Crystal Forms. The event does not trigger when using Jetform Forms.

Use the properties found in the XMLDatastream object when working with Form Datastream Created events. The XMLDatastream.Document object is a .NET Linq to XML XDocument object that represents the datastream.

- Each datastream document contains a job (JOBXXXXDEF). The job node contains various information about the job including the form type, the number of copies to print, which Crystal form to use, and other information about the job itself.

- The job contains one or more forms (FORMXXXDEF) nodes. Each form contains a header (the HDRXXXXDEF) node, one or more lines (LINEXXXDEF nodes that do not contain a lineKey attribute), and a total (TOTALSXDEF) node.
- Each line can contain line groups, such as lot numbers and bins associated with an order line. Groups are not organized within the associated line nodes; rather they appear as LINEXXXDEF nodes in the XML document, identified by the lineKey attribute.

  For easy access to line groups, use the public IEnumerable<XElement> GetGroup(string groupName, XElement fromElement) helper method.

  You must have the Linq to XML XElement object that represents the line from which you want to get the group. Further, you must know the name of the group definition node for which you are looking. For example, lot numbers for order lines are stored in nodes called LOTXXXXDEF.
- The public void AddGroup(XElement groupToAdd, XElement addToElement) helper method enables you to add your own groups to forms by adding all of the appropriate keys, and nesting the group in the appropriate place in the XML document. Use this method to add line groups, as well as for to add groups to other sections of the document.
- Use the Sort helper methods in the XMLDatastream to sort lines or line groups on a form.

Rules created from the Form Datastream Created event must be triggered from outside of the Prophet 21 application. As such, the P21SQLConnection is not available for the rule.

Pre-SQL Hooks

Use Pre-SQL hooks to trigger functionality when Prophet 21 runs specific SQL statements or to modify SQL statements executed by Prophet 21. As suggested by the name, these rules run immediately before the SQL statement runs on the database. You can only assign one rule to a SQL statement.

Using Pre-SQL hooks to modify SQL statements should only be used by advanced users. While you can change the entire SQL statement by the rule, many changes have the potential to cause problems with the functionality of Prophet 21. You should only modify:

The WHERE clause of a SELECT statement

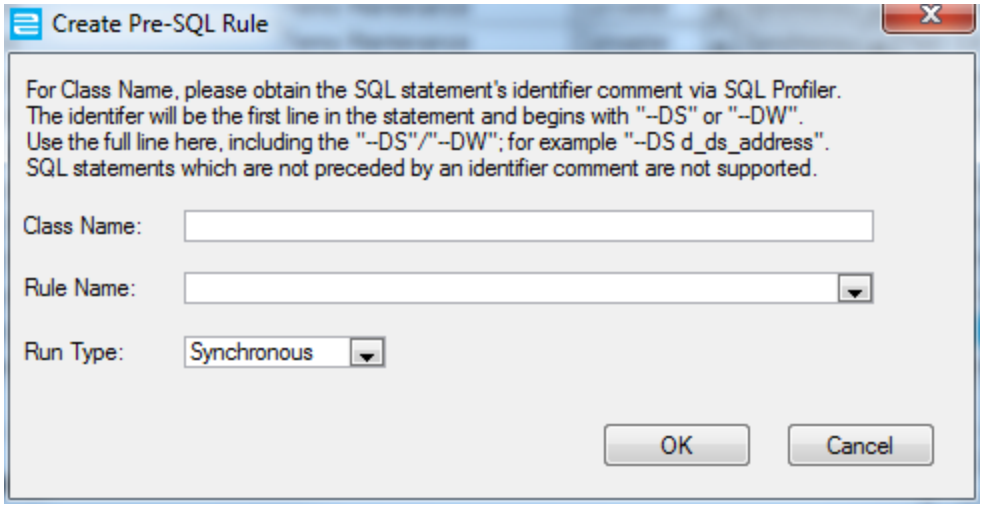
The values inserted or updated in the INSERT or UPDATE statement

1. Open SQL Server Profiler and run it against your Prophet 21 database.
2. Open Prophet 21.
3. Navigate to the window and tab where you want to link your business rule.

4. Run the process that triggers the SQL statement that you want to modify or use as a trigger.
5. SQL Server Profiler returns all SQL statements triggered by the process in Prophet 21. Locate the SQL statement and its identifier comment.

> The identifier comment is the first line of the SQL statement and begins with --DS or --DW followed by the name that corresponds with the data container class that the SQL statement is tied to in Prophet 21.

6. In Prophet 21, from the Tools menu, select DynaChange > DynaChange Rules > **Organizer**.
7. Click **Create Pre-SQL Rule**. The Create Pre-SQL Rule window opens.



   a. In **Class Name**, enter the identifier comment from the SQL statement. Remember that the identifier comment begins with --DS or --DW.
   b. In **Rule Name**, select the DynaChange rule to run.
   c. In **Run Type**, select Synchronous.
   d. Click **OK** to return to the Business Rule Organizer.
8. Pre-SQL rules are automatically assigned to all users. Update the permissions as needed.
9. Click **Apply Change**, and then close the Business Rule Organizer.

Simulating Keystrokes

Use the Keystroke property on the RuleResult object to direct Prophet 21 to simulate a keystroke upon completion of the rule. This is currently only implemented for On-Demand type rules and does not support letter keys (A-Z). You can use key modifiers to form key combinations. The keystroke simulation functionality works on both the desktop application as well as WWMS to simulate F-key functions.

The following keys are supported to use through the keystroke property: Number keys (**0-9**), F-Keys (**F1 – F12**), **Enter**, **Tab**, **Home**, **End**, **PgDn**, **PgUp**, **Delete**, **Backspace**, **Esc**, and the **Space** key.

You can also use the modifiers, **Ctrl**, **Shift**, and **Alt**. Modifiers and keys should be separated by a plus (+) sign with the modifier always coming first and the key value coming after the plus. Only one modifier can be applied at a time. For example, you can use **Shift+F2**, but cannot use **Ctrl+Alt+F2**.

Not all combinations of setting data and simulating keystrokes will work. So, when attempting to set data and simulate a keystroke from the same rule, you should test the rule thoroughly to ensure that the expected results are achieved.